

第一章 微信小程序环境搭建

申请账号

开发小程序的第一步，你需要拥有一个小程序账号，通过这个账号你就可以管理你的小程序。

进入[小程序注册页](#) 根据指引填写信息和提交相应的资料，就可以拥有自己的小程序账号。

小程序注册页: <https://mp.weixin.qq.com/wxopen/waregister?action=step1>

小程序注册

① 账号信息 — ② 邮箱激活 — ③ 信息登记

每个邮箱仅能申请一个小程序

邮箱

作为登录账号，请填写未被微信公众平台注册，未被微信开放平台注册，未被个人微信号绑定的邮箱

密码

字母、数字或者英文符号，最短8位，区分大小写

确认密码

请再次输入密码

验证码

 换一张

☐ 你已阅读并同意《微信公众平台服务协议》、《微信小程序平台服务条款》、《微信公众平台个人信息保护指引》

注册

主体类型选择：

- 个人：（学习可选择个人）不支持微信支付和高级接口能力，每年要30元认证费，账号可获得“被搜索”和“被分享”能力。未完成微信认证不影响后续版本发布
- 个体工商户或企业等：才支持微信支付和高级接口能力，每年要300元认证费。

注册国家/地区
中国大陆

主体类型

如何选择主体类型？

个人不支持微信支付，个体工商户或公司才支持微信支付

个人

企业

政府

媒体

其他组织

个人类型包括：由自然人注册和运营的公众账号。

账号能力：个人类型暂不支持微信认证、微信支付及高级接口能力。

注册成功后，登录 [小程序管理后台](#)，你可以管理你的小程序的权限，查看数据报表，发布小程序等操作。

在管理后台的首页，可以完善下面信息：

点击菜单“管理”-“开发管理”，找到“开发设置”看到小程序的 AppID 了，AppID 相当于小程序平台的一个身份证，后续你会在很多地方要用到 AppID。

有了小程序账号之后，我们需要一个工具来开发小程序。

安装开发工具

前往 [开发者工具下载页面](#)，根据自己的操作系统下载对应的安装包进行安装。

开发者工具下载页面 <https://developers.weixin.qq.com/miniprogram/dev/devtools/download.html>

课程配套软件位于：04-配套软件 > IDE > 根据自己电脑操作系统选择对应版本进行安装



下载安装成功后，打开小程序开发者工具，用微信扫码登录开发者工具，准备开发你的第一个小程序吧！

如果打开 微信开发者工具 有问题，则单击右键以管理员身份运行 微信开发者工具



创建微信小程序

创建微信小程序项目，如下图点击 +



小程序

小程序

Q 搜索

导入 管理



小游戏



多端应用



代码片段



公众号网页



其他



在创建小程序页面填入信息：

- 项目名称 `mxg-miniprogram-core`
- 目录选择：目录中最后一级为项目名称；目录名不要有中文、空格和特殊字符；
- AppID：选择自己注册的小程序 AppID
- 后端服务：选择 不使用云服务
- 模板选择： `JS-基础模板`

Q 输入项目名称

小程序项目

小程序

小游戏

代码片段

公众号网页项目

其他

创建小程序

项目名称

mxg-miniprogram-core

目录

02-projectCode/mxg-miniprogram/mxg-miniprogram-core

AppID

wx4...

注册 或使用 测试号 ?

开发模式

小程序

后端服务

☐ 微信云开发

☒ 不使用云服务

模板选择

全部来源

全部分类

TS-基础模版
官方

JS-Skyline 基础模版
官方

JS-基础模版
官方

17:28

80%

Weixin

Hello World

注销 >

取消

确定

创建后项目目录结果如下：

如果创建后结构不一样，可在开发工具中导入梦学谷提供模板代码，

位于：03-配套资料》模板代码》mxg-miniprogram-core

资源管理器: MXG-...

- pages
 - index
 - index.js
 - index.json
 - index.wxml
 - index.wxss
 - logs
 - logs.js
 - logs.json
 - logs.wxml
 - logs.wxss
 - utils
 - util.js
 - .eslinttrc.js
 - app.js
 - app.json
 - app.wxss
 - project.config.json
 - project.private.config.json
 - sitemap.json

微信小程序项目结构

- pages 下存放小程序所有页面文件，每个小程序页面四个文件组成，分别是：

注意：为了方便开发者减少配置项，四个页面文件所在的目录名，要和页面文件名相同

文件类型	必需	作用
js	是	页面逻辑
wxml	是	页面结构
json	否	页面配置，对本页面的表现进行配置，如：此页面的状态栏/导航栏效果等配置
wxss	否	页面样式表

- 一个小程序主体部分由三个文件组成，必须放在项目的根目录，如下：

文件	必需	作用
app.js	是	小程序逻辑，如：初始化操作，定义全局变量、方法等
app.json	是	小程序全局配置文件，用于配置小程序的页面路径和状态栏、导航栏等全局属性
app.wxss	否	小程序公共样式表 CSS

- utils 目录是用于存放通用的 js 工具文件，非必要的。
- .eslintrc.js 文件，关于 Eslint 的配置。Eslint 可以帮助我们查找并修复 JavaScript 代码中的问题。
Eslint 的配置参考：<https://eslint.nodejs.cn/docs/latest/use/configure/>
- project.config.json 和 project.private.config.json 项目配置文件，配置项目和开发者的信息
参考：<https://developers.weixin.qq.com/miniprogram/dev/devtools/projectconfig.html>
 - project.config.json 用于配置项目的公共配置，多名开发人员共享配置。
 - project.private.config.json 用于配置项目的个人配置，不同开发人员可进行个性化配置。在 .gitignore 中忽略 git 控制。
- sitemap.json 配置小程序页面是否允许被微信索引。当开发者允许微信索引时，微信会通过爬虫的形式，为小程序的页面内容建立索引。当用户的搜索词条触发该索引时，小程序的页面将可能展示在搜索结果中。

如果项目未配置 `sitemap.json` 文件，默认所有页面都能被索引

参考: <https://developers.weixin.qq.com/miniprogram/dev/framework/sitemap.html>

微信开发者工具快捷键

快捷键	说明
Ctrl+S	保存并编译项目
Ctrl+B	编译项目
Ctrl+A	全选
Ctrl+C	复制选中的内容
Ctrl+V	粘贴已复制的内容
Ctrl+X	剪切选中的内容
Ctrl+Z	撤销当前编辑
Ctrl+/	添加单行注释
Shift+Alt+A	添加多行注释
Ctrl+Shift+K	删除当前行
Ctrl+F	当前文件内搜索
Ctrl+Shift+F	全局搜索
Ctrl+H	当前文件内搜索与替换
Ctrl+Shift+H	全局搜索与替换
Alt+↑	将当前行向上移动一行
Alt+↓	将当前行向下移动一行
Shift+Alt+↑	向上复制当前行
Shift+Alt+↓	向下复制当前行
Shift+Alt+F	格式化代码

第二章 小程序核心配置文件

注意：json文件中为一个JSON 对象，不支持写注释，也不可以多有任何一个逗号、括号等

app.json 全局配置文件

官网参考：<https://developers.weixin.qq.com/miniprogram/dev/reference/configuration/app.html>

在小程序根目录下的 `app.json` 文件，用来对微信小程序进行全局配置。

文件内容为一个JSON 对象，有以下配置项：

属性	类型	必填	描述	最低版本
entryPagePath	string	否	小程序默认启动首页	
pages	string[]	是	页面路径列表	
window	Object	否	全局的默认窗口配置	
tabBar	Object	否	底部 <code>tab</code> 栏的配置	
networkTimeout	Object	否	各类网络请求的超时时间，单位均为毫秒	
debug	boolean	否	是否开启 debug 模式，默认关闭	
subpackages	Object[]	否	分包结构配置，写成 <code>subPackages</code> 也支持，参考 使用分包	1.7.3
preloadRule	Object	否	分包预下载规则	2.3.0
requiredBackgroundModes	string[]	否	需要在后台使用的能力，可选 <code>audio</code> ：后台音乐播放， <code>location</code> ：后台定位	
requiredPrivateInfos	string[]	否	调用的地理位置相关隐私接口，且要在小程序管理后台，「开发」-「开发管理」-「接口设置」中自助开通该接口权限，如未开通，将在代码提审环节进行拦截。	
permission	Object	否	小程序接口权限相关设置，如：获取用户位置权限	微信客户端 7.0.0
resizable	boolean	否	PC 小程序是否支持用户任意改变窗口大小（包括最大化窗口）；iPad 小程序是否支持屏幕旋转。默认关闭	2.3.0
usingComponents	Object	否	全局 自定义组件 配置	开发者工具 1.02.1810190
sitemapLocation	string	是	指明 <code>sitemap.json</code> 的位置（ <code>sitemap.json</code> 文件用来配置小程序及其页面是否允许被微信索引）	
style	string	否	启用新版的组件样式，配置 <code>"style": "v2"</code> 启用新版的组件样式	2.8.0
useExtendedLib	Object	否	指定需要引用的扩展库，可选 <code>kbone</code> ： 多端开发框架 ， <code>weui</code> ： WeUI 组件库	2.2.1
resolveAlias	Object	否	用来自定义模块路径的映射规则，如 <code>resolveAlias: {"@/*": "/*"}</code>	
renderer	string	否	全局默认的渲染后端，可选 <code>webview</code> （默认）， skyline	2.30.4
rendererOptions	Object	否	渲染后端选项，主要针对Skyline 渲染引擎的 相关配置项	2.31.1
componentFramework	string	否	组件框架，配置项： glass-easel	2.30.4
<code>static</code>	Object	否	正常情况下默认所有资源文件都被打包发布到所有平台，可以通过 <code>static</code> 字段配置特定每个目录/文件只能发布到特定的平台(多端场景) 相关文档	
<code>convertRpxToVw</code>	boolean	否	配置是否将 rpx 单位转换为 vw 单位，开启后能修复某些 rpx 下的精度问题	3.3.0

pages/entryPagePath

`pages` 属性的值是一个字符串数组：用于指定小程序由哪些页面组成，每一项都对应一个页面的 路径（含文件名）信息。

配置项中的文件名不需要写文件后缀，框架会自动去寻找对应位置的 `.json`、`.js`、`.wxml`、`.wxss` 四个文件进行处理。

- **app.json 配置示例：**

```
1 {
2   "pages": [
3     "pages/index/index",
4     "pages/shopping/shopping",
5     "pages/order-list/order-list",
6     "pages/user/user"
7   ]
8 }
```

快速创建页面：在 `pages` 中添加一个新的页面路径，保存配置后，IDE会自动创建对应目录和文件。

生成的目录结构为：

```
1 |— pages
2 |   |— index
3 |   |   |— index.wxml
4 |   |   |— index.js
5 |   |   |— index.json
6 |   |   |— index.wxss
7 |   |— shopping
8 |   |   |— shopping.wxml
9 |   |   |— shopping.js
10 |   |   |— shopping.json
11 |   |   |— shopping.wxss
12 |   |— order-list
13 |   |   |— order-list.wxml
14 |   |   |— order-list.js
15 |   |   |— order-list.json
16 |   |   |— order-list.wxss
17 |   |— user
18 |   |   |— user.wxml
19 |   |   |— user.js
20 |   |   |— user.json
21 |   |   |— user.wxss
22 |— app.js
23 |— app.json
24 |— app.wxss
```

- **注意事项：**

- 文件名不需要写文件后缀，框架会自动去寻找对应位置的 `.json`, `.js`, `.wxml`, `.wxss` 四个文件进行处理。
- 数组的第一项代表小程序的初始页面（首页），但是如果指定了 `entryPagePath`，则以 `entryPagePath` 指定的页面为首页。
- 小程序中新增/减少页面，都需要对 `pages` 数组进行修改。
- 最后不要有多余的逗号。错误写法：`"pages": [ "pages/index/index", ]`

tabBar

如果小程序是一个多 tab 应用（客户端窗口的底部或顶部有 tab 栏可以切换页面），可以通过 `tabBar` 配置项指定 tab 栏的表现，以及 tab 切换时显示的对应页面。

- **app.json配置示例：**

图片文件位于：`03-配套资料/assets/tab` 目录，复制 `assets` 目录粘贴到项目根目录下

```
1  {
2    "tabBar": {
3      "color": "#8a8a8a",
4      "selectedColor": "#e40030",
5      "backgroundColor": "#ffffff",
6      "borderStyle": "white",
7      "list": [
8        {
9          "pagePath": "pages/index/index",
10         "text": "首页",
11         "iconPath": "assets/tab/index.png",
12         "selectedIconPath": "assets/tab/index-active.png"
13       },
14       {
15         "pagePath": "pages/shopping/shopping",
16         "text": "点单",
17         "iconPath": "assets/tab/shopping.png",
18         "selectedIconPath": "assets/tab/shopping-active.png"
19       },
20       {
21         "pagePath": "pages/order-list/order-list",
22         "text": "订单",
23         "iconPath": "assets/tab/order.png",
24         "selectedIconPath": "assets/tab/order-active.png"
25       }
26     ]
27   },
28 }
```

• tabBar 配置项说明：

属性	类型	必填	默认值	描述	最低版本
color	HexColor	是		tab 上的文字默认颜色，仅支持十六进制颜色	
selectedColor	HexColor	是		tab 上的文字选中时的颜色，仅支持十六进制颜色	
backgroundColor	HexColor	是		tab 的背景色，仅支持十六进制颜色	
list	Array	是		tab 的列表，详见 <code>list</code> 属性说明，最少 2 个、最多 5 个 tab	
borderStyle	string	否	black	tabbar 上边框的颜色，仅支持 <code>black</code> / <code>white</code>	
position	string	否	bottom	tabBar 的位置，仅支持 <code>bottom</code> / <code>top</code>	
custom	boolean	否	false	自定义 tabBar，见 详情	2.5.0

其中 list 接受一个数组，只能配置最少 2 个、最多 5 个 tab。tab 按数组的顺序排序，每个项都是一个对象，其属性值如下：

属性	类型	必填	说明
pagePath	string	是	页面路径，必须在 <code>pages</code> 中先定义
text	string	是	tab 上按钮文字
iconPath	string	否	图片路径，icon 大小限制为 40kb，建议尺寸为 81px * 81px，不支持网络图片。当 <code>position</code> 为 <code>top</code> 时，不显示 icon。
selectedIconPath	string	否	选中时的图片路径，icon 大小限制为 40kb，建议尺寸为 81px * 81px，不支持网络图片。当 <code>position</code> 为 <code>top</code> 时，不显示 icon。

自定义tabBar：<https://developers.weixin.qq.com/miniprogram/dev/framework/ability/custom-tabbar.html>

window

用于设置小程序的状态栏、导航条、标题、窗口背景色等。

参考：<https://developers.weixin.qq.com/miniprogram/dev/reference/configuration/app.html#window>

• app.json 配置示例：

```

1  {
2    "window": {
3      "navigationBarTitleText": "梦学谷配置演示", // 导航栏标题文字内容
4      "navigationBarBackgroundColor": "#ffffff", // 导航栏背景颜色

```



```

5      "navigationBarTextStyle": "black", // 导航栏和状态栏颜色，仅支持 `black` /
      `white`
6      "navigationStyle": "default", // default采用window配置的导航栏，custom自定义导
      航栏
7      "enablePullDownRefresh": false, // 是否开启下拉刷新
8      "backgroundTextStyle": "light", // 下拉刷新的加载中样式，仅支持 `dark` / `light`
9      "backgroundColor": "#e40000", // ios上下回弹背景色，开启下拉刷新后也是下拉背景色
10     "backgroundColorTop": "#888888", // ios顶部回弹色，会覆盖backgroundColor配置
11     "backgroundColorBottom": "#000000", // ios底部回弹色，会覆盖backgroundColor配
      置
12     "pageOrientation": "auto" // 小程序在【手机】上支持屏幕旋转；在app.json根对象上配
      置"resizable": true 是开启ipad旋转，PC窗口大小任意调整。
13   }
14 }
  
```

• window 配置项说明：

属性	类型	默认值	描述	最低版本
navigationBarBackgroundColor	HexColor	#000000	导航栏背景颜色，如 #000000	
navigationBarTextStyle	string	white	导航栏标题、状态栏颜色，仅支持 black / white	
navigationBarTitleText	string		导航栏标题文字内容	
navigationStyle	string	default	导航栏样式，仅支持以下值： default 默认样式 custom 自定义导航栏，只保留右上角胶囊按钮。参见 注 2。	iOS/Android 微信客户端 6.6.0, Windows 微信客户端 不支持
homeButton	boolean	false	在非首页、非页面栈最底层页面或非tabbar内页面中的 导航栏展示home键	微信客户端 8.0.24
backgroundColor	HexColor	#ffffff	窗口的背景色	
backgroundTextStyle	string	dark	下拉 loading 的样式，仅支持 dark / light	
backgroundColorTop	string	#ffffff	顶部窗口的背景色，仅 iOS 支持	微信客户端 6.5.16
backgroundColorBottom	string	#ffffff	底部窗口的背景色，仅 iOS 支持	微信客户端 6.5.16
enablePullDownRefresh	boolean	false	是否开启全局的下拉刷新。详见 Page.onPullDownRefresh	
onReachBottomDistance	number	50	页面上拉触底事件触发时距页面底部距离，单位为 px。详见 Page.onReachBottom	
pageOrientation	string	portrait	屏幕旋转设置，支持 auto 随着屏幕旋转 / portrait 竖向 / landscape 横向 详见 响应显示区域变化	2.4.0 (auto) / 2.5.0 (landscape)
restartStrategy	string	homePage	重新启动策略配置	2.8.0
initialRenderingCache	string		页面 初始渲染缓存 配置，支持 static / dynamic	2.11.1
visualEffectInBackground	string	none	切入系统后台时，隐藏页面内容，保护用户隐私。支持 hidden / none	2.15.0
handleWebviewPreload	string	static	控制 预加载下个页面的时机 。支持 static / manual / auto	2.15.0

resolveAlias

1. 未配置 `resolveAlias` 时，在 `pages/index/index.js` 中导入 `utils/util.js` 时，默认需要指定相对路径才可导入成功：

```
1 // pages/index/index.js
2 // 使用相对路径导入util文件，比较麻烦
3 var common = require('../../utils/util')
4
5 // 下面这种会报错找不到文件
6 //var common = require('/utils/util')
```

2. 配置 `resolveAlias` 时，在 `app.json` 中添加 `resolveAlias` 配置项来进行路径匹配，如下：

```
1 {
2   "resolveAlias": {
3     "@/*": "/"
4   }
5 }
```

配置了之后，会对 `require` 里的模块路径，进行规则匹配，并映射成配置的路径

```
1 // var common = require('../../utils/util')
2
3 // 在项目根路径下的查找 utils/util.js
4 var common = require('@utils/util')
5
6 const date = common.formatTime(new Date())
7 console.log('date', date)
```

`resolveAlias` 注意：

1. `resolveAlias` 进行的是路径匹配，其中的 key 和 value 须以 `/*` 结尾。
2. 如果在 `project.config.json` 中指定了 `miniprogramRoot`，则 `/*` 指代的根目录是 `miniprogramRoot` 对应的路径，而不是开发者工具项目的根目录。如：`miniprogramRoot: miniprogram/`，则 `/*` 指代的根目录是 `miniprogram/*`

页面名.json 页面配置文件

`app.json` 中的部分配置，也支持对单个页面进行配置，可以在页面对应的 `.json` 文件来对本页面的表现进行配置。

页面中配置项在当前页面会覆盖 `app.json` 中相同的配置项，但是注意 `app.json` 中的 `window` 配置项，在这里不需要额外指定 `window` 属性，直接在根对象下指定 `window` 中的属性即可，如下：

```
1 // 在 app.json 中的window配置如下：
```

```
2 {
3   "window": {
4     "navigationBarBackgroundColor": "#ffffff", // 导航栏背景颜色
5     "navigationBarTextStyle": "black", // 导航栏和状态栏颜色，仅支持 `black` / `white`
6     "navigationBarTitleText": "梦学谷配置演示"
7   }
8 }
9
10 // 在user页面的 user.json 中不需要写window，如下：
11 {
12   "navigationBarBackgroundColor": "#000000", // 导航栏背景颜色
13   "navigationBarTextStyle": "white", // 导航栏和状态栏颜色，仅支持 `black` / `white`
14   "navigationBarTitleText": "用户页面"
15 }
```

注意：

- 并不是所有 `app.json` 中的配置都可以在页面覆盖或单独指定，仅限于官方文档中包含的选项，即如下：

属性	类型	默认值	描述	最低版本
navigationBarBackgroundColor	HexColor	#000000	导航栏背景颜色，如 #000000	
navigationBarTextStyle	string	white	导航栏标题、状态栏颜色，仅支持 black / white	
navigationBarTitleText	string		导航栏标题文字内容	
navigationStyle	string	default	导航栏样式，仅支持以下值：default 默认样式；custom 自定义导航栏，只保留右上角胶囊按钮。	iOS/Android 客户端 7.0.0 以下版本，navigationStyle 只在 app.json 中生效
homeButton	boolean	false	在非首页、非页面栈最底层页面或非tabbar内页面中的导航栏展示home键	微信客户端 8.0.24
backgroundColor	HexColor	#ffffff	窗口的背景色	
backgroundColorContent	HexColor	#RRGGBBAA	页面容器背景色， 点击查看设置背景色详情	
backgroundTextStyle	string	dark	下拉 loading 的样式，仅支持 dark / light	
backgroundColorTop	string	#ffffff	顶部窗口的背景色，仅 iOS 支持	微信客户端 6.5.16
backgroundColorBottom	string	#ffffff	底部窗口的背景色，仅 iOS 支持	微信客户端 6.5.16
enablePullDownRefresh	boolean	false	是否开启当前页面下拉刷新。详见 Page.onPullDownRefresh	
onReachBottomDistance	number	50	页面上拉触底事件触发时距页面底部距离，单位为 px。详见 Page.onReachBottom	
pageOrientation	string	portrait	屏幕旋转设置，支持 auto / portrait / landscape 详见 响应显示区域变化	2.4.0 (auto) / 2.5.0 (landscape)
disableScroll	boolean	false	设置为 true 则页面整体不能上下滚动。只在页面配置中有效，无法在 app.json 中设置	
usingComponents	Object	否	页面 自定义组件 配置	1.6.3
initialRenderingCache	string		页面 初始渲染缓存 配置，支持 static / dynamic	2.11.1
style	string	default	启用新版的组件样式	2.10.2
singlePage	Object	否	单页模式相关配置	2.12.0
restartStrategy	string	homePage	重新启动策略配置	2.8.0
handleWebviewPreload	string	static	控制 预加载下个页面的时机 。支持 static / manual / auto	2.15.0
visualEffectInBackground	string	否	切入系统后台时，隐藏页面内容，保护用户隐私。支持 hidden / none，若对页面单独设置则会覆盖全局的配置，详见 全局配置	2.15.0
enablePassiveEvent	Object或boolean	否	事件监听是否为 passive，若对页面单独设置则会覆盖全局的配置，详见 全局配置	2.24.1
renderer	string	否	渲染后端	2.30.4
rendererOptions	Object	否	渲染后端选项，详情 相关文档	3.1.0
componentFramework	string	否	组件框架，详情 相关文档	2.30.4

项目配置文件 project.[private.]config.json

项目配置文件说明

项目配置文件参考：<https://developers.weixin.qq.com/miniprogram/dev/devtools/projectconfig.html>

- `project.config.json` 用于配置项目的公共配置，多名开发人员共享配置。
- `project.private.config.json` 用于配置项目的个人配置，不同开发人员可进行个性化配置。

项目配置文件说明：

1. 项目根目录中的 `project.config.json` 和 `project.private.config.json` 文件可以对项目进行配置，
2. `project.private.config.json` 中的相同设置优先级高于 `project.config.json`
3. 可以在 `project.config.json` 文件中配置公共的配置，在 `project.private.config.json` 配置个人的配置，可以将 `project.private.config.json` 写到 `.gitignore` 避免版本管理的冲突。
4. `project.private.config.json` 中有的字段，开发者工具内的设置修改会优先覆盖 `project.config.json` 的内容。

比如：在 `project.private.config.json` 有 `appid` 字段，那么在 详情-基本信息 中修改了 `appid`，会写到 `project.private.config.json` 中，不会覆盖掉 `project.config.json` 的 `appid` 字段的内容

5. 开发阶段相关的设置修改优先同步到 `project.private.config.json` 中，但与最终编译产物有关的设置无法在 `project.private.config.json` 中生效，界面上的改动也不会同步到 `project.private.config.json` 文件中。

```
1 {
2   "miniprogramRoot": "miniprogram/", // 小程序源码的目录(需为相对路径)，就是`项目根目录`，
   添加后会自动添加一个相同值的srcMiniprogramRoot属性
3   "srcMiniprogramRoot": "miniprogram/", //配置miniprogramRoot，此会自动生成相同值的此属
   性
4   "cloudfunctionRoot": "cloudfunctions/", // 存放云函数的根目录(需为相对路径)
5   "projectname": "mxg-miniprogram-core", // 项目名称
6   "appid": "xxxxxxxxxx", //项目的appid，在【详情-基本信息】中可设置
7   "compileType": "miniprogram", // 编译类型: miniprogram 普通小程序项目，plugin 小程序插
   件项目
8   "description": "项目配置文件", // 描述信息
9   "libVersion": "3.5.5", // 指定项目运行的基础库具体的版本号,也可指定: trial 最新的基础库;
   widelyUsed使用比例最高的基础库; latest最新的非灰度中的基础库,灰度测试是指在产品正式发布之前，将
   产品先给一部分用户进行测试和试用；非灰度就是灰度测试后发布的最新版本
10  "editorSetting": { // 指定自动生成的文件的 tabIndent 和 tabSize
11    "tabIndent": "insertSpaces", // tab键按空格缩进
12    "tabSize": 2 // 一个Tab缩进两个空格键，可试着改为 4 看效果
13  },
14  "packOptions": { // 打包配置
15    "ignore": [], // 配置打包时需要强制带上的文件或文件夹
16    "include": [] // 配置打包时忽略的文件或文件夹
17  },
18  "setting": {
19    // 下面是对应开发工具的右上角【详情】本地设置】的勾选项
```

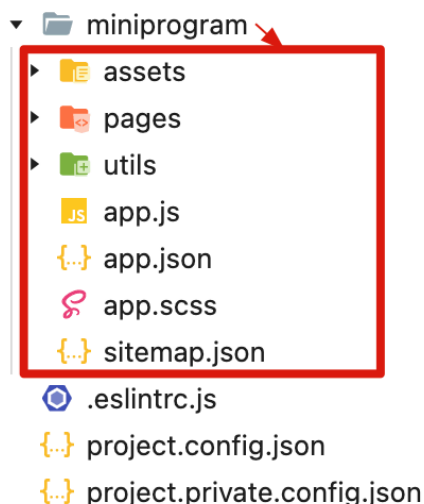
```
20  "condition": false, // 是否启用条件编译，启用它意义不大：因为当前只针对微信小程序开发，不
    会生成APP，参考：
    https://dev.weixin.qq.com/docs/framework/dev/framework/operation/condition-
    compile.html
21  "es6": true, // 是否启用 es6 转 es5 (需 enhance 同时为 true/false，对应于将JS编译成
    ES5)
22  "enhance": true, // 是否打开增强编译 (需 es6 同时为 true/false，对应于将JS编译成ES5)
23  "postcss": false, // 自动为CSS代码添加必要的运行环境前缀，以确保样式在各种运行环境中的兼容
    性
24  "minified": true, //上传代码时是否自动压缩脚本文件
25  "minifyWXSS": true, //上传代码时是否自动压缩样式文件
26  "minifyWXML": true, //上传代码时是否自动压缩WXML文件
27  "swc": false, // 使用 SWC 编译模式，SWC 是既可用于编译也可用于打包的一款工具，比Babel编
    译快
28  "uglifyFileName": false, // 上传时进行代码保护
29  "ignoreUploadUnusedFiles": true, // 上传时是否过滤无依赖文件
30  "autoAudits": false, // 是否自动运行体验评分
31  "urlCheck": false, // 是否检查安全域名和 TLS 版本
32  "compileHotReload": true, //是否开启文件保存后自动热重载，开启不用刷新下编辑，而是无感应
    的生效
33  "preloadBackgroundData": false, // 小程序加载时是否数据预拉取，参
    考:https://developers.weixin.qq.com/miniprogram/dev/framework/ability/pre-
    fetch.html
34  "lazyloadPlaceholderEnable": false, // 是否启用懒注入占位组件调试，首页优化方式，参考：
    https://developers.weixin.qq.com/miniprogram/dev/framework/custom-
    component/placeholder.html
35  "useStaticServer": false, //仅在小游戏项目有效，是否开启静态资源服务器
36  "bigPackageSizeSupport": false, //预览及真机调试的时主包、分包体积上限调整为4M (小程
    序)、8M (小游戏)
37  "skylineRenderEnable": false, // 是否开启 skyline 渲染调试
38  // 下面不在【本地设置】页面中勾选的，需要在此配置文件中编写配置
39  "useCompilerPlugins": [ // 编译插件配置，目前支持编译插件有 typescript、less、sass
40    "typescript", // ts语法支持
41    "sass" // sass语法编写css代码，将 .wxss 文件改为 .scss 即可
42  ],
43  }
44  }
```

项目根目录 miniprogramRoot

- project.config.json 配置了 "miniprogramRoot": "miniprogram/"，可将项目结构调整如下效果：

```
1  {
2    "miniprogramRoot": "miniprogram/",
3    "srcMiniprogramRoot": "miniprogram/" // 配置了 miniprogramRoot 保存后这个会自动生
    成，没有生成就自己手动加
4  }
```

资源管理器: MXG-... + [] ↺ ☰



ES6 转 SE5 兼容性

- 启用ES6 转 SE5 配置，要将 `es6` 和 `enhance` 两个同时开关才有效

```
1 {
2   "setting": {
3     "es6": true, // 是否启用 es6 转 es5 (需 enhance 同时为 true/false, 对应于将JS编译成ES5)
4     "enhance": true // 是否打开增强编译 (需 es6 同时为 true/false, 对应于将JS编译成ES5)
5   }
6 }
```

- 上面同时开启后，可使用 ES6 语法，

比如：使用 `export` 导出模块、`import` 导入模块

- 修改 `utils/util.js` 文件，使用 `export` 导出 `formatTime` 方法

```
1  /* es5 导出方法
2  module.exports = {
3    formatTime
4  }
5  */
6
7  // es6 使用 export 导出一个默认对象
8  export default {
9    formatTime
10 }
```

- 在 `pages/index/index.js` 中使用 `import` 导入 `utils/util.js` 的 `formatTime` 方法


```
1 // es5 require 导入
2 // var common = require('../utils/util')
3
4 // 在项目根路径下的查找 utils/util.js
5 // var common = require('@utils/util')
6
7
8 // es6 import 导入
9 import common from '@utils/util'
10
11 const date = common.formatTime(new Date())
12 console.log('date', date)
```

- 如果将 `es6` 和 `enhance` 两个同时设置为 `false`，则上面代码就会报错：

```
✖ SyntaxError: Unexpected token 'export'
    at loadScriptSync (getmainpackage.js:223)
    at getmainpackage.js:226
    at getmainpackage.js:284
    at d.loadScripts (index.js:1)
    (env: macOS,mp,1.06.2405020; lib: 3.5.5)

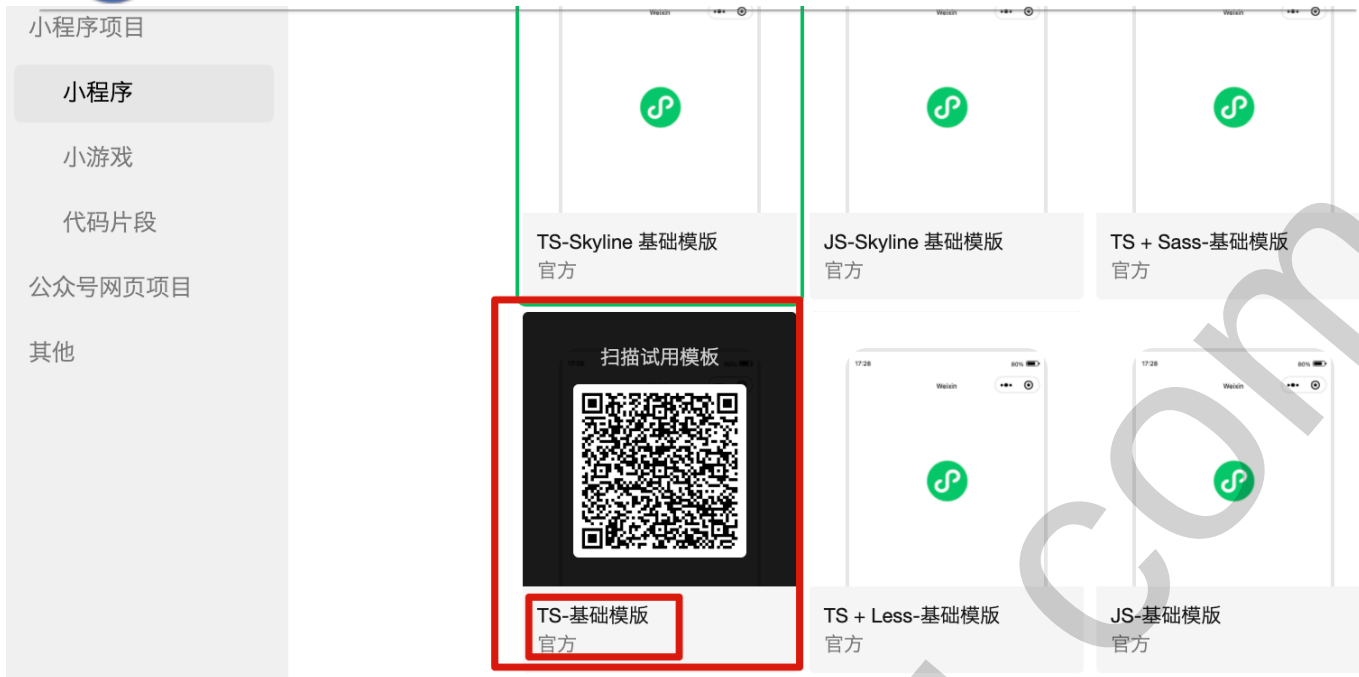
✖ app.js错误:
Error: module 'utils/util.js' is not defined, require args is './utils/util'
    at q (WASubContext.js?+we-600333028&v=3.5.5.1)
    at r (WASubContext.js?+we-600333028&v=3.5.5.1)
```

Sass 和 Typescript 支持

在项目配置文件中，使用 `useCompilerPlugins` 配置编译插件，目前支持编译插件有 `typescript`、`less`、`sass`

```
1 {
2   "setting": {
3     "useCompilerPlugins": [ // 编译插件配置，目前支持编译插件有 typescript、less、sass
4       "typescript", // ts语法支持
5       "sass" // sass语法编写css代码，将 .wxss 文件改为 .scss 即可
6     ]
7   }
8 }
```

- 关于 `Typescript`，可创建一个新项目，选择 `TS-基础模板` 创建即可，里面已经配置好了，如下：



- 关于 `sass`，在上面配置后，我们将定义样式的 `.wxss` 文件后缀名改为 `.scss` 即可。

SASS官网: <https://www.sass.hk/>

如：将 `app.wxss` 重命名为 `app.scss`，就可以按 Sass 语法来编写样式代码。

```
1 // css语法
2 .container .box{
3   width: 100%;
4   background-color: red;
5 }
6 // scss语法
7 .container {
8   .box {
9     width: 100%;
10    background-color: yellow;
11  }
12 }
```

sitemap.json 配置小程序索引

`sitemap.json` 配置小程序页面是否允许被微信索引。当开发者允许微信索引时，微信会通过爬虫的形式，为小程序的页面内容建立索引。当用户的搜索词条触发该索引时，小程序的页面将可能展示在搜索结果中。

- 如果项目未配置 `sitemap.json` 文件，默认所有页面都能被索引

参考: <https://developers.weixin.qq.com/miniprogram/dev/framework/sitemap.html>

1. 在 `app.json` 中引用 `sitemap.json` 文件


```
1 {  
2   "sitemapLocation": "sitemap.json"  
3 }
```

2. 项目根目录下创建 `sitemap.json` 文件

```
1  // 如下配置：所有页面都会被微信索引（默认情况）  
2  {  
3    "rules": [  
4      {  
5        "action": "allow", // allow 或 disallow, 未显式指明disallow默认都被索引  
6        "page": "*" // * 所有页面，或指定具体页面  
7      }  
8    ]  
9  }  
10  
11 // 如下配置 pages/index/index 页面被索引，其余页面不被索引  
12 {  
13   "rules": [  
14     {  
15       "action": "allow",  
16       "page": "pages/index/index"  
17     },  
18     {  
19       "action": "disallow",  
20       "page": "*"   
21     }  
22   ]  
23 }
```

`app.js` 全局逻辑文件(应用生命周期)

每个小程序都需要在 `app.js` 中调用 `App` 方法注册小程序实例，绑定生命周期回调函数、错误监听和页面不存在监听函数等。

`app.json` 逻辑代码：

```
1  // app.js  
2  App({  
3  
4    onLaunch (options) { // 仅被调用1次  
5      console.log('应用生命周期-onLaunch: 小程序初始化操作')  
6    },  
7  
8    onShow (options) { // 被调用1+n次  
9      console.log('应用生命周期-onShow: 小程序启动，或从后台进入前台显示时触发')
```

```
10 },
11
12 onHide () { // 被调用1+n次
13   console.log('应用生命周期-onHide: 小程序从前台进入后台隐藏时触发')
14 },
15
16 onError (error) {
17   console.log('onError: 小程序出现脚本错误或 API 调用报错时触发', error)
18 },
19
20 /* 意义不大, 页面跳转时找不到页面不被触发 */
21 onPageNotFound() {
22   console.log('onPageNotFound: 小程序要打开的页面不存在时触发, 可用来跳转到一个未发现页面')
23 },
24
25 // 开发者可以添加任意的函数或数据 App中, 用 getApp() 访问
26
27 // 自定义全局变量
28.globalData: '我是全局数据', // 调用: getApp().globalData
29
30 // 自定义全局方法: 通用型方法可定义在此
31.globalMethod() { // 调用: getApp().globalMethod
32   console.log('App自定义全局方法被调用')
33 },
34
35 })
```

整个小程序只有一个 App 实例，是全部页面共享的。

开发者可以通过 `getApp` 方法获取到全局唯一的 App 实例，获取 App 上的数据或调用开发者注册在 `App` 上的函数。

```
1 // pages/index/index.js
2
3 Page({ // 每个页面使用 Page 构造器注册
4
5   testApp() {
6     // 获取 App 实例
7     const app = getApp()
8     console.log(app.globalData) // 我是全局数据
9
10    // 调用app.json中的自定义全局方法
11    app.globalMethod()
12
13    const i = 1
14    // const 定义的是常量，不能重新赋值，所以报错，App.json中的onError可监听到错误
15    // i = 10
16  }
```

```
17 // wx.navigateTo 已经判断了页面不存在，所以拦截跳转并触发了 fail 回调，因此不会触发
    app.json中的onPageNotFound
18 wx.navigateTo({
19   url: '/pages/course/course',
20   fail: (e) => {
21     console.log('未找到页面', e)
22   }
23 })
24
25 },
26
27 })
```

第三章 WXSS 样式语法

WXSS (WeiXin Style Sheets)是一套样式语言，它具有 CSS 大部分特性，并对 CSS 进行了扩充以及修改。

WXSS 用来决定 WXML 的组件应该怎么显示。

WXSS 与 CSS 相比，WXSS 扩展的特性有：

- 尺寸单位：rpx
- 样式导入：@import

响应式尺寸单位 rpx

- rpx 详细说明：

在市面上有很多手机的屏幕宽度都不相同，而设计师在提供页面设计图时，一般只提供一个分辨率的图，严格按设计图标注的 px 做开发，在不同宽度的手机上界面很容易变形。而且主要是宽度变形。高度一般因为有滚动条，不容易出问题。

由此，开发者在开发小程序时，需要考虑小程序页面如何适配不同宽度的手机屏幕。

微信小程序为了解决屏幕适配问题，WXSS 扩展了 rpx 单位。

- rpx (responsive pixel)：称为响应式像素，根据屏幕宽度自适应的动态单位。

微信小程序规定屏幕基准宽度为 750rpx，也就是 750rpx 恰好为屏幕宽度，微信小程序可以根据屏幕宽度进行自适应，

屏幕变宽，rpx 实际显示效果会等比放大，反之等比变小。

- 开发建议：
 - 设计师在设计小程序页面时，将设计稿宽度定为 750px。
 - 手机可以用 iPhone6 作为视觉稿的标准，因为 iPhone6 手机屏幕宽度是 375px，而 750px 宽的设计稿在 iPhone6 上正好缩小一倍，可以不变形的完整展示在手机上。

即：在 iPhone6 上，rpx 和 px 的转换关系：1rpx = 0.5px, 750rpx = 375px

- 总结：设计稿是多少px（如750px），开发时就写多少rpx（750rpx），最终不同宽度屏幕上展示时，只是会等比放大/缩小。

- rpx转px计算公式：

```
1 | px值 = 屏幕的宽度 / 750 * rpx值
```

```
1 | // rpx转px
2 | function rpx2px(rpx) {
3 |   const { windowWidth } = wx.getWindowInfo()
4 |   return windowWidth / 750 * rpx
5 | }
```

- 示例：

```
1 | // index.wxml
2 | <text class="title">Hello 梦学谷</text>
3 |
4 | // index.scss
5 | .title {
6 |   font-size: 30px;
7 |   //font-size: 60rpx;
8 | }
```

演示效果：

变换不同宽度的机型，查看字体变化，30px在不同宽度下字体始终保持一样大，30rpx会根据宽度不同进行自动缩放（是基于750px的基准下动态计算的）。

全局和局部样式

全局样式：在 `app.wxss` / `app.scss` 文件中定义的样式，会作用于每一个页面，如：布局、字体色、背景色等全局样式

局部样式：在 `pages` 目录下的 `页面名.wxss` / `页面名.scss` 文件中定义的样式，只作用在对应的页面，并会覆盖 `app.wxss` / `app.scss` 文件中相同选择器的相同样式。

1. app.scss中定义全局样式

```
1  /**app.scss**/  
2  .flex-center {  
3    display: flex;  
4    align-items: center;  
5    justify-content: center;  
6  }  
7  
8  .main-bg {  
9    background-color: red;  
10   height: 50px;  
11 }
```

2. 在 pages/index/index.scss 中定义index页面的局部样式

```
1  /**index.scss**/  
2  
3  .main-bg {  
4    color: #fff;  
5    background-color: blue;  
6  }
```

pages/index/index.wxml 中引用样式，

```
1  <view class="flex-center main-bg">  
2    <view>测试样式</view>  
3  </view>
```

效果如下， 使用了 index.scss 中的定义的背景色，且app.scss中height样式还是有效的



样式导入 @import

在 `app.wxss/app.scss` 中使用 `@import` 语句可以导入外联样式文件，一样作用于每个页面，`@import` 后跟需要导入的外联样式文件的相对路径，用 `;` 表示语句结束。

示例代码：

```
1  /** 自己创建文件 assets/css/global.css */
2  .flex-center {
3    display: flex;
4    align-items: center;
5    justify-content: center;
6  }
7
8  /** app.wxss */
9  @import "../assets/css/global.css"; /* 每个页面公共css，最后分号不要少了分号； */
```

iconfont 字体图片

下载图标库

1. 进入阿里巴巴矢量图标库 <https://www.iconfont.cn/>，搜索需要的图标，添加至项目
2. 进入【我的项目】，找到要下载的项目后点击【项目设置】，如下：



在项目设置窗口，【字体格式】只勾选 **WOFF2** 和 **Base64**，然后点击保存，如下：

项目设置

项目名称

梦学谷点餐小程序

项目描述

请输入项目描述（可选）

GUID ?

生成 GUID

FontClass/
Symbol 前缀

icon-

Font Family

iconfont

字体格式

☐ 彩色 ?

☒ WOFF2

☐ WOFF

☐ TTF

☐ EOT

☐ SVG

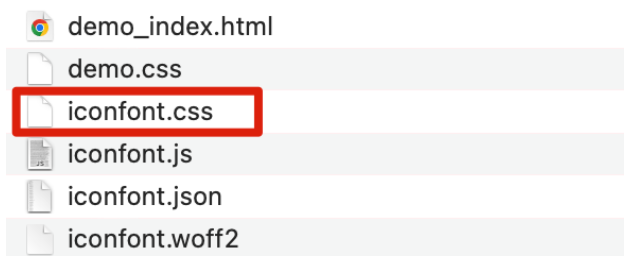
☒ Base64 ?

3. 选择【Font class】，点击【下载至本地】，如下：

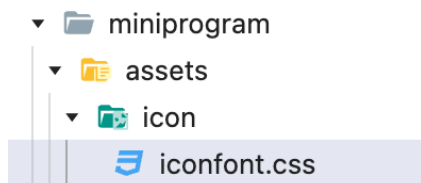


4. 下载后解压，复制 `iconfont.css` 到项目的 `assets/icon/` 目录下（目录自己先创建好）

点餐小程序图标位于：`03-配套资料\assets\icon\iconfont.scss`



拷贝后的效果：



5. 修改 iconfont.css 图标字体大小

(开发工具取消自动换行 (视图 > 切换自动换行)，不管多长都在一行显示)

```
1 .iconfont {  
2   font-family: "iconfont" !important;  
3   font-size: 24rpx; /* 16px 改为 24rpx */  
4   font-style: normal;  
5   -webkit-font-smoothing: antialiased;  
6   -moz-osx-font-smoothing: grayscale;  
7 }
```

6. 如果是 app.scss，即是 .scss 后缀名，需要将 iconfont.css 改为 iconfont.scss



7. 在 app.scss 文件的第一行引入 iconfont.css 字体图标文件

```
1 /* 如果当前文件scss即 app.scss，要将iconfont.css改为iconfont.scss 再导入，  
2    最后分号不要少了分号； */  
3 @import './assets/icon/iconfont.scss';
```

8. 挑选相应图标并获取类名，应用于页面

注意：iconfont 和 icon-saoma 都要在 iconfont.css 文件中声明，icon-saoma 是具体的图标类名

```
1 <!--pages/shopping/shopping.wxml-->
2 <text class="iconfont icon-saoma"></text>
```

内联样式

框架组件上支持使用 style、class 属性来控制组件的样式。

- style: 静态的样式统一写到 class 中。请尽量避免将静态的样式写进 style 中，以免影响渲染速度。

```
1 <!--pages/shopping/shopping.wxml-->
2 <view style="color:red;" > style样式 </view>
```

- class: 用于指定样式规则，其属性值是样式规则中类选择器名(样式类名)的集合，样式类名不需要带上 .，样式类名之间用空格分隔。

```
1 <!--pages/shopping/shopping.wxml-->
2 <view class="header-box" >class类名</view>
3
4
5 .header-box {
6     font-size: 40rpx;
7     color: blue;
8 }
```

选择器

目前支持的选择器有：

下面未列出来的其他CSS选择器，也可能支持。如果需要，可以先写 如果没有效果再替换下面选择器。

选择器	样例	样例描述
.class	<code>.intro</code>	选择所有拥有 class="intro" 的组件
#id	<code>#firstname</code>	选择拥有 id="firstname" 的组件
element	<code>view</code>	选择所有 view 组件
element, element	<code>view, checkbox</code>	选择所有文档的 view 组件和所有的 checkbox 组件
::after	<code>view::after</code>	在 view 组件后边插入内容
::before	<code>view::before</code>	在 view 组件前边插入内容

第四章 小程序框架组件

参考：<https://developers.weixin.qq.com/miniprogram/dev/component/>

在 WXML 文件中编写页面模板，页面模板中所有元素标签都必须闭合，如：

```
1 <view>Hello, 小程序</view>
2
3 <input name="username" ></input>
4 或者
5 <input name="username" />
```

view 组件

它类似于传统html中的div，用于包裹各种元素内容。

属性说明

属性名	类型	默认值	说明
hover-class	String	none	指定按下去的样式类。当 hover-class="none" 时，没有点击态效果
hover-stop-propagation	Boolean	false	指定是否阻止本节点的祖先节点出现点击态
hover-start-time	Number	50	按住后多久出现点击态，单位毫秒
hover-stay-time	Number	400	手指松开后点击态保留时间，单位毫秒

示例代码：

```
1 <!--pages/order-list/order-list.wxml-->
2
3 <view hover-class="view-hover" style="height: 300rpx; background-color: red;">
4   <!--
5     hover-class 指定按下去的样式类
6     hover-stay-time 手指松开后点击态保留时间，单位毫秒
7     hover-start-time 按住后多久出现点击态，单位毫秒
8     hover-stop-propagation 是否阻止祖先节点出现点击态 -->
9   <view hover-class="view-hover" hover-stay-time="2000" hover-start-time="2000"
10     hover-stop-propagation>
11     我有点击效果
12   </view>
</view>
```

```
1 /*pages/order-list/order-list.scss*/
2
3 .view-hover {
4   opacity: 0.6; /* 透明度 */
5 }
```

text 文本组件

text 组件用于显示文本内容。

常用属性：

属性	默认值	说明
user-select	false	文本是否可选
space		显示连续空格，无默认值，可取值： ensp 中文字符空格一半大小， emsp 中文字符空格大小， nbsp 根据字体设置的空格大小
decode	false	是否解码，可以解析的有 < > & '     ; 对应渲染后的效果：<，>，&，'，半个中文空格，1个中文空格，一个空格

注意：

1. text 组件内只支持 text 嵌套，如果嵌套其他组件不会显示。
2. 除了 text 文本节点以外的其他节点都无法长按选中。
3. 如果文本内容换行了，展示时也会换行展示。

示例代码：

```

1 <!--pages/order-list/order-list.wxml-->
2
3 <!-- text 文本组件
4   user-select 是否长按选择文本
5   space 显示连续空格，可取值：ensp/emsp/nbsp
6   decode 是否解码，可以解析的有：
7       `&lt; &gt; &amp; &apos; &ensp; &emsp; &nbsp;`
8   渲染效果： < > & ' 半个中文空格 1个中文空格 一个空格
9 -->
10 <text user-select space="nbsp" decode>小程序 组件
    &lt;text&gt;&amp;&apos;&ensp;&emsp;&nbsp;<text>轻松学</text><view>嵌套view不会展示
    </view></text>

```

渲染后的效果：

小程序 组件<text>&' 轻松学

rich-text 渲染html代码

渲染富文本，文本内容中包含了 html 代码，让html代码生效渲染，而不是以文本方式渲染。

常用属性：

属性	默认值	说明
nodes		HTML 字符串
user-select	false	文本是否可选，该属性会使文本节点显示为block
space		显示连续空格，无默认值，可取值： ensp 中文字符空格一半大小， emsp 中文字符空格大小， nbsp 根据字体设置的空格大小

示例：

1. 在 Page 中的 data 中定义一个htmlContent属性并初始化值

```

1 // pages/order-list/order-list.js
2
3 Page({
4   /** 页面的初始数据 */
5   data: {
6     htmlContent: "<div style='color: red'>我是html格式数据</div>"
7   },
8 })

```

2. 模板中使用双大括号引用 `htmlContent`，查看渲染后的效果

```
1 <text>{{htmlContent}}</text>
2 <rich-text nodes="{{htmlContent}}"/>
```

渲染后的效果：

```
<div style='color: red'>我是html格式数据</div>
我是html格式数据
```

image 图片组件

image 组件用于展示图片，支持 JPG、PNG、SVG、WEBP、GIF 等格式，[2.3.0](#) 起支持云文件ID

image 组件默认宽度320px、高度240px

```
1 <!-- image 显示图片
2   src 图片资源地址，/ 开头表示项目根目录（project.config.json中配置了miniprogramRoot，代表的是配置的路径）
3   mode 图片裁剪、缩放的模式，默认值：scaleToFill
4   show-menu-by-longpress 默认false，如果设置true：长按会弹出菜单，菜单项有 发送给朋友、收藏、保存图片、识别二维码或小程序码等
5   lazy-load图片懒加载，默认false，在页面进入一定范围（上下三屏）时才开始加载图片。
6 -->
7 <image src="/assets/images/advert.jpg" mode="scaleToFill" show-menu-by-longpress
  lazy-load/>
```

swiper 和 swiper-item 轮播图

滑块视图容器。`swiper` 是多个区域左右、上下切换。比如：banner 轮播图。

其中只可放置[swiper-item](#)组件，否则会导致未定义的行为。

```
1 <!-- swiper 滑块视图容器
2   autoplay 是否自动切换，默认false
3   circular 是否衔接循环滑动，默认false
4   vertical 滑动方向是否为纵向，默认false
5   interval 自动切换时间间隔，默认5000毫秒
6   indicator-dots 是否显示面板指示点，默认false
7   indicator-color 指示点颜色 默认 rgba(0, 0, 0, .3)
8   indicator-active-color 当前选中的指示点颜色，默认#000000
9   display-multiple-items 同时显示的滑块数量，默认1
10  previous-margin 前边距，可用于露出前一项的一小部分，接受 px 和 rpx 值
11  next-margin 后边距，可用于露出后一项的一小部分，接受 px 和 rpx 值
12 -->
13 <swiper class="swiper-box"
14   autoplay
```

```
15 circular
16 vertical="{{false}}"
17 interval="2000"
18 indicator-dots
19 indicator-color="#e7e7e7"
20 indicator-active-color="#fff"
21 display-multiple-items="1"
22 previous-margin="0"
23 next-margin="10px"
24 >
25 <swiper-item>
26   <image class="swiper-img" src="/assets/images/banner0.png" />
27 </swiper-item>
28 <swiper-item>
29   <image class="swiper-img" src="/assets/images/banner1.png" />
30 </swiper-item>
31 <swiper-item>
32   <image class="swiper-img" src="/assets/images/banner2.png" />
33 </swiper-item>
34 </swiper>
```

CSS 样式:

```
1 .swiper-box {
2   height: 550rpx;
3   width: 100%;
4   .swiper-img {
5     height: 100%;
6     width: 100%;
7   }
8   // 轮播图指示点样式（基于官网示例演示，浏览器控制台可查看到对应class类名来修改样式）
9   .wx-swiper-dots {
10    left: auto !important;
11    right: 0 !important;
12    bottom: 30rpx !important;
13  }
14  // 选中的指定点样式
15  .wx-swiper-dot-active {
16    width: 23px !important;
17    border-radius: 6px !important;
18  }
19 }
```

scroll-view 滚动视图区域

可滚动视图区域。用于某个区域左右横向滚动、上下纵向滚动。

- 横向滚动

```
1  <!-- 滚动视图区域
2      scroll-x 允许横向滚动，默认false
3      scroll-y 允许纵向滚动，默认false
4      enhanced 是否启用 scroll-view 增强特性，启用后配置如下属性才会生效：
5          - show-scrollbar 是否显示滚动条，默认true。官方bug：IOS下设置true开启后，横向滚动还
            是不显示滚动条
6          - bounces 在IOS下是否边界弹性控制，默认true
7          - fast-deceleration 在IOS下是否滑动减速速率控制，默认false
8      scroll-left 设置横向滚动条位置
9      scroll-top 设置纵向滚动条位置（基于scroll-y）
10 -->
11 <scroll-view
12     class="scroll-view-x"
13     scroll-x
14     enhanced show-scrollbar bounces fast-deceleration="{{false}}"
15     scroll-left="{{0}}"
16 >
17 <image src="/assets/images/hot1.png" />
18 <image src="/assets/images/hot2.png" />
19 <image src="/assets/images/hot3.png" />
20 </scroll-view>
```

CSS 样式：

```
1  // 横向滚动区域
2  .scroll-view-x {
3      white-space: nowrap;
4      image {
5          display: inline-block; // 同一行显示
6          width: 600rpx;
7          height: 260rpx;
8          border-radius: 16rpx;
9          margin-right: 20rpx;
10     }
11 }
```

- 纵向滚动：需要给 `<scroll-view>` 一个固定高度，通过 css 设置 height。

```
1 <!-- 纵向滚动, scroll-y , 并且要给 scroll-view 设置一个height -->
2 <scroll-view
3   class="scroll-view-y"
4   scroll-y
5   enhanced show-scrollbar="{{false}}"
6 >
7   <image src="/assets/images/hot1.png" />
8   <image src="/assets/images/hot2.png" />
9   <image src="/assets/images/hot3.png" />
10 </scroll-view>
```

CSS 样式:

```
1 // 纵向滚动区域
2 .scroll-view-y {
3   height: 300rpx;
4   image {
5     width: 100%;
6     height: 360rpx;
7     border-radius: 16rpx;
8   }
9 }
```

其他组件使用方式参见文档，后面用的时候会讲。

第五章 事件处理

参考: <https://developers.weixin.qq.com/miniprogram/dev/framework/view/wxml/event.html>

什么是事件

- 事件是视图层到逻辑层的通讯方式。
- 事件可以将用户的行为反馈到逻辑层进行处理。
- 事件可以绑定在组件上，当达到触发事件，就会执行逻辑层中对应的事件处理函数。

详细说明：

- 微信小程序中不能通过 `on` 的方式来绑定事件，而是使用 `bind` 关键字来进行绑定事件，并且小程序需要使用 `tap` 事件来替代 `click` 点击事件。

绑定事件有如下两种方式：

- 方式一： `bind:事件名`，`bind` 后面是英文状态下的冒号，冒号后面是事件名。

如： `<view bind:tap="handleConfirm">确定</view>`

- 方式二： `bind事件名`，`bind` 后面是直接跟事件名。

如：<view bindtap="handleConfirm">确定</view>

- 绑定事件后，去对应页面的 .js 文件中的 Page 构造器中定义事件处理函数。当达到触发事件时，就会执行 .js 中对应的事件处理函数。

事件的使用方式

- bindtap 事件，当用户点击该组件的时候，会在该页面对应的 Page 中找到相应的事件处理函数。

```
1 <!--pages/event-study/event-study.wxml-->
2 <button id="confrimBtn" bind:tap="handleConfirm">确定</button>
```

- 在相应的Page定义中写上相应的事件处理函数，并接收小程序自动传递的事件对象event。

```
1 // pages/event-study/event-study.js
2 Page({
3
4   handleConfirm(event) {
5     console.log('handleConfirm', event)
6   },
7
8 })
```

- log打印出来事件参数 event 信息大致如下：

```
1 {
2   type: "tap", // 代表事件的类型
3   timeStamp: 3741, // 页面打开到触发事件所经过的毫秒数
4   target: { // 触发事件的来源组件
5     id: "confrimBtn", //事件来源组件的id
6     dataset: {}, // 事件来源组件上由data-开头的自定义属性组成的对象
7     offsetLeft: 96, // 事件来源组件距离左侧偏移量
8     offsetTop: 1179 // 事件来源组件距离顶部（导航栏）的偏移量
9   },
10  currentTarget: { // 事件绑定的当前组件
11    id: "confrimBtn", // 当前组件的id
12    dataset: {}, // 当前组件上由data-开头的自定义属性组成的集合
13    offsetLeft: 96, // 当前组件距离左侧偏移量
14    offsetTop: 1179 // 当前组件距离顶部（导航栏）的偏移量，不会随着页面的滚动改变而改变
15  },
16  detail: { // 自定义事件所携带的数据
17    x: 188.765625, // x, y 是距离文档左上角的距离坐标，与 pageX, pageY 相同
18    y: 1198.703125
19  },
20  touches: [{ // 当前停留在屏幕上的触摸点数据
21    identifier: 0, // 触摸点的标识符
```

```
22   clientX: 188.765625, // clientX,clientY距离页面可显示区域（屏幕除去导航条）左上角
    距离，横向为x轴，纵向为y轴，会随着页面的滚动改变而改变
23   clientY: 700.703125,
24   pageX: 188.765625, // pageX,pageY 距离文档左上角的距离，文档的左上角为原点，横x轴，
    纵y轴，不会随着页面的滚动改变而改变
25   pageY: 1198.703125,
26   screenX: 201.1536102294922, // screenX,screenY距离屏幕左上角距离，横向为x轴，纵
    向为y轴，会随着页面的滚动改变而改变
27   screenY: 943.81494140625
28 },
29 changedTouches:[{ // 有变化的触摸点数据，数据格式与touches一样
30   identifier: 0,
31   clientX: 188.765625,
32   clientY: 700.703125,
33   pageX: 188.765625,
34   pageY: 1198.703125,
35   screenX: 201.1536102294922,
36   screenY: 943.81494140625
37 }]
38 }
```

事件传参 data-*

- 事件传参说明：

在微信小程序中，并不能直接在绑定事件处理函数时，加上括号传入自定义数据，如 `bind:tap="handleAdd(100)"` 是不支持的。

微信小程序如果想向事件处理函数传递自定义数据，我们需要在绑定事件的组件上使用标签属性的方式

`data-*` 传递数据，其中 `*` 是参数名，参数名是多个单词由连字符 `-` 连接，连字符写法会转换成驼峰写法，而大写字符会自动转成小写字符。如果传递多个数据，则写多个 `data-*`。

如：`<view bindtap="handleConfirm" data-total-num="10" data-userName="hello" />`

- `data-total-num`，最终会呈现为 `event.currentTarget.dataset.totalNum`；
- `data-userName`，最终会呈现为 `event.currentTarget.dataset.username`。

- 事件传参使用方式：

如：向事件处理函数 `handleConfirm` 传递一个 `name` 参数，其参数值为 `hello`

1. 模板中的对应组件上添加一个 `data-name="hello"`

```
1 <button id="confirmBtn" bind:tap="handleConfirm" data-name="hello">确定
  </button>
```

2. 在 `index.js` 中的 `handleConfirm` 函数中还是通过 `event` 来接收传递的数据

```
1 // pages/event-study/event-study.js
2
3 Page({
4
5   handleConfirm(event) {
6     console.log('handleConfirm', event)
7   },
8 })
```

注意查看 `event` 对象的 `currentTarget` 和 `target` 属性值，发现它们两个的 `dataset` 属性都有 `name` 参数值。

那我们从 `currentTarget` 和 `target` 中的哪个获取传递的参数更合适呢？

- 先说结论：大多数情况下直接从 `currentTarget`（事件绑定的当前组件）中获取

```
1 {
2   currentTarget: {
3     dataset: {
4       name: "hello"
5     },
6     id: "confrimBtn",
7     offsetLeft: 96,
8     offsetTop: 1179,
9   },
10  target: {
11    dataset: {
12      name: "hello"
13    },
14    id: "confrimBtn",
15    offsetLeft: 96,
16    offsetTop: 1179,
17  }
18 }
```

- 分析：从 `currentTarget` 和 `target` 中的哪个获取传递的参数更合适呢？

1. 在确定按钮上添加一个父元素 `view`，并声明一个 `id`、绑定一个 `tap` 事件和传递参数 `name`

```
1 <view id="parentView" bind:tap="handleParent" data-name="parent"
2   style="padding: 30rpx; background: red;">
3   <button id="confrimBtn" bind:tap="handleConfirm" data-name="hello">确定
4 </button>
5 </view>
```

2. 点击【确定】按钮，查看 `handleParent` 和 `handleConfirm` 事件处理函数中打印的日志

```
1 // pages/event-study/event-study.js
2
3 handleConfirm(event) {
4   console.log('handleConfirm', event)
5 },
6
7 handleParent(e) {
8   console.log('handleParent', e)
9 },
```



通过上面日志对比：

- `currentTarget` 对象数据是自身所绑定事件的组件上所定义的 `id` 和 `data-*` 数据：
handleConfirm 获取的是 button 组件上的数据，handleParent 获取的是 view 组件上的数据。
- `target` 对象数据是来源组件上所定义的 `id` 和 `data-*` 数据，因为点的是 button 按钮，来源组件就是 button 组件，所以：
handleConfirm 和 handleParent 获取的都是 button 组件上的数据。

• 总结：

- 我们开发中一般关注的是：

事件绑定在哪个组件上，这个事件处理函数所要获取的数据，应该就是这个组件上定义的数据，而不太关心来源组件的数据，所以通常使用 `currentTarget` 来获取数据。如果关心来源组件则通过 `target` 获取数据。

绑定并阻止事件冒泡 catch

事件绑定除了 `bind` 外，也可以用 `catch` 来绑定事件。与 `bind` 不同，`catch` 会阻止事件向上冒泡。

`catch` 绑定事件也是两种方式：

- `catch`: 事件名：中间有英文冒号，如：<view catch:tap="add" />
- `catch` 事件名：省略中间的英文冒号，如：<view catchtap="add" />

示例代码：

- 点击 **确定1** 按钮，会先后调用 `handleConfirm` 和 `handleParent`，因为button是基于 `bind` 绑定的事件会冒泡到父节点 `view`，从而触发父节点 `view` 上的事件。
- 点击 **确定2** 按钮，只会触发 `handleConfirm`，因为button是基于 `catch` 绑定的事件会阻止了 `tap` 事件冒泡，不再向父节点传递。

```
1 <view bind:tap="handleParent" id="parentView" data-name="parent"
2   style="padding: 30rpx; background: red;">
3   <button id="confrimBtn1" bind:tap="handleConfirm" data-name="hello1">确定1</button>
4   <button id="confrimBtn2" catch:tap="handleConfirm" data-name="hello2">确定
5   2</button>
6 </view>
```

事件传参 `mark:*`

除了使用 `data-*` 自定义属性方式传递参数外，还可以使用 `mark:*` 传递自定义数据。

`mark` 和 `dataset` 很相似，主要区别在于：`dataset` 仅包含一个节点的 `data-` 属性值；`mark` 会包含从触发事件的节点到根节点上所有的 `mark:` 属性值。

如：`<view mark:username="梦学谷" bindtap="handle"/>`

细节注意事项：

- 当事件触发时，事件冒泡路径上所有的 `mark` 会被合并，并返回给事件回调函数（即使事件不是冒泡事件，也会 `mark`）。
- 如果父子节点存在同名的 `mark`，子节点会覆盖父节点同名的 `mark` 值。
- 不同于 `dataset`，节点的 `mark` 不会做连字符 - 和大小写转换，即：传递的是什么名称，接收就是什么名称。

示例代码：

- **确定1** 按钮使用 `bind` 绑定的事件会事件冒泡，**确定2** 按钮使用 `catch` 绑定的事件会阻止冒泡，两个按钮点击将父节子的`mark`属性值合并返回，并且同名的子节点参数会覆盖父节点的，和参数名不会作的转换。
- 获取到的`mark`数据均为 `event.mark: {job-name: "经理", userName: "涛姐", sex: "男"}`

```
1 <view mark:userName="父亲" mark:sex="男" bind:tap="handleParent">
2   <button mark:userName="涛姐" mark:job-name="经理" bind:tap="handleConfirm">确定
3   1</button>
4   <button mark:userName="涛姐" mark:job-name="经理" catch:tap="handleConfirm">确定
5   2</button>
6 </view>
```


第六章 WXML 模板语法

数据绑定

在微信小程序的 WXML 模板文件中，数据绑定是通过 Mustache 风格的语法（即双大括号 {{ }}）将变量包起来。

注意事项：

- 双大括号 {{ }} 中不支持使用字符串内置方法(如substring)、对象内置方法(如Object.keys)、数组内置方法(如indexOf)等。

普通文本与运算

例如：在 WXML 模板文件中，可以直接写入 `{{ message }}` 来展示对应页面 data 对象中的 message 属性值。

```
1 <!-- pages/wxml-study/wxml-study.wxml -->
2 <view>
3   <view>=====普通文本与运算=====</view>
4   <view>普通文本: {{name}}</view>
5   <!-- <view>错误的写法: {{name salary}}</view> -->
6   <view>获取两个状态属性: {{name}}{{salary}}</view>
7
8   <view>算数运算: {{salary}} + {{bonus}} = {{salary + bonus}}</view>
9
10  <view>字符串拼接: {{ "Hello, " + name }}</view>
11  <view>字符串长度: {{ name.length }}</view>
12  <view>字符串方法(不支持): {{ name.substring(2) }}</view>
13
14  <view>三元运算符: {{ sex == 1 ? '男': '女' }}</view>
15
16  <view>对象数据: {{dept}}, {{dept.deptName}}</view>
17  <view>对象方法(不支持): {{Object.keys(dept)}}</view>
18
19  <view>数组数据: {{likes}}-----{{likes[0]}}, {{likes.length}}</view>
20  <view>数组方法(不支持): {{likes.indexOf('美女')}}</view>
21  <!-- 双大括号体内是不支持调用Page中定义的方法-->
22  <view>不支持调用自定义方法: {{ test('hello') }}</view>
23 </view>
```

```
1 // pages/wxml-study/wxml-study.js
2 Page({
3
4   // 定义状态属性
5   data: {
6     name: '小梦',
```

```
7 salary: 10000,  
8 bonus: 800, //奖金  
9 sex: 1, // 1-男, 2-女  
10 dept: {  
11   deptId: 1,  
12   deptName: '研发部'  
13 },  
14 likes: ['打码', '美女', '篮球'],  
15 },  
16  
17 // 自定义方法test  
18 test(title) {  
19   return '我是测试方法, ' + title  
20 },  
21  
22 })
```

渲染后效果:

=====普通文本与运算=====

普通文本: 小梦

获取两个状态属性: 小梦10000

算数运算: $10000 + 800 = 10800$

字符串拼接: Hello, 小梦

字符串长度: 2

字符串方法(不支持):

三元运算符: 男

对象数据: [object Object], 研发部

对象方法(不支持):

数组数据: 打码,美女,篮球-----打码, 3

数组方法(不支持):

组件属性 (标签属性)

在组件上的属性值, 可以通过双大括号 {} 动态接收数据。

1. 示例代码: 动态id值, style 动态样式, class动态类名: 当id等于1时, class属性值会添加 actived 类名

```
1 <!-- pages/wxml-study/wxml-study.wxml -->  
2 <view>=====组件属性=====</view>  
3 <view id="item-{{id}}"  
4   style="color: {{color}}; background-color: {{bgColor}}; text-align: center;"  
5   class="view-box {{id == 1 ? 'actived': ''}}"  
6 >  
7   组件属性  
8 </view>
```

```
1  /* pages/wxml-study/wxml-study.scss */
2
3  .view-box {
4    padding: 10rpx;
5    &.activated {
6      background-color: blue !important;
7      color: #fff !important;
8    }
9  }
```

```
1  /* pages/wxml-study/wxml-study.js */
2  Page({
3
4    // 定义状态属性
5    data: {
6      //.....省略其他属性
7
8      id: 1,
9      color: 'red',
10     bgColor: '#888888'
11   }
12
13 })
```

渲染效果:

=====组件属性=====

组件属性

2. 属性值接收布尔值(true、false), 需要在双引号之内

- true: boolean 类型的 true, 代表真值。
- false: boolean 类型的 false, 代表假值。

特别注意: 下面不要直接写 `checked="false"`, 其计算结果不是布尔false, 是一个字符串"false" 一样会勾选

```
1  是否选中<checkbox checked="{{ false }}" />
```

3. 属性值接收数字, 在双引号之内才表示数字, 不然就是字符串

```
1  <button bind:tap="handleBindNum" data-sex="1" data-age="{{18}}" >绑定数字</button>
```

```
1 // 点击绑定数值时调用
2 handleBindNum(e) {
3   // console.log('handleBindNum', e)
4   const { sex, age } = e.currentTarget.dataset
5   // this.data.sex获取data选项中的sex值为数字1，传递的sex为字符串1，严格比较是不相等的
6   const isEqual = this.data.sex === sex
7   console.log('handleBindNum', isEqual, sex, age) // false, 字符串 1, 和数字18
8 }
```

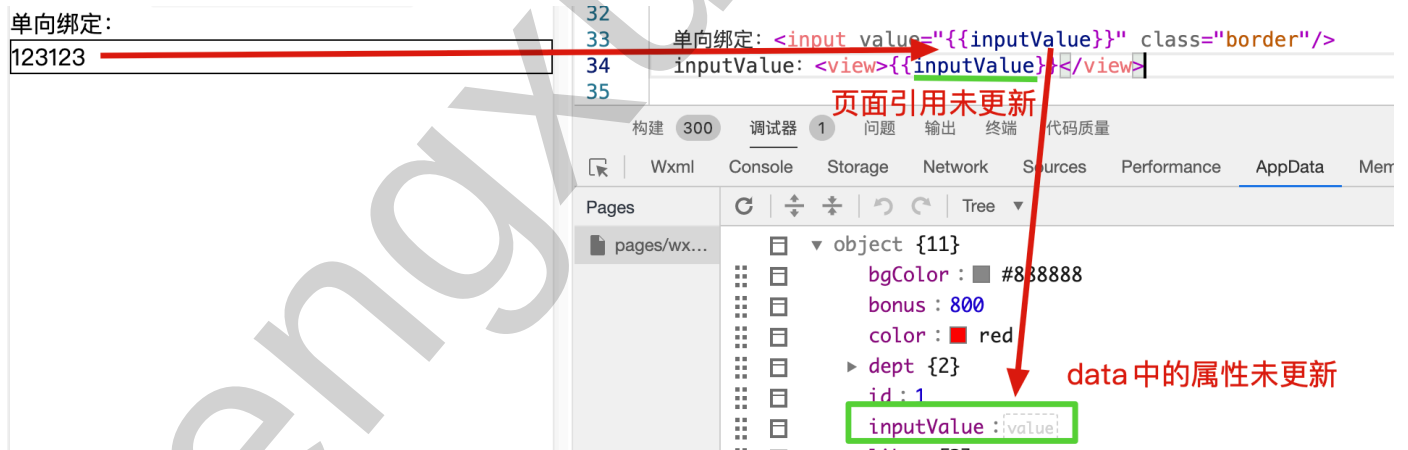
简易双向绑定 `model:`

在 WXML 中，普通的属性的绑定是单向的，单向指的是用户输入的值，不会同时改变 data 中属性值。

例如：

```
1 单向绑定: <input value="{{inputValue}}" class="border"/>
2  inputValue: <view>{{inputValue}}</view>
3
```

```
1 .border {
2   border: 1px solid #000;
3 }
```



如果需要在用户输入的同时改变 `this.data.value`，需要借助简易双向绑定机制。

此时，可以在对应项目之前加入 `model:` 前缀，例如：

```
1  
2 单向绑定: <input value="{{inputValue}}" class="border"/>  
3 简易双向绑定: <input model:value="{{inputValue}}" class="border"/>  
4  
5 inputValue: <view>{{inputValue}}</view>
```

这样，如果输入框的值被改变了，`this.data.value` 也会同时改变。同时，WXML 中所有绑定了 `value` 的位置也会被一同更新，[数据监听器](#) 也会被正常触发。

注意事项：用于双向绑定的表达式有如下限制：

- 只能是一个单一字段的绑定，例如错误用法：

```
1 <!--错误的-->  
2 不支持多字段双向绑定: <input model:value="{{ salary + bonus }}" class="border"/>
```

- 不支持 data 路径，也就是不支持对象属性和数组元素，例如错误用法：

```
1 <!--错误的-->  
2 不支持对象属性双向绑定: <input model:value="{{ dept.deptName }}" class="border"/>  
3 不支持数组元素双向绑定: <input model:value="{{ likes[0] }}" class="border"/>
```

条件渲染 wx:if

条件关键字

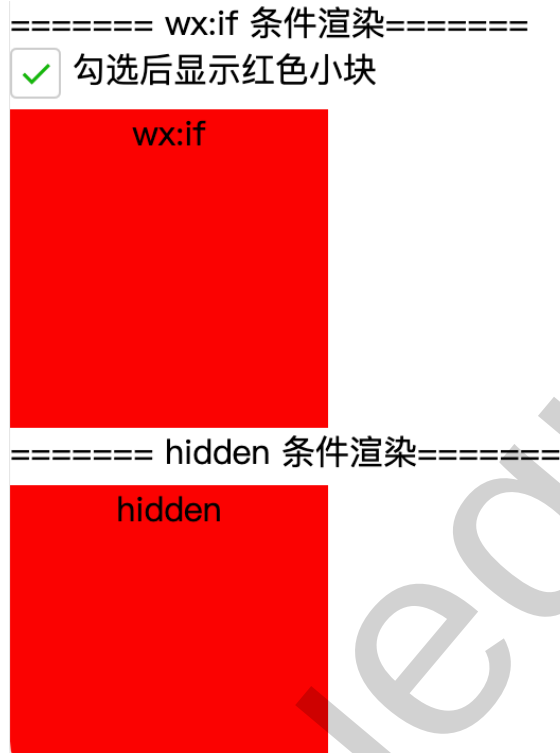
条件渲染主要用于控制页面结构的显示和隐藏，在微信小程序中实现条件渲染的控制属性有：

- `wx:if` 是否渲染当前组件（元素/标签）
`wx:elif` 不能单独使用，必须结合 `wx:if` 使用，可以出现0~n次
`wx:else` 不能单独使用，必须结合 `wx:if` 使用，可以出现0或1次

- `hidden` 是否隐藏，只是元素始终会被渲染并保留在 WXML 中，只是简单切换元素的 CSS 属性 `display` 来显示或隐藏。

案例实现

效果图：



实现步骤：

1. 编写页面代码

```
1  <!--pages/if-study/if-study.wxml-->
2
3  <view>===== wx:if 条件渲染=====</view>
4  <checkbox model:checked="{{show}}" />勾选后显示红色小块
5
6  <!-- wx:if 中双大括号的值为true，则显示当前组件元素，
7       一定不要少了双大括号 -->
8  <view wx:if="{{show}}" class="box">wx:if</view>
9  <!-- 当show为false，就是往下 wx:elif 判断为 true 则渲染，反之进入 wx:else -->
10 <view wx:elif="{{inputValue == 2}}" class="box" style="background-color:
    blue;">wx:elif</view>
11 <view wx:else>红块已隐藏</view>
12
13 -----
14 <input model:value="{{inputValue}}" style="border: 1px solid;" />
15
16 <view>===== hidden 条件渲染=====</view>
17 <view hidden="{{!show}}" class="box">hidden</view>
```

2. 编写样式

```
1 .box {  
2   width: 300rpx;  
3   height: 300rpx;  
4   background-color: red;  
5   margin-top: 10rpx;  
6   text-align: center;  
7 }
```

3. 定义 data 状态属性 show

```
1 Page({  
2   // 定义状态属性  
3   data: {  
4     show: false  
5   },  
6 })
```

wx:if 与 hidden 比较

1. 什么时候元素被渲染

- `wx:if` 如果在初始条件为假，则什么也不做，每当条件为真时，都会重新渲染条件元素，条件为假时进行销毁。
- `hidden` 不管初始条件是什么，元素总是会被渲染，并且只是简单地基于 CSS 进行切换 (display:none)。

2. 使用场景选择

- `v-if` 有更高的切换开销。
- `v-show` 有更高的初始渲染开销。

因此，如果需要非常频繁地切换，则使用 `hidden` 较好；如果在运行后 条件很少改变，则使用 `wx:if` 较好。

<block/> 包装元素

因为 `wx:if` 是一个控制属性，需要将它添加到一个标签上。如果要一次性判断多个组件标签，可以使用一个 `<block/>` 标签将多个组件包装起来，并在上边使用 `wx:if` 控制属性。

```
1 <block wx:if="{{ show }}">  
2   <view> view1 </view>  
3   <view> view2 </view>  
4 </block>
```

注意：

- `<block/>` 并不是一个组件，它仅仅是一个包装元素，不会在页面中做任何渲染，只接受控制属性。
- `hidden` 不能用于 `<block/>` 上，因为无效。
- 这样做的原因主要是：不会影响页面结构。

列表渲染 wx:for

列表渲染主要作用是通过循环遍历一个数组或对象，将其中每个元素的数据渲染到页面上。

wx:for 遍历数组

在组件上使用 `wx:for` 控制属性绑定一个数组（`wx:for="array"`），即可将数组中每一项的数据重复渲染该组件。

在遍历数组时：

- 默认当前项的变量名默认为 `item`，当前项的下标变量名为 `index`

```
1 <view wx:for="{{array}}">
2   当前项: {{item}}, 当前项下标: {{index}}
3 </view>
```

- 使用 `wx:for-item` 可以修改当前项的变量名，`wx:for-index` 可以修改当前下标的变量名：

```
1 <view wx:for="{{array}}" wx:for-item="alias" wx:for-index="idx">
2   当前项: {{alias}}, 当前项下标: {{idx}}
3 </view>
```

注意：不要用双大括号 `wx:for-item="{{alias}}" wx:for-index="{{idx}}"`

- 建议同时加上 `wx:key` 属性，来指定列表中项目的唯一的标识符。因为当数据改变触发渲染层重新渲染的时候，会校正带有 `key` 的组件，框架会确保他们被重新排序，而不是重新创建，以确保使组件保持自身的状态，并且提高列表渲染时的效率。

`wx:key` 的值以两种形式提供：

1. 值为字符串，代表是循环数组中当前项（`item`）的某个属性，该属性的值需要是列表中唯一的字符串或数字，且不能动态改变。

比如：`wx:key="id"` 代表的是获取循环数组中当前项（`item`）的 `id` 属性值，

注意：属性值不能用双大括号语法，`wx:key="{{id}}"` 这是错误的

```
1 <!--比如: this.data.array: [{id: 1, name: '张三'}] -->
2 <!-- wx:key="id" 属性值不要双大括号, 错误的 wx:key="{{id}}" -->
3 <view wx:for="{{array}}" wx:key="id">
4   当前项: {{item.name}}, 当前项下标: {{index}}
5 </view>
```

2. 保留关键字 `*this` 代表在 for 循环中的 item 本身, 这种表示需要 item 本身是一个唯一的字符串或者数字。

```
1 <view wx:for="{{ [1, 2, 3] }}" wx:key="*this">
2   {{item}}
3 </view>
```

案例演示:

1. `wx:for` 遍历对象数组

```
1  /* pages/for-study/for-study.js */
2
3  Page({
4    // 定义状态属性
5    data: {
6      orderList: [
7        {
8          id: 1,
9          shopName: '梦家1店',
10         orderAmount: 1000,
11         orderNum: 10
12       },
13       {
14         id: 2,
15         shopName: '梦学谷',
16         orderAmount: 2000,
17         orderNum: 8
18       }
19     ]
20   },
21   ))
```

```
1 <!--pages/for-study/for-study.wxml-->
2 <view>===== wx:for 遍历对象数组</view>
3 <!--
4     wx:for 指定要遍历的数组名称，要加上双大括号，不加就字符串遍历
5     wx:key 指定数组元素中的某个属性名，属性值要唯一。不能用双大括号{{id}}括起来；
6     item 代表循环数组的当前项变量名；
7     index 代表当前项的数组下标名，从0开始；
8     -->
9 <view wx:for="{{orderList}}" wx:key="id">
10   <text>当前下标: {{index}}, </text>
11   <text>店名: {{item.shopId}}, 订单金额: {{item.orderAmount}}, 数量:
    {{item.orderNum}}</text>
12 </view>
```

===== wx:for 遍历对象数组

当前下标: 0, 店名: 梦家1店, 订单金额: 1000, 数量: 10

当前下标: 1, 店名: 梦学谷, 订单金额: 2000, 数量: 8

2. wx:for 遍历字符串数组/数值数组等

```
1 Page({
2
3   // 定义状态属性
4   data: {
5     likes: ['打码', '美女', '篮球'],
6   },
7
8 })
```

```
1 <view>===== wx:for 遍历字符串数组/数值数组等=====</view>
2 <!-- `*this` 代表的是当前项数据 -->
3 <!-- <view wx:for="{{likes}}" wx:key="*this"> -->
4 <!-- index 就是当前项的下标-->
5 <view wx:for="{{likes}}" wx:key="index">
6   {{item}}
7 </view>
```

===== wx:for 遍历字符串数组/数值数组等=====

打码

美女

篮球

3. 自定义当前项的变量名和变量，不使用默认当前项变量名 item 和下标名 index。

```
1 <view>=== wx:for 自定义当前项变量名和下标名 ===</view>
2 <!--
3   wx:for-item 自定义当前项变量名，不要用双大括号
4   wx:for-index 自定义当前项下标名，不要用双大括号
5   -->
6 <view wx:for="{{ orderList }}" wx:key="id" wx:for-item="alias" wx:for-
  index="idx">
7   当前下标: {{idx}}, 店名: {{alias.shopName}}, 订单金额: {{alias.orderAmount}}, 数
  量: {{alias.orderNum}}
8 </view>
```

=== wx:for 自定义当前项变量名和下标名 ===

当前下标: 0, 店名: 梦家1店, 订单金额: 1000, 数量: 10

当前下标: 1, 店名: 梦学谷, 订单金额: 2000, 数量: 8

wx:for 遍历对象

1. wx:for 遍历对象时，index 代表循环对象中当前项的key，item 代表循环对象中当前项的value。

```
1 <!--pages/for-study/for-study.wxml-->
2
3 <view>===== wx:for 遍历对象 =====</view>
4 <!--
5   wx:for 遍历对象时:
6     index 代表循环对象中当前项的属性名
7     item 代表循环对象中当前项的属性值
8   -->
9 <view wx:for="{{dept}}" wx:key="index">
10   属性名: {{index}}, 属性值: {{item}}
11 </view>
12
13 <!--
14   wx:for 遍历对象时:
15     wx:for-index 针对当前项属性名取别名 key
16     wx:for-item 针对当前项属性值取别名 value
17     wx:key 当 wx:for-index 取了别名 key 后, wx:key也引用新名 key
18   -->
19 <view wx:for="{{dept}}" wx:for-index="key" wx:for-item="value" wx:key="key" >
20   属性名: {{key}}, 属性值: {{value}}
21 </view>
```

```
1 // pages/for-study/for-study.js
2
3 Page({
4   data: {
5
6     dept: {
7       deptId: 1,
8       deptName: '技术部'
9     }
10  }
11 })
```

渲染效果：

```
===== wx:for 遍历对象 =====
属性名：deptId, 属性值：1
属性名：deptName, 属性值：研发部
属性名：deptId, 属性值：1
属性名：deptName, 属性值：研发部
```

setData 修改数据

在微信小程序中修改数据，不推荐直接向 `this.data` 中的属性赋值，因为直接赋值无法更新页面的数据。

而是调用 `this.setData(Object data, Function callback)` 函数进行修改。

setData 函数的作用：

1. 更新 `this.data` 中对应的属性值（是同步更新的）
2. 驱动视图更新，将数据发送到(页面)视图层（是异步更新的）

setData 函数的参数说明：

字段	类型	必填	描述
data	Object	是	这次要改变的数据，是对象 <code>key: value</code> 的形式表示，将 <code>this.data</code> 中的 <code>key</code> 对应的值改变成 <code>value</code> 。其中 <code>key</code> 可以以数据路径的形式给出，支持改变数组中的某一项或对象的某个属性，如 <code>array[2].message</code> ， <code>a.b.c.d</code> ，并且不需要在 <code>this.data</code> 中预先定义。
callback	Function	否	setData 引起的页面更新渲染完毕后的回调函数

注意：

1. 单次设置的数据不能超过1024kB，请尽量避免一次设置过多的数据。
2. 请不要把 data 中任何一项的 value 设为 `undefined`，否则这一项将不被更新为undefined，更新后页面还是原来的值，并可能遗留一些潜在问题。

修改单属性

修改 data 选项中的 num 属性值

num: 1

更新num的值

代码实现：

```
1 <!--pages/setData-study/setData-study.wxml-->
2
3 <text>num: {{num}}</text>
4 <button bind:tap="updateNum">更新num的值</button>
```

```
1 // pages/setData-study/setData-study.js
2
3 Page({
4
5   data: {
6     num: 1,
7   },
8
9   updateNum() {
10
11     // this.data.xxx 获取data选项中定义的状态属性
12     const num = this.data.num
13     // console.log('num', num)
14
15     // 直接修改data选项中的属性值，可以修改数据，但是不会更新页面数据
16     // this.data.num += 1
17     // console.log('直接修改后的num', this.data.num)
18
19     /**
20      * 通过 this.setData 更新data选项中的属性值，且更新页面数据
21      * 参数1 (必填)：是接收一个对象，key是data选项中的属性名，value 是要更新的值
22      * 参数2 (非必填)：页面更新渲染完毕后的回调函数
23      */
24     //this.setData({ num: this.data.num + 1 }, () => {
```

```
25 // console.log('更新页面数据num完成')
26 //})
27 //console.log('setData修改后的num', this.data.num)
28
29 // 更新时，要注意更新的值是否为 undefined ，如果是这一项将不被会更新为undefined，
30 // 更新后页面还是原来的值，这样并可能遗留一些潜在问题
31 this.setData({num: undefined}) // 更新后页面还是 1
32 }
33
34 })
```

修改对象数据

修改/新增多级属性，key 直接写成数据路径的形式 a.b.c.d 。

```
1 <view>姓名: {{user.nickname}}, 年龄: {{user.age}}</view>
2 <button bindtap="updateUser">更新对象数据</button>
```

```
1 Page({
2   data: {
3     num: 1,
4     user: {
5       id: 1,
6       nickname: '桃姐'
7     }
8   },
9   // 更新对象数据
10  updateUser() {
11
12    /*
13     // 修改/新增多级属性，key 直接写成数据路径的形式 a.b.c.d
14     this.setData({
15       'user.nickname': '小梦', // 修改 user.nickname 属性值为 小梦
16       'user.age': 18 // 新增一个user.age属性，值为18
17     })
18    */
19
20    /*
21     // 如果要修改的属性比较多，可以先取出原来对象数据，再进行修改/新增属性
22     const { user } = this.data
23     user.nickname = '祥哥' // 修改对象属性值
24     user.age = 30 // 新增对象属性
25     this.setData({ user: user })
26    */
27  }
28 })
```



```
29 // 全部替换为一个新数据
30 const newUser = {id: 2, nickname: '飞哥', salary: 19999}
31 this.setData({user: newUser})
32
33 },
34
35 })
```

修改数组对象

实现：修改数组某个元素数据，向数组中追加一个元素，删除数组中某个元素，追加一个新的数组

```
1 <view wx:for="{{array}}" wx:key="index">{{item.name}}</view>
2 <button bindtap="updateArray">更新数组</button>
```

```
1 Page({
2   data: {
3     num: 1,
4     user: {
5       id: 1,
6       nickname: '桃姐'
7     },
8     array: [
9       {id: 1, name: '白白'},
10      {id: 2, name: '默默'}
11    ]
12  },
13
14  // 更新数组数据
15  updateArray() {
16    /*
17     // 修改array数组中下标为0的name属性值,
18     this.setData({
19       'array[0].name': '小刘'
20     })
21    */
22
23    /*
24     // 取出原数组数据
25     const { array } = this.data
26     // 向数组中追加一个元素
27     array.push({id: 3, name: '小张'})
28
29     this.setData({
30       array, // 等价于 array: array
31     })
```

```
32      */
33
34      /*
35      const { array } = this.data
36      // 删除数组中下标为0的1个元素
37      array.shift(0, 1)
38      this.setData({
39          array, // 等价于 array: array
40      })
41      */
42
43      // 将一个新的数组追加在原有数组后面
44      let { array } = this.data
45      const newArray = [{id: 3, name: '小花'}, {id: 4, name: '小张'}]
46      this.setData({
47          array: array.concat(newArray) // concat将两个数组合并后返回一个新的数组
48      })
49
50      },
51
52      })
```

第七章 WXS 脚本语言

参考: <https://developers.weixin.qq.com/miniprogram/dev/reference/wxs/>

WXML 文件中不支持直接调用页面的 .js 文件中定义的函数，但是在 WXML 文件中可以直接调用 WXS 中定义的函数。因此，下面我们将学习 WXS 语法。

WXS (WeiXin Script) 是小程序的一套脚本语言，**WXS 与 JavaScript 是不同的语言**，有自己的语法，并不和 JavaScript 一致。

- wxs 不支持ES6 及以上语法：
 - 不支持：let / const 声明变量、箭头函数、解构赋值、对象属性简写
 - 支持：var 声明变量、传统的函数 function () {} 等 ES5 语法
- wxs 有自己的数据类型：
 - number：数值
 - string：字符串
 - boolean：布尔值
 - object：对象
 - function：函数
 - array：数组

- `date`：日期
- `regexp`：正则
- wxs 遵循 CommonJS 规范
 - `module.exports` 导出模块
 - `require` 导入模块

WXS 模块

WXS 代码可以编写在 `wxml` 文件中的 `<wxs>` 标签内，或以 `.wxs` 为后缀名的文件内。

每一个 `.wxs` 文件和 `<wxs>` 标签都是一个单独的模块。

每个模块都有自己独立的作用域。即在一个模块里面定义的变量与函数，默认为私有的，对其他模块不可见。

一个模块要想对外暴露其内部的私有变量与函数，只能通过 `module.exports` 实现。

.wxs 文件

1. 与 `pages` 同级目录下，创建一个 `wxs` 文件夹，在此文件夹下创建一个 `common.wxs` 文件，在文件中编写 WXS 脚本：

```
1  // 注意：wxs 中不支持es6语法，如 const、let、箭头函数、{add: add} 对象简写成 {add} 都不支持。。。
2
3  // 变量，不能用es6中的 const、let 声明
4  var title = 'hello wxs'
5
6  // 方法/函数，不能用es6中的箭头函数 var toLower = () => { xxx }
7  function toLower (str) { // 字符串转小写
8      return str.toLowerCase()
9  }
10
11 // 每个 `wxs` 模块均有一个内置的 `module` 对象，通过`module.exports` 属性可以对外共享本模块的私有变量与函数。
12 // 导出方式1：一个一个导出
13 /*
14 module.exports.title = title
15 module.exports.toLower = toLower
16 */
17
18 // 导出方式2：对象形式，等号=不要少了
19 module.exports = {
20     title: title, // 不可以简写成 title
21     toLower: toLower
22 }
```

这个 `common.wxs` 文件可以被其他的 `.wxs` 文件 或 WXML 中的 `<wxs>` 标签引用。

`<wxs>` 标签引用.wxs文件

在 WXML 文件中使用 `<wxs>` 标签引用 `.wxs` 文件，`<wxs>` 标签属性有：

- `src` 指定路径，以/开头表示从项目根目录开始找
- `module` 为此wxs文件模块取一个模块名，通过此名称来调用wxs中的方法/变量等，`module` 属性值的命名必须符合下面两个规则：
 - 首字符必须是：字母（a-zA-Z），下划线（_）
 - 剩余字符可以是：字母（a-zA-Z），下划线（_），数字（0-9）

```
1 <!--pages/wxs-study/wxs-study.wxml-->
2
3 <!-- 1、引入wxs模块：
4   src 指定路径，以/开头表示从项目根目录开始找
5   module 为此wxs文件模块取一个别名，通过此名称来调用wxs中的方法/变量等
6   -->
7 <wxs src="/wxs/common.wxs" module="common" />
8
9 <!-- 2、调用 common 模块中的 title 变量 -->
10 <view>title: {{ common.title }}</view>
11 <!-- 2、调用 common 模块中的 toLower 方法，并在括号中直接传递参数，
12   PS：双大括号体内是不支持直接调用Page中定义的方法 -->
13 <view>调用wxs中的方法：{{ common.toLower('ABC') }}</view>
```

页面渲染效果：

title: hello wxs
调用wxs中的方法： 2

`<wxs>` 标签体定义模块

在 WXML 文件，使用 `<wxs>` 标签体定义模块代码，代码如下：

```
1 <!-- 直接在 wxml 文件中使用 wxs标签定义wxs模块代码
2   必须使用 module 属性为此模块取一个别名，通过此别名调用模块中导出的变量/方法等
3   -->
4 <wxs module="inner">
```

```
5 // 使用 require 函数导入wxs文件模块才行
6 var common = require('../wxs/common.wxs')
7 var content = '我是wxs标签内部定义的wxs模块'
8
9 // 实现一个将英文字母小写转大写
10 function toUpper(str) {
11     return str.toUpperCase()
12 }
13
14 /*
15 报错: common is not defined
16 原因: 内部wxs模块代码不能直接引用上面wxs标签导入的模板;
17 解决: 在此wxs模板中使用 require 函数导入wxs文件模块才行
18 */
19 console.log('inner', common.title) // inner hello wxs
20
21 module.exports = {
22     content: content,
23     toUpper: toUpper
24 }
25 </wxs>
26
27 <view>===调用wxml文件中使用wxs标签定义的wxs模块变量、方法===</view>
28
29 <view>内容为: {{inner.content}}</view>
30 <view>调用方法转大写: {{inner.toUpper('MengXuegu')}}</view>
```

.wxs 文件相互引用

在其他的 `.wxs` 文件中使用 `require` 函数引用 `common.wxs` 文件,

引用的时候, 要注意以下几点:

- 只能引用 `.wxs` 文件模块, 且必须使用相对路径。
- `wxs` 模块均为单例, `wxs` 模块在第一次被引用时, 会自动初始化为单例对象。多个页面, 多个地方, 多次引用, 使用的都是同一个 `wxs` 模块对象。
- 如果一个 `wxs` 模块在定义之后, 一直没有被引用, 则该模块不会被解析与运行。

示例代码: 在 `wxs` 目录下新建一个 `tools.wxs` 文件模块, 在此模块中引用 `common.wxs` 文件

```
1 // wxs/tools.wxs
2
3 // 引用另外一个wxs文件
4 var common = require('./common.wxs')
5
6 // 调用common模块的变量和方法
```

```
7 console.log(common.title) // hello wxs
8 console.log(common.toLowerCase('ABC')) // abc
9
10 function format() {
11   return '您好，我是tools.wxs的format方法' + common.title
12 }
13
14 module.exports = {
15   id: 10,
16   format: format
17 }
```

```
1 <!--pages/wxs-study/wxs-study.wxml-->
2
3 <wxs src="/wxs/tools.wxs" module="tools" />
4
5 <!-- 标签属性也可以调用wxs模块方法和变量 -->
6 <view id="{{ tools.id }}">{{ tools.format() }}</view>
```

第八章 页面生命周期 Page

页面生命周期指的是指小程序页面从 加载 -> 显示 -> 渲染 -> 隐藏 -> 销毁 的整个过程。

页面生命周期函数：

生命周期	描述
onLoad	页面加载时触发。 一个页面只会调用一次，可以在 onLoad 的参数中获取打开当前页面路径中的参数，和做一些初始化工作。
onShow	页面显示时触发。
onReady	页面初次渲染完成时触发。 一个页面只会调用一次，代表页面已经准备妥当，可以和视图层进行交互。 注意：对界面内容进行设置的 API 如 wx.setNavigationBarTitle ，请在 onReady 中进行
onHide	页面隐藏时触发。 如： wx.navigateTo 或底部 tab 切换到其他页面，小程序切入后台等。
onUnload	页面卸载时触发。 一个页面只会调用一次。 如： wx.redirectTo 或 wx.navigateBack 到其他页面时。

页面生命周期函数 直接定义在 `Page` 构造器的第一级参数中。

示例代码：

1. 在 app.json 中添加新页面配置：pages/page-life-cycle/page-life-cycle
2. 在 page-life-cycle 页面中添加页面生命周期函数：

```
1  // pages/page-life-cycle/page-life-cycle.js
2  Page({
3
4    /**
5     * 页面的初始数据
6     */
7    data: {
8      msg: 'hello 页面生命周期'
9    },
10
11   /**
12    * 生命周期函数--监听页面加载
13    */
14   onLoad (options) {
15     console.log('onLoad 页面被加载')
16   },
17
18   /**
19    * 生命周期函数--监听页面初次渲染完成
20    */
21   onReady () {
22     console.log('onReady 页面初次渲染完成')
23   },
24
25   /**
26    * 生命周期函数--监听页面显示
27    */
28   onShow () {
29     console.log('onShow 页面已显示')
30   },
31
32   /**
33    * 生命周期函数--监听页面隐藏
34    */
35   onHide () {
36     console.log('onHide 页面已隐藏')
37   },
38
39   /**
40    * 生命周期函数--监听页面卸载
41    */
42   onUnload () {
43     console.log('onUnload 页面已卸载')
44   },
```


45
46 })

第九章 组件化开发 Component

介绍

组件化开发就是将页面内的重复代码提取出来，封装成一个个可复用的视图组件，以便在不同的页面中重复使用，最后将这些组件拼接成一块就构成了一个完整的系统。它能够提高开发效率，增强可维护性。自定义组件在使用时与页面组件非常相似。

例如：项目可能会有轮播图、滑动广告推荐、内容区等组件，每个组件又包含了其它的像图片、导航链接等组件。

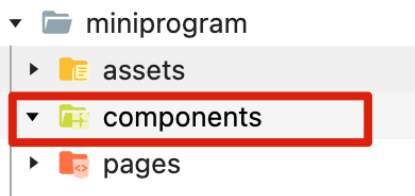


创建自定义组件

自定义组件类似于页面，一个自定义组件由 `json`、`wxml`、`wxss` (或 `scss`)、`js` 4 个文件组成。

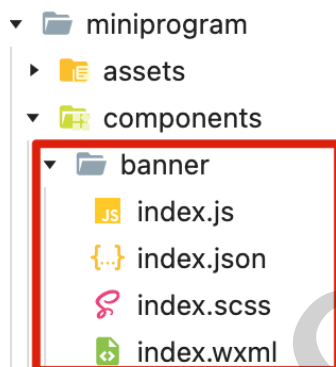
要编写一个自定义组件，按如下步骤操作：

1. 在项目根目录下创建 `components` 文件夹，用来专门存放自定义组件的。



2. 在 `components` 文件夹下，创建自定义组件相关文件： `json`、`wxml`、`wxss`、`js`

- 右键单击 `components` 文件夹，在弹出口点击新建文件夹（如 `banner`）
- 在新建的文件夹（如 `banner`）上单击右键，然后点击 新建 Component，输入自定义组件名（如 `index`）
- 此时开发工具会自动为自定义组件生成 `json`、`wxml`、`wxss(或scss)`、`js` 4个文件。



3. 在自定义组件的 `json` 文件中，进行自定义组件声明（将 `component` 属性设为 `true` 表示当前为自定义组件）：

```
1 {  
2   "component": true  
3 }
```

4. 在自定义组件的 `wxml` 文件中编写组件模板，在 `wxss` 文件中加入组件样式，它们的写法与页面的写法类似

注意：在组件 `wxss` 中不应使用 ID 选择器、属性选择器和标签名选择器，但是其实也是可以使用的。

```
1 <!--components/banner/index.wxml-->  
2  
3 <!-- 自定义轮播图组件 -->  
4 <swiper class="banner-box"  
5   autoplay  
6   circular  
7   indicator-dots  
8   interval="2000"  
9   >  
10  <swiper-item>  
11    <image src="/assets/images/banner0.png" />  
12  </swiper-item>
```

```
13 <swiper-item>
14   <image src="/assets/images/banner1.png" />
15 </swiper-item>
16 </swiper>
```

```
1  /* components/banner/index.wxss */
2
3  .banner-box {
4    height: 550rpx;
5    width: 100%;
6    image {
7      height: 100%;
8      width: 100%;
9    }
10 }
```

5. 在自定义组件的 `js` 文件中，需要使用 `Component()` 来注册组件，并提供组件的属性定义、内部数据和自定义方法等

具体使用方式后面再一一详解。

```
1  // components/banner/index.js
2  Component({
3
4    /** 组件的属性列表 */
5    properties: {
6
7    },
8
9    /** 组件的初始数据 */
10   data: {
11
12   },
13
14   /** 组件的方法列表 */
15   methods: {
16
17   },
18
19 })
```

使用自定义组件

创建好自定义组件后，使用已注册的自定义组件前，首先需要在页面的 `json` 文件中进行引用声明（使用 `usingComponents` 属性声明）才可以使用。

我们需要使用创建好的自定义组件，首先必须将自定义组件进行局部注册(页面.json/组件.json)或者全局注册(app.json)

1. 局部注册：注册后当前页面/组件可以使用

在需要引用自定义组件的页面 `json` 或者 自定义组件 `json` 文件中，使用 `usingComponents` 属性进行注册。

- `usingComponents` 属性的Key是 每个自定义组件的标签名，Value是对应的自定义组件文件路径：

```
1 // pages/index/index.json
2 {
3   "usingComponents": {
4     // "标签名": "自定义组件路径"
5     "banner": "/components/banner/index"
6   }
7 }
```

注册后，在此页面/组件的 `wxml` 中像使用基础组件一样使用自定义组件。节点名就是自定义组件的标签名，节点属性即传递给组件的属性值。



2. 全局注册：注册后所有页面、所有组件可以使用

在 `app.json` 中使用 `usingComponents` 属性进行全局注册。

```
1 // app.json
2 {
3   "pages": [
4   ],
5   "window": {
6   },
7   "usingComponents": {
8     "banner": "/components/banner/index"
9   }
10 }
```

注册后所有页面和所有自定义组件的 `wxml` 中都可以使用。

注意事项：

- 自定义组件的标签名只能是小写字母、中划线和下划线的组合。
- 自定义组件也是可以引用自定义组件的，引用方法类似于页面引用自定义组件的方式（在组件.json中使用 `usingComponents` 字段）。
- 自定义组件和页面所在项目根目录名不能以“wx-”为前缀，否则会报错。

组件间通信方式介绍

1. `properties`：子组件接收父组件数据。

- 子组件使用 `properties` 选项声明要接收父组件传递的数据属性；属性名采用驼峰写法（`propertyName`）

```
1  Component({
2
3    /**组件的属性列表 */
4    properties: {
5      // 属性名采用驼峰写法 (propertyName) ,
6      // 属性值为数据类型 String/ Number/ Boolean/ Object/ Array 选其一，也可以为
7      // null 表示不限制类型。
8      bannerList: Array
9    },
10 })
```

- 父组件以标签属性的方式向子组件传递 `properties` 属性数据，可以使用横杠（`property-name`）和驼峰写法（`propertyName`）指定传递的属性名，但是小程序官方使用横杠写法（`property-name`）。

```
1  <banner banner-list="{{ bannerList }}" />
```

2. 自定义事件：用于子组件向父组件传递数据。

- 在子组件通过 `this.triggerEvent('事件名', 要传递的数据)` 触发父组件绑定的自定义事件。
- 在父组件模板中可以使用 `bind:自定义事件="事件处理函数"` 或 `bind自定义事件="事件处理函数"` 来绑定自定义事件

```
1  // 父组件绑定 navTo 自定义事件
2  <banner bind:navTo="handleNavTo" />
3
4  // 子组件触发父组件绑定的 navTo 事件
5  this.triggerEvent('navTo', 111)
```

3. 父组件访问子组件数据：

- 父组件可以通过 `this.selectComponent(CSS选择器)` 方法获取子组件实例对象，获取后可以直接访问子组件的数据和方法。

4. 插槽分发内容 `slot`，动态向子组件传递模板内容

子组件接收父组件的数据 `properties`

子组件 `properties` 声明组件属性

子组件接收父组件数据，先在子组件使用 `properties` 声明组件属性，父组件通过标签属性的方式动态传递数据。

- 使用 `properties` 选项在组件对象中声明组件属性，用于接收父组件传递的数据

```
1 Component({
2
3   /** 组件的属性列表 */
4   properties: {
5     // 此处值有下面3种方式
6   },
7
8 })
```

- 方式1: key 是属性名 和 value 是数据类型 (支持 **String/ Number/ Boolean/ Object/ Array** 选其一，也可以为 `null` 表示不限制类型)

```
1 properties: {
2   _id: Number,
3   name: String,
4   isPublished: Boolean,
5   commentIds: Array,
6   course: Object,
7 }
```

- 方式2: key 是属性名, value 使用对象类型指定: `type`数据类型、`value` 默认值


```
1  properties: {
2    _id: { // 多种约束，使用对象形式
3      type: Number, // 数据类型，同方式1支持的类型
4      value: 10 // 默认值
5    },
6    commentIds: { // 这个属性可以是 String、Number、Array 三种类型中的一种
7      type: String,
8      optionalTypes: [Number, Array], // 指定多个类型
9      value: 0
10   }
11 }
```

示例代码

子组件代码实现

1. 创建子组件 **components/childComponent/index**.四个文件



2. 查看 **childComponent/index.json** 文件中有一个配置：标识当前为组件，如下：

```
1  {
2    "component": true
3  }
```

3. 使用 **properties** 选项声明接收父组件传递的数据。

```
1  // pages/component/childComponent/index.js
2  Component({
3
4    /**组件的属性列表：用于接收父组件传递的数据 */
5    properties: {
6      // 方式1: key是属性名（用 camelCase 驼峰写法），value是数据类型或约束对象
7      _id: Number, // 注意： id 是保留字，不要取名为 id，不然接收不到传递的数据
8      name: {
9        type: String,
```

```
10     value: 'mxg' // 默认值
11   },
12   isPublished: Boolean,
13   commentIds: { // 支持的类型为 String/Number/Array
14     type: String,
15     optionalTypes: [Number, Array], // 指定多个类型
16     value: [0]
17   },
18   course: Object,
19 },
20
21 /** 组件的内部数据, 和 properties 一同用于组件的模板渲染 */
22 data: {
23 },
24
25 /**组件的方法列表*/
26 methods: {
27   // 自定义方法
28   print() {
29     // 解构出 _id, name 属性
30     const { _id, name } = this.properties
31     console.log('通过this访问properties', _id, name,
32 this.properties.isPublished)
33   },
34 }
35 })
```

```
1 <!--pages/component/childComponent/index.wxml-->
2 <text>====子组件====</text>
3
4 <!-- 模板中直接通过【属性名】可获取数据, 不需要指定this.properties -->
5 <view>
6   _id: {{ _id }}, name: {{ name }}, isPublished: {{ isPublished }}
7   , commentIds: {{ commentIds }}, course: {{ course.cname }}
8 </view>
9
10 <button bind:tap="print" type="primary">打印properties</button>
```

父组件代码实现

1. 在app.json中的 pages 添加页面配置 `pages/component/component`，开发工具自动创建其页面文件。
2. 在页面配置 `pages/component/component.json` 导入子组件 `childComponent`。
 - o `usingComponents` 对象：
 - key 是组件名，在模板中通过此名引用组件；组件名可以使用 camelCase 驼峰形式和 kebab-case

横杠（官方推荐 kebab-case 横杠形式）

- value 是组件文件路径，开头的 / 代表项目根目录（如 miniprogram）

```
1 {
2   "usingComponents": {
3     "child-component": "/components/childComponent/index"
4   }
5 }
```

3. 父组件基于上面 导入的组件名来引用其子组件，并向子组件传递 properties

```
1 <!--pages/component/component.wxml-->
2
3 <view>=====父组件引用自定义子组件=====</view>
4 <!--
5   父组件向子组件传递 数据 时，可以使用 kebab-case 横杠 和 camelCase 驼峰，
6   官方推荐更贴近 HTML 的书写风格方式 kebab-case 横杠形式。
7   1、向 properties 传递静态值：
8     _id="1" 传递的是字符串 '1'，id="{{1}}" 传递的才是数字1
9     name="梦学谷" 传递的是字符串 '梦学谷'
10    is-published="true" 传递的是字符串 'true'，
11    is-published="{{true}}" 传递的是布尔值 true
12    总结：没有 {{ }} 传递的都是字符串静态值。
13  -->
14 <child-component
15   _id="1"
16   name="梦学谷"
17   isPublished="{{true}}"
18   comment-ids="{{[1, 2, 3]}}"
19   course="{{ {cId: 10, cname: '小程序教程'} }}"
20 />
```

properties 注意事项

1. 不要在子组件中直接修改父组件传递的数据，需要后面讲解的 自定义事件 修改
2. 数据初始化在哪个组件，更新数据的方法(函数)就应该定义在哪个组件。

自定义事件 triggerEvent

说明：

- 不要在子组件中直接修改父组件传递的数据，如果要修改可通过自定义事件方式，间接修改父组件数据。

- 子组件需要触发调用父组件方法（函数），可通过自定义事件来实现。

自定义事件：

- 子组件中使用 `this.triggerEvent('事件名' [,传递的参数])` 函数触发自定义事件
- 在父组件模板中可以使用 `bind` 来绑定事件监听器。如：`bind:search-goods="searchGoodsHandle"`

子组件触发自定义事件

在子组件中触发自定义事件，从而调用在父组件中绑定的事件处理函数。

- 子组件模板中添加一个 input 输入框，点击手机输入法上的【完成】按钮时调用子组件中的函数，在函数中触发自定义事件

```
1 <!--pages/component/childComponent/index.wxml-->
2
3 <view>==== 自定义事件 =====</view>
4 <!-- bind:confirm 手机点击【完成】按钮时触发 -->
5 <input
6   model:value="{{ keyword }}"
7   bind:confirm="search"
8   placeholder="请输入商品信息"
9   style="border: 1px solid; height: 80rpx;margin: 30rpx;"
10 />
```

- 子组件中js代码中，使用 `this.triggerEvent('事件名' [,传递的参数])` 函数触发自定义事件

```
1 // pages/component/childComponent/index.js
2 Component({
3
4   /** 组件的内部数据，和 properties 一同用于组件的模板渲染 */
5   data: {
6     keyword: ''
7   },
8
9   /**组件的方法列表*/
10  methods: {
11
12    search() {
13      //console.log('搜索', this.data.keyword)
14      // 使用 this.triggerEvent('事件名' [,传递的参数]) 触发事件
15      this.triggerEvent('searchGoods', this.data.keyword)
16    },
17
18  }
```

```
19  
20  })
```

父组件监听自定义事件

在父组件模板的子组件标签上，通过 `bind:自定义事件名="事件处理函数"` 或 `bind自定义事件名="事件处理函数"` 来监听子组件触发的自定义事件和绑定事件处理函数，当子组件触发自定义事件后，就会调用父组件中绑定的事件处理函数。

- `pages/component/component.wxml` 页面中监听事件，绑定事件处理函数

```
1  <!--pages/component/component.wxml-->  
2  
3  <view>=====父组件引用自定义子组件=====</view>  
4  <!-- 在父组件中，通过 bind或bind: 绑定子组件的自定义事件  
5       在子组件中，当触发 searchGoods 事件后，就会调用到父组件的 searchGoodsHandle 事件处理函  
6       数 -->  
7  <child-component  
8      bind:searchGoods="searchGoodsHandle"  
9  
10     _id="1"  
11     name="梦学谷"  
12     isPublished="{{true}}"  
13     comment-ids="{{[1, 2, 3]}}"  
14     course="{{ {cId: 10, cname: '小程序教程'} }}"  
15  />
```

```
1  // pages/component/component.js  
2  Page({  
3  
4      searchGoodsHandle(e) {  
5          // 通过 e.detail 获取传递的数据  
6          const keyword = e.detail  
7          console.log("父组件监听到 search-goods 事件", e, keyword)  
8      },  
9  
10 })
```

父组件获取子组件实例 selectComponent

在父组件中可以通过调用 `this.selectComponent` 函数，来获取子组件的实例对象，从而访问子组件的数据和方法。

调用时需要传入一个匹配选择器 `selector`，如：`this.selectComponent("#my-component")`。

selector 详细语法可查看 [selector 语法参考文档](https://developers.weixin.qq.com/miniprogram/dev/api/wxml/SelectorQuery.select.html)。链接：<https://developers.weixin.qq.com/miniprogram/dev/api/wxml/SelectorQuery.select.html>。

代码示例：

父组件获取模板中 `id` 为 `my-component` 的子组件实例对象，即子组件的 `this`。

```
1 // pages/component/component.js
2 Page({
3
4   // 父组件：获取子组件实例
5   getChildComponent () {
6     // 将会获取模板中 id 为 my-component 的子组件实例对象，即子组件的 this
7     const child = this.selectComponent('#my-component')
8
9     // 获取子组件的数据：properties和data选项中的数据都包含了
10    const childData = child.data
11    console.log('获取到子组件实例为', child, childData)
12
13    // 调用子组件的方法
14    child.print()
15  },
16
17 })
```

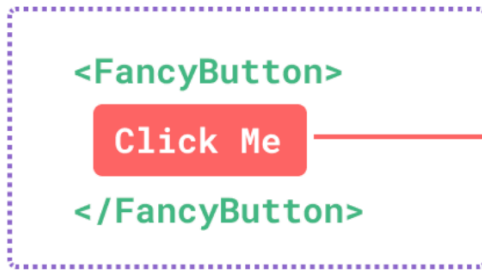
```
1 <!--pages/component/component.wxml-->
2
3 <view>=====父组件引用自定义子组件=====</view>
4 <!-- 在引用的子组件上添加一个 id 属性值-->
5 <child-component
6   id="my-component"
7
8   ... 省略其他
9 />
10
11 <!-- 通过 this.selectComponent(选择器) 获取子组件实例 -->
12 <button bind:tap="getChildComponent" type="primary">获取子组件实例</button>
```

组件 wxml 的插槽 slot

作用：用于子组件接收父组件传递的 模板内容，子组件接收后在对应位置渲染。（而上面properties和自定事件只是传递数据）

场景：一般是某个位置需要经常动态切换显示效果。

parent template



<FancyButton> template



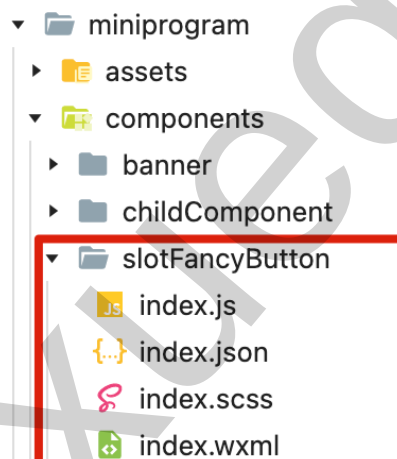
● slot content

● slot outlet

默认插槽

在子组件中定义插槽, 当父组件向指定插槽传递模板内容后, 模板内容就会在插槽处渲染出现, 父组件没有传递内容则插槽出口不显示内容。

1. 创建 slotFancyButton 组件 (miniprogram/components/slotFancyButton/index) :



2. 在 slotFancyButton 子组件声明插槽: <slot>

```
1 <!--components/slotFancyButton/index.wxml-->
2
3 <!-- slot 定义默认插槽 -->
4 <button class="fancy-btn">
5   <!-- <slot> 元素是一个插槽出口, 标示了父元素提供的插槽内容将在此处被渲染。-->
6   <slot></slot>
7 </button>
```

```
1 /* components/slotFancyButton/index.wxss */
2 .fancy-btn {
3   margin: 10rpx;
4 }
```


3. 在父组件的页面配置 `pages/component/component.json` 导入子组件。

```
1 {
2   "usingComponents": {
3     "fancy-button": "/components/slotFancyButton/index",
4     // ....省略其他
5   }
6 }
```

4. 父组件模板内容中引用子组件 `<fancy-button>`，并传递模板内容。

```
1 <!--pages/component/component.wxml-->
2
3 <view>===== 向 slot 插槽传递模板内容 =====</view>
4
5 <fancy-button>
6   <!-- 插槽模板内容：替换到fancy-button的slot元素 -->
7   Click Me
8 </fancy-button>
9
10 <fancy-button>
11   <!-- 插槽模板内容：可以是任意合法的模板内容，不局限于文本 -->
12   <span style="color:red">点我! </span>
13 </fancy-button>
14
15 <fancy-button>
16   <!-- 插槽模板内容：插槽内容可以访问到父组件的数据作用域，因为插槽内容本身是在父组件模板中定义的 -->
17   {{ message }}
18 </fancy-button>
```

```
1 // pages/component/component.js
2 Page({
3
4   data: {
5     message: '父组件数据'
6   },
7
8 })
```

上面渲染后的效果：

===== 向 slot 默认插槽传递模板内容 =====

Click Me

点我!

父组件数据

具名插槽

默认情况下，一个组件的 wxml 中只能有一个 `<slot>` 插槽出口。

如果组件中需要使用多个 `<slot>` 插槽出口，则可以在组件 `js` 文件中声明启动，然后在 `<slot>` 元素上添加一个属性 `name` 来给每个插槽分配唯一的 ID，这类带 `name` 的插槽被称为具名插槽。

```

Component({
  options: {
    // 当前组件开启多 slot 具名插槽
    multipleSlots: true
  },
  properties: {},
  data: {}
})

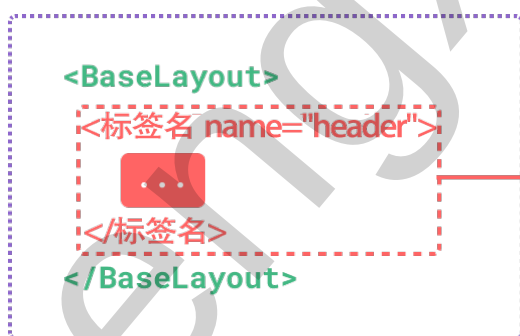
```

```

1
2
3
4
5
6 <slot name="header"></slot>
7 <slot name="main"></slot>
8 <slot name="footer"></slot>
9
10
11
12
13

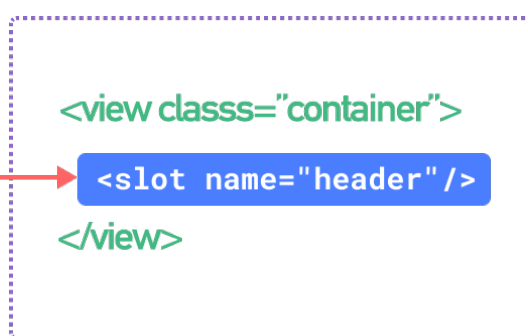
```

parent template



named slot content

<BaseLayout> template

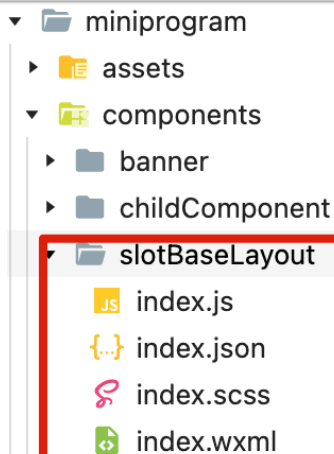


named slot outlet

replace

举例来说：

1. 创建 slotBaseLayout 组件（miniprogram/components/slotBaseLayout/index）：



2. 在 slotBaseLayout 子组件声明默认插槽和具名插槽

- 默认插槽 `<slot>`：没有提供 name 的 `<slot>` 出口。
- 具名插槽 `<slot name="插槽名">`：通过 name 属性值指定唯一插槽名，父组件通过此名确定此处的模板内容。

```
1  <!--components/slotBaseLayout/index.wxml-->
2
3  <!--
4      具名插槽：在组件中使用多个 <slot> 插槽出口，需要如下操作
5      1. 在组件js中开启具名插槽，如 index.js
6      2. 在 <slot> 元素上添加一个 name 属性来指定此插槽的唯一标识
7  -->
8  <view class="container">
9
10     <view class="header">
11         <!-- 具名插槽：name 属性值指定唯一插槽名，父组件通过此名确定此处的模板内容 -->
12         <slot name="header"></slot>
13     </view>
14
15     <view class="main">
16         <!-- 默认插槽：未指定name属性 -->
17         <slot></slot>
18     </view>
19
20     <view class="footer">
21         <!-- 具名插槽 -->
22         <slot name="footer"></slot>
23     </view>
24
25 </view>
```

```
1  /* components/slotBaseLayout/index.wxss */
2  .container {
3    font-size: 36rpx;
4    line-height: 60rpx;
5  }
```

3. 在父组件的页面配置 `pages/component/component.json` 导入子组件。

```
1  {
2    "usingComponents": {
3      "base-layout": "/components/slotBaseLayout/index",
4      // ....省略其他
5    }
6  }
```

4. 父组件要为具名插槽传入模板内容，我们需要使用一个对应内容元素上添加 `slot` 属性，并将目标插槽的名字传给该属性：

```
1  <!--pages/component/component.wxml-->
2  <view>===== 向 slot 具名插槽传递模板内容 =====</view>
3
4  <base-layout>
5
6    <!-- slot="插槽名", 将下面模板内容传入子组件的 header 插槽中 -->
7    <view slot="header">
8      <text>页面头部区域：梦学谷-mengxuegu.com</text>
9    </view>
10
11    <!-- 默认插槽：在 base-layout 标签体中，没有被 有 slot 属性的标签所包裹的内容都会显示到
    默认插槽处-->
12    <view>
13      <text>页面核心区域：展示您想要展示的内容</text>
14    </view>
15
16    <view slot="footer">
17      页面底部区域：展示您的联系方式
18    </view>
19
20  </base-layout>
```

数据监听器 observers

监听器说明

数据监听器可以用于监听和响应任何属性和数据字段的变化。

使用 `observers` 选项来监听一个或多个属性(data/properties)的变化，当属性值发生响应式变化时，对应监听器的回调函数会自动调用。

注意事项：

1. `observers` 选项中能用于组件 `Component` 中 `Component({ observers: {} })`，不能用于页面 `Page` 中 `Page({ observers: {} })`。
2. 数据监听器监听的是 `setData` 或者模板中 `model` 双向绑定所涉及到的数据属性；通过直接赋值则不会被监听到。
3. 如果在数据监听器函数中使用 `setData` 设置本身监听的数据字段，可能会导致死循环，需要特别留意。

监听器的基本使用

1. 监听模板中使用 `model` 进行简单双向绑定的属性变化

- 监听输入框 `keyword` 数据的变化

```
1 <!--pages/component/childComponent/index.wxml-->
2
3 <input
4   model:value="{{keyword}}"
5   placeholder="请输入商品信息"
6   style="border: 1px solid; height: 80rpx;margin: 30rpx;"
7 />
8
9 <view>搜索结果: {{ result }}</view>
```

- 在组件js文件的 `Component` 中添加 `observers` 选项，来监听keyword变化

```
1 // pages/component/childComponent/index.js
2 Component({
3
4   /** 组件的内部数据，和 properties 一同用于组件的模板渲染 */
5   data: {
6     keyword: '',
7   },
8
9   // 用于定义监听器
10  observers: {
11
12    // 当 keyword 被响应式更新时，执行这个函数
13    keyword (newValue) {
14      console.log('监听到 keyword 值的改变', newValue)
```

```
15
16     if (newValue.includes('meng')) {
17         this.setData({result: 'hello, mengxuegu'})
18     } else {
19         this.setData({result: '未搜索到数据'})
20     }
21
22 },
23
24 },
25
26 })
```

2. 监听当属性被 setData 有时，执行一些操作，

如：`this.data.sum` 永远是 `this.data.numberA` 与 `this.data.numberB` 的和。此时，可以使用数据监听器进行如下实现：

```
1  // pages/component/childComponent/index.js
2  Component({
3
4      /**组件的方法列表*/
5      methods: {
6
7          // 测试监听器：当 setData 更新数据时
8          updateData() {
9              // 直接赋值不会被监听到
10             /* this.data.numberA = 10
11             this.data.numberB = 20 */
12
13             // 更新numberA和numberB的属性值
14             this.setData({
15                 numberA: 1,
16                 numberB: 2
17             })
18
19
20         },
21
22     },
23
24     // 用于定义监听器
25     observers: {
26
27         // 在 numberA 或者 numberB 被 setData 时，执行这个函数
28         'numberA, numberB' (numberA, numberB) {
29             console.log('监听到数字的变化', numberA, numberB)
30             this.setData({ sum: numberA*1 + numberB*1})
31         }
32     }
33 })
```

```
31     },  
32  
33     },  
34  
35  })
```

```
1  <button bind:tap="updateData" type="warn">监听setData改变属性值</button>  
2  <view>numberA和numberB总和为: {{ sum || 0 }}</view>
```

3. 监听对象某个属性，注意：用 `.` 分隔对象属性路径。

```
1  // pages/component/childComponent/index.js  
2  Component({  
3  
4    /** 组件的内部数据，和 properties 一同用于组件的模板渲染 */  
5    data: {  
6      keyword: '',  
7      query: {  
8        username: ''  
9      }  
10   },  
11  
12   /**组件的方法列表*/  
13   methods: {  
14     // 测试监听器：当 setData 更新数据时  
15     updateData() {  
16       // 更新对象属性值  
17       this.setData({  
18         'query.username': '涛姐'  
19       })  
20     },  
21   },  
22  
23   // 用于定义监听器  
24   observers: {  
25  
26     // 当前query对象中的 username 属性值变化，注意：不要少了引号  
27     'query.username' (username) {  
28       console.log('监听到username变化', username)  
29     },  
30  
31   },  
32  
33 })
```


4. 如果需要监听对象中所有子属性的变化，可以使用通配符 `**`。

```
1 // 监听对象中所有子属性的变化，可以使用通配符 **
2 'query.**' (query) { // 注意：少了**，写 'query' (){}是监听不到对象属性变化的
3   console.log('监听到query变化', query)
4 }
```

组件间代码共享 behaviors

`behaviors` 是用于组件间代码共享的特性，类似于 Vue.js 编程语言中的“mixins”。

每个 `behavior` 可以包含一组属性、数据、生命周期函数和方法。

组件引用它时，它的属性、数据和方法会被合并到组件中，生命周期函数也会在对应时机被调用。

每个组件可以引用多个 `behavior`，`behavior` 也可以引用其它 `behavior`。

详细的参数含义和使用请参考 [Behavior 参考文档](#)。

behaviors使用方式

1. 创建 `miniprogram/behaviors/my-behavior.js` 文件，注册一个 `behavior`

```
1 // 注册一个 behavior，接受一个 Object 类型的参数。
2 export default Behavior({ // 首字母大写
3
4   // 可引用其他 behavior
5   behaviors: [],
6
7   // 定义接收父组件的属性
8   properties: {
9     myBehaviorProperty: {
10      type: String,
11      value: '我是behavior的Prop'
12    }
13  },
14
15   // 定义当前组件的数据属性
16   data: {
17     myBehaviorData: {
18       name: '我是behavior的Data'
19     }
20  },
21
22   methods: {
```

```
23
24     myBehaviorMethod () {
25         console.log('我是behavior方法')
26     }
27
28 },
29
30 // 其他组件/页面选项
31
32 })
```

2. 在 pages/component/childComponent/index.js 中导入 Behavior 实例文件并引用

```
1 // pages/component/childComponent/index.js
2 // 1、导入 behavior
3 import myBehavior from '@/behaviors/my-behavior.js'
4
5 Component({
6
7     // 2、引用 behavior
8     behaviors: [myBehavior],
9
10    // 组件生命周期函数-在组件实例进入页面节点树时执行
11    attached() {
12        // console.log('当前组件实例', this)
13        this.myBehaviorMethod()
14    },
15
16 })
```

3. 在组件模板中访问 my-behavior.js 中的数据和方法

```
1 <!--pages/component/childComponent/index.wxml-->
2
3 <view>===== 组件引用 behavior 代码共享 =====</view>
4 <view>
5 访问 my-behavior.js 中的数据:
6 {{ myBehaviorProperty }} , {{ myBehaviorData.name }}
7 </view>
8 <button bind:tap="myBehaviorMethod">调用 my-behavior.js 中的方法</button>
9
```

4. 也可以在页面js中导入并引用 my-behavior.js

```
1 // pages/component/component.js
2
3 // 导入 behavior
4 import myBehavior from '@/behaviors/my-behavior.js'
5
6 Page({
7   // 引用 behavior
8   behaviors: [myBehavior],
9
10  data: {
11  },
12
13 })
```

同名字段的覆盖和组合规则

组件和它引用的 `behavior` 中可以包含同名的字段，对这些字段的处理方法如下：

- 如果有同名的属性 (properties) 或方法 (methods):

1. 若组件本身有这个属性或方法，则组件内的属性或方法会覆盖 `behavior` 中的同名属性或方法；
2. 若组件本身无这个属性或方法，则在组件的 `behaviors` 字段中定义靠后的 `behavior` 的属性或方法会覆盖靠前的同名属性或方法；
3. 在 2 的基础上，若存在嵌套引用 `behavior` 的情况，则规则为：引用者 `behavior` 覆盖 被引用的 `behavior` 中的同名属性或方法。

- 如果有同名的数据字段 (data):

- 若同名的数据字段都是对象类型，会进行对象合并；
- 其余情况会进行数据覆盖，覆盖规则为：引用者 `behavior` > 被引用的 `behavior`、靠后的 `behavior` > 靠前的 `behavior`。（优先级高的覆盖优先级低的，最大的为优先级最高）

- 生命周期函数和 `observers` 不会相互覆盖，而是在对应触发时机被逐个调用：

- 对于不同的生命周期函数之间，遵循组件生命周期函数的执行顺序；
- 对于同种生命周期函数和同字段 `observers`，遵循如下规则：
 - `behavior` 优先于组件执行；
 - 被引用的 `behavior` 优先于 引用者 `behavior` 执行；
 - 靠前的 `behavior` 优先于 靠后的 `behavior` 执行；
- 如果同一个 `behavior` 被一个组件多次引用，它定义的生命周期函数和 `observers` 不会重复执行。

组件样式

组件样式注意事项

组件对应 `wxss/scss` 文件的样式，只对组件 `wxml` 内的节点生效。

编写组件样式时，需要注意以下几点：

- 组件和引用组件的页面不能使用id选择器（`#a`）、属性选择器（`[a]`）和标签名选择器，请改用class选择器。
- 组件和引用组件的页面中使用后代选择器（`.a .b`）在一些极端情况下会有非预期的表现，如遇：请避免使用。
- 子元素选择器（`.a>.b`）只能用于 `view` 组件与其子节点之间，用于其他组件可能导致非预期的情况。
- 继承样式，如 `font`、`color`，会从组件外继承到组件内。
- `app.wxss`、`app.scss` 中定义的全局样式，默认对自定义组件无效（除非更改组件样式隔离选项）。

总结：在开发项目时，建议统一使用 `class` 选择器。

示例代码如下：

```
1  /* pages/component/childComponent/index.wxss */
2
3
4  /*
5    1、自定义组件中，不允许id选择器、属性选择器，标签选择器（支持，尽量不用防止不支持）
6  */
7  /* id选择器不支持 */
8  #title {
9    color: blue;
10 }
11
12 /* id选择器不支持 */
13 [id=title] {
14   color: blue;
15 }
16
17 /* 标签选择器有效，尽量不用 */
18 text {
19   background-color: red !important;
20 }
21
22
23 /* 2、子元素选择器（`.a > .b`）只能用于 `view` 组件与其子节点之间，用于其他组件可能导致非预期的情况。*/
24 .title > .sub-title { /* 不支持 */
25   color: blue;
26 }
27
28 /*
29   3、继承样式，如 font、color，会从组件外继承到组件内
```

```
30  示例：在 component.wxml 中添加如下代码，则childComponent组件中所有字体颜色和大小都发生改
    变了
31  <view style="color: blue; font-size: 23px">
32    <child-component />
33  </view>
34  */
35
36  /*
37  4、app.scss/app.wxss中定义的全局样式：默认情况下对自定义组件无效，需要在自定义组件中设置样式隔
    离级别才有效
38  */
```

```
1  <!--pages/component/childComponent/index.wxml-->
2
3  <view>===== 自定义组件样式 =====</view>
4  <text id="title" class="title">
5    <text class="sub-title">测试自定义组件支持的css选择器</text>
6  </text>
7
8  <view class="flex-center">引用app.scss定义的全局样式</view>
```

组件样式隔离

默认情况下，自定义组件的样式只受到自定义组件 wxss 的影响。

如果我们需要组件使用者中定义的样式(如:页面.wxss)或全局样式(app.wxss) 能够在组件中生效，则需要组件中指定特殊的样式隔离选项 `styleIsolation`。

即：在自定义组件 JS 文件中使用 `options: { styleIsolation: xxx}`，或者在组件 JSON 文件中使用 `styleIsolation: xxx` 选项配置以下值：

- `isolated`（默认值）表示启用样式隔离，在自定义组件内外，使用 `class` 指定的样式将不会相互影响；
- `apply-shared`（一般需要app.wxss全局样式时配置）表示页面 wxss 样式将影响到自定义组件，但自定义组件 wxss 中指定的样式不会影响页面；
- `shared`（基本不用）表示页面 wxss 样式将影响到自定义组件，自定义组件 wxss 中指定的样式也会影响页面和其他设置了 `apply-shared` 或 `shared` 的自定义组件。

注意：也可使用 `addGlobalClass: true` 等价于 `styleIsolation: 'apply-shared'`

示例代码如下：

1. 组件 JS 文件中，使用 `options` 选项来指定 `styleIsolation` 或 `addGlobalClass` 属性进行配置样式隔离级别

```

1
2 Component({
3   options: {
4     /**
5      * styleIsolation 指定样式隔离级别: isolated / apply-shared / shared
6      */
7     // styleIsolation: 'apply-shared',
8     addGlobalClass: true // 等价于 `styleIsolation: 'apply-shared'`
9   },
10
11 })
12

```

2. 组件 JSON 文件中，指定 `styleIsolation` 属性进行配置样式隔离级别

```

1 {
2   "styleIsolation": "apply-shared",
3   "component": true,
4   "usingComponents": {}
5 }

```

组件生命周期

组件的生命周期，指的是组件自身的一些函数，这些函数在特殊的时间点或遇到一些特殊的框架事件时被自动触发。

组件生命周期函数：

生命周期	参数	描述
created	无	在组件实例刚刚被创建时执行 1、不能进行 setData 更新属性字段（3.5.5+可以） 2、给组件的 this 添加一些自定义的属性字段
attached	无	在组件实例进入页面节点树时执行，组件还未渲染 1、这个函数很有用，绝大数用于初始化操作(如发送请求获取初始数据) 2、可调用 setData 更新属性字段
ready	无	在组件在视图层布局完成后执行，组件渲染完成
moved	无	在组件实例被移动到节点树另一个位置时执行
detached	无	在组件实例被从页面节点树移除时执行，组件被销毁 1、退出页面时，会触发页面中每一个自定义组件的 detached 函数 2、一般用于释放、清理资源的工作
error	Object Error	每当组件方法抛出错误时执行

定义生命周期函数可以直接定义在 `Component` 构造器的第一级参数中；也可以在 `lifetimes` 字段内进行声明（这是推荐的方式，其优先级最高）。

示例代码如下：

1. 创建自定义组件 `components/lifetimes/index`

```
1 // components/lifetimes/index.js
2 Component({
3
4   data: {
5     msg: '我是子组件'
6   },
7
8   // 组件生命周期声明对象
9   lifetimes: {
10
11     // 组件实例已创建，一般用于添加一些自定义属性字段，提供给组件实例使用
12     created() {
13       console.log('created 组件实例已创建')
14
15       // 3.5.5 版本created钩子中以前不能调用 this.setData 修改字段值， 3.5.5 版本开始
16       // 可以调用。
17       // this.setData({msg: 'hello world'})
18
19       // 添加自定义属性字段
20       this.isOpen = true
21     },
22
23     // 组件模板已挂载到页面中，绝大多数初始化工作可以在这个时机进行
24     attached() {
25       console.log('attached 组件模板已初始并挂载到页面中，还未渲染出来')
26
27       console.log('访问组件自定义属性', this.isOpen)
28
29       // 可调用setData修改状态值
30       this.setData({ msg: 'hello mengxuegu' })
31     },
32
33     // 组件被销毁，一般用于释放资源
34     detached() {
35       console.log('detached 组件被销毁')
36     },
37   },
38
39 })
```


2. 在页面中导入和引用组件，然后测试销毁组件

- 页面json中导入组件

```
1 {
2   "usingComponents": {
3     "lifetimes": "/components/lifetimes/index"
4   }
5 }
```

- 页面模板中引用组件，点击按钮销毁组件

```
1 <!-- 组件生命周期-组件 -->
2 <!-- isShow 为false时，触发组件中的 detached 生命周期函数 -->
3 <lifetimes wx:if="{{ isShow }}" />
4 <button bind:tap="handleDetachedComponent" type="warn">销毁组件</button>
```

```
1 Page({
2
3   data: {
4     isShow: true
5   },
6
7   // 点击销毁组件
8   handleDetachedComponent() {
9     this.setData({
10       isShow: !this.data.isShow
11     })
12   }
13
14 })
```

组件所在页面生命周期

组件还有一些特殊的生命周期，它们并非与组件有很强的关联。但有时候组件需要在页面被展示或被隐藏时，进行一些组件内部的业务处理，这样的生命周期称为“组件所在页面的生命周期”，在组件的 `pageLifetimes` 字段中定义。

其中可用的页面生命周期函数：

生命周期	参数	描述
show	无	组件所在的页面被展示时执行
hide	无	组件所在的页面被隐藏时执行
resize	Object Size	组件所在的页面尺寸变化时执行

注意：自定义 tabBar 的 pageLifetime 不会触发。

```
1 // components/lifetimes/index.js
2 Component({
3
4   // 组件生命周期声明对象
5   lifetimes: {
6   },
7
8   //组件所在页面的生命周期声明对象
9   pageLifetimes: {
10
11     show() {
12       console.log('show 组件所在页面被展示')
13     },
14
15     hide() {
16       console.log('hide 组件所在页面被隐藏')
17     },
18
19     resize(obj) {
20       // 在页面json中配置 "pageOrientation": "auto" 开启旋转功能后，旋转屏幕可触发此钩子
21       console.log('resize 组件所在页面尺寸已变化', obj)
22     }
23   }
24 })
25
26
```

使用 Component 构造器构造页面

因为 Component 构造方法功能比 Page 构造方法功能更加强大，如果能使用 Component 构造页面，就可以实现更加复杂的页面逻辑开发。

而事实上，小程序的页面也可以视为自定义组件。因此，页面也可以使用 `Component` 构造器构造。

使用 `Component` 构造器构造页面，注意事项：

- 页面.json 文件中必须包含 `usingComponents` 字段。
- 页面.js 文件中使用 `Component` 构造器构造，配置项需要和组件配置项一致。
- 可以使用组件的 `properties` 选项来接收其他页面传递过来的参数，然后通过 `this.data` 访问参数值。

如：访问页面 `/pages/index/index?paramA=123¶mB=xyz`，如果 `properties` 选项中有声明属性 `paramA` 或 `paramB`，则它们会被赋值为 `123` 或 `xyz`，然后可通过 `this.data.paramA` 或 `this.data.paramB` 访问参数值。

- 小程序框架提供的以 `on` 开头的：页面生命周期函数、事件监听函数，放在 `methods` 选项中

示例代码如下：

```
1 // pages/component2/component2.js
2 /**
3  * 使用Component构建页面：
4  * 1. 在页面.json文件中必须包含 "usingComponents": {}
5  * 2. 小程序框架提供的以on开头的：页面生命周期函数、事件监听函数，放在 methods 选项中
6  * 3. 路由参数通过properties声明接收，再通过this.data访问参数，
7  * 如 pages/index/index?paramA=10&paramB=meng
8  */
9 Component({
10
11   data: {
12     msg: '此页面是通过Component构建的'
13   },
14
15   // 接收路由参数
16   properties: {
17     paramA: Number,
18     paramB: String,
19   },
20
21
22   methods: {
23     // 小程序框架提供的以on开头的：页面生命周期函数、事件监听函数，放在 methods 选项中
24     onLoad: function() {
25       const {paramA, paramB} = this.data
26       console.log('获取到路由参数', paramA, paramB)
27     }
28   }
29
30
31 })
```

测试：

- 添加编译模式：设置启动参数为 `paramA=10¶mB=meng`

组件

编译 预览 真机调试 清缓存

+

添加编译模式 ?

添加方式

手动输入

模式名称

Component构建页面

启动页面

pages/component2/component2

启动参数

paramA=10¶mB=meng

路由参数

小程序模式

默认

进入场景

默认

编译设置

☐ 下次编译时模拟更新 (需 1.9.90 及以上基础库版本)

局部编译

☐ 本地开发、预览 / 真机调试时，仅编译以下页面 ?

pages/component2/component2

+

删除该模式

取消

确定

第十章 小程序常用 API

路由跳转 API

wx.navigateTo(Object object)

wx.navigateTo(Object object) 方法用于保留当前页面，跳转到应用内的某个页面。但是不能跳到 **tabbar** 页面。使用 [wx.navigateBack](#) 可以返回到原页面。小程序中页面栈最多十层。

object 参数:

属性	类型	默认值	必填	说明
url	string		是	需要跳转的应用内非 tabBar 的页面的路径 (代码包路径), 路径后可以带参数。参数与路径之间使用 <code>?</code> 分隔, 参数键与参数值用 <code>=</code> 相连, 不同参数用 <code>&</code> 分隔; 如 <code>'path?key=value&key2=value2'</code>
events	Object		否	页面间通信接口, 用于监听被打开页面发送到当前页面的数据。基础库 2.7.3 开始支持。
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数 (调用成功、失败都会执行)

- object.success 回调函数
 - 参数Object res

属性	类型	说明
eventChannel	EventChannel	和被打开页面进行通信

示例代码:

```

1  wx.navigateTo({
2    url: '/pages/user/user?id=1',
3    success (res) {
4      // 通过eventChannel向被打开页面传送数据
5      res.eventChannel.emit('userInfoEvent', {username: 'mengxuegu'})
6    },
7    events: {
8      // 添加一个监听器, 获取被打开页面传送到当前页面的数据
9      acceptData (data) {
10        console.log(data)
11      },
12      //...其他监听器
13    }
14  })
  
```

```

1  // pages/user/user.js
2  Page({
3
4    onLoad(option) {
5      console.log(option.id)
6      const eventChannel = this.getOpenerEventChannel()
7      // 监听userInfoEvent事件, 获取上一页面通过eventChannel传送到当前页面的数据
8      eventChannel.on('userInfoEvent', (data) => {
  
```

```

9      console.log(data)
10    })
11
12    // 向上一页面监听器传递数据
13    //eventChannel.emit('acceptData', {data: 'test'})
14  },
15
16  // 点击后退，传递数据给上一页面
17  navTo() {
18    const eventChannel = this.getOpenerEventChannel()
19    // 向上一页面监听器传递数据
20    eventChannel.emit('acceptData', {data: 'test'})
21    // 后退
22    wx.navigateTo()
23  }
24
25  })

```

```

1 <button bind:tap="navTo">
2   后退
3 </button>

```

wx.navigateBack(Object object)

wx.navigateBack(Object object) 方法用于关闭当前页面，返回上一页面或多级页面。可通过 [getCurrentPages](#) 获取当前的页面栈，决定需要返回几层。

object 参数：

属性	类型	默认值	必填	说明
delta	number	1	否	返回的页面数，如果 delta 大于现有页面数，则返回到首页。
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

示例代码：

```

1 // 注意：调用 navigateTo 跳转时，调用该方法的页面会被加入堆栈，而 redirectTo 方法则不会。见下
  // 方示例代码
2
3 // 此处是A页面

```

```
4 wx.navigateTo({
5   url: 'B?id=1'
6 })
7
8 // 此处是B页面
9 wx.navigateTo({
10  url: 'C?id=1'
11 })
12
13 // 在C页面内 navigateBack, 将返回A页面
14 wx.navigateBack({
15   delta: 2
16 })
```

wx.switchTab(Object object)

`wx.switchTab(Object object)` 方法用于跳转到 tabBar 页面，并关闭其他所有非 tabBar 页面

object 参数:

属性	类型	默认值	必填	说明
url	string		是	需要跳转的 tabBar 页面的路径 (代码包路径) (需在 app.json 的 tabBar 字段定义的页面)， 路径后面不能带参数。
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

示例代码:

```
1 wx.switchTab({
2   url: '/pages/index/index', // 不支持带参数 ?paramA=11
3   success() {
4     console.log('跳转成功')
5   }
6 })
```

消息提示 API

参考: <https://developers.weixin.qq.com/miniprogram/dev/api/ui/interaction/wx.showToast.html>

- wx.showToast 显示消息提示框

wx.hideToast 隐藏消息提示框

```
1 wx.showToast({
2   title: '成功',
3   icon: 'success',
4   duration: 2000
5 })
```

- wx.showModal 显示消息对话框

```
1 wx.showModal({
2   title: '提示',
3   content: '这是一个对话框弹窗',
4   success (res) {
5     if (res.confirm) {
6       console.log('用户点击确定')
7     } else if (res.cancel) {
8       console.log('用户点击取消')
9     }
10  }
11 })
```

- wx.showLoading 显示 loading 提示框。需主动调用 wx.hideLoading 才能关闭提示框

wx.hideLoading 才能关闭 loading 提示框

```
1 wx.showLoading({
2   title: '加载中',
3 })
4
5 setTimeout(function () {
6   wx.hideLoading() // 隐藏
7 }, 2000)
```

- wx.showActionSheet 显示操作菜单

```
1 wx.showActionSheet({
2   itemList: ['选项1', '选项2', '选项3'],
3   success (res) {
4     // res.tapIndex 是用户点击的按钮序号，从上到下的顺序，从0开始
5     console.log(res.tapIndex)
6   }
7 })
```

wx.redirectTo(Object object)

wx.redirectTo(Object object) 方法用于关闭当前页面，跳转到应用内的某个页面。但是不允许跳转到 **tabbar** 页面。

object 参数:

属性	类型	默认值	必填	说明
url	string		是	需要跳转的应用内非 tabBar 的页面的路径 (代码包路径), 路径后可以带参数。参数与路径之间使用 <code>?</code> 分隔, 参数键与参数值用 <code>=</code> 相连, 不同参数用 <code>&</code> 分隔; 如 <code>'path?key=value&key2=value2'</code>
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数 (调用成功、失败都会执行)

object 参数:

```
1 wx.redirectTo({  
2   url: '/pages/user/user?paramA=111',  
3 })
```

```
1 // pages/user/user  
2 Page({  
3   onLoad (option) {  
4     console.log(option.paramA) // 111  
5   }  
6 })
```

wx.reLaunch(Object object)

wx.reLaunch(Object object) 方法用于关闭所有页面，打开到应用内的某个页面

object 参数:

属性	类型	默认值	必填	说明
url	string		是	需要跳转的应用内页面路径， 路径后可以带参数 。参数与路径之间使用?分隔，参数键与参数值用=相连，不同参数用&分隔；如 'path?key=value&key2=value2'
success	function		否	接口调用成功的回调函数
fail	function		否	接口调用失败的回调函数
complete	function		否	接口调用结束的回调函数（调用成功、失败都会执行）

object 参数:

```
1 wx.reLaunch({  
2   url: '/pages/user/user?paramA=111',  
3 })
```

```
1 // pages/user/user  
2 Page({  
3   onLoad (option) {  
4     console.log(option.paramA) // 111  
5   }  
6 })
```

第十一章 小程序分包加载

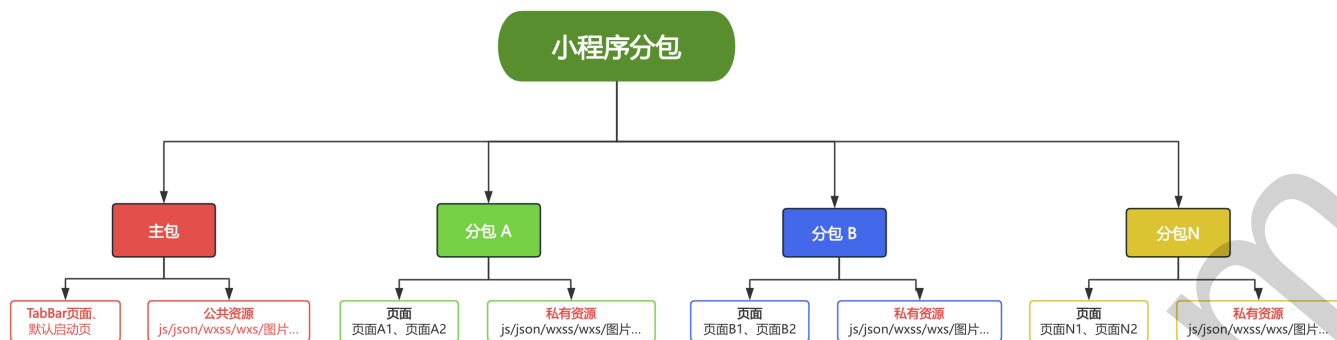
分包加载说明

分包前：小程序项目代码通常由很多页面、组件和资源组成；如果小程序项目代码体积过大，会导致用户在首次打开小程序加载过慢，从而影响用户的使用体验。

分包后：开发者可以将小程序划分成不同的子包，在构建时打包成不同的分包，用户在使用时按需进行加载。

在构建小程序分包项目时，构建会输出**一个主包和多个分包**。

- **主包：**就是放置默认启动页面 或 **TabBar** 页面，以及一些所有分包都需用到公共资源/JS 脚本；
- **分包：**就是开发者根据功能模块的不同进行配置划分，放置**非TabBar**页面和分包的私有资源。



对小程序进行分包的优点：

- 可以优化小程序首次启动的下载时间
- 在多团队共同开发时可以更好的解耦协作

分包加载机制：在小程序启动时，默认会下载主包并启动主包内页面，当用户进入分包内某个页面时，客户端会把对应分包下载下来，下载完成后再进行展示。

目前小程序分包大小有以下限制：

- 单个分包/主包大小不能超过 **2M**
- 整个小程序所有分包大小不超过 **30M**（服务商代开发的小程序不超过 20M）

使用分包

主包和分包的引用原则：

- 主包不能引用分包内的私有资源
- 分包之间不能相互引用私有资源
- 分包可以引用主包内的公有资源

举例，支持分包的小程序目录结构如下：

```
1 | app.js
2 | app.json
3 | app.wxss
4 | pages          // 主包所有页面
5 |   | shopping
6 |   | order-list
7 | packageA       // 第1个分包
8 |   | pages      // 第1个分包所有页面
9 |     | list
```

```
10 |      |   └─ cart
11 | └─ packageB      // 第2个分包
12 |     |   └─ pages  // 第2个分包所有页面
13 |         |   └─ apple
14 |         |   └─ banana
15 | └─ utils
```

在 `app.json` 文件中通过 `subPackages` 或 `subpackages` 字段声明项目分包结构：

```
1  {
2    "pages": [ // 配置主包中的所有页面路径
3      "pages/shopping/shopping",
4      "pages/order-list/order-list"
5    ],
6    "subPackages": [ // 配置分包的结构
7      {
8        // 第1个分包的根目录，会在项目根目录下创建packages/goodsPackage目录
9        "root": "packages/goodsPackage",
10       "name": "goodsPackage", // 当前分包的别名，分包预下载时可以使用。非必配。
11       "pages": [ // 配置当前分包下的所有页面路径，会在packages/goodsPackage下创建pages存放
下面页面
12         "pages/list/list",
13         "pages/cart/cart"
14       ]
15     },
16     {
17       "root": "packages/payPackage", // 第2个分包的根目录
18       "name": "payPackage", // 当前分包的别名
19       "pages": [ // 配置当前分包下的所有页面路径
20         "pages/payment/payment"
21       ]
22     }
23   ]
24 }
```

以上分包配置保存后，会自动创建对应的分包目录和页面到项目结构中，如下：

components

packages

goodsPackage

pages

cart

list

payPackage

pages

payment

pages

utils

查看分包大小：点击开发工具右上角的【详情】》【基本信息】中的本地代码查看分包大小：

上传

版本管理

详情

消息

基本信息

性能质量

本地设置

项目配置

梦学谷

发布状态 未发布

AppId

修改

复制

项目名称

修改

本地目录

复制

文件系统

复制

本地代码 1149 KB

代码依赖分析

路径 大小

主包 1143.4 KB

/packages/payPackage/ 0.9 KB

/packages/goodsPackage/ 3.7 KB

上次预览 1118 KB (上星期二09:50)

上次上传 无

独立分包

独立分包说明

1. 独立分包是什么：

- 独立分包本质也是一种分包，只不过它是与普通分包区别是：它可以独立于主包和其他分包而单独运行。

2. 普通分包和独立分包的区别：是否需要依赖主包才能运行

- 普通分包：必须依赖主包才能运行。访问普通分包时，必须先下载主包，然后从主包中跳转到分包内。
- 独立分包：不需要下载主包，可以独立运行。也就是可以直接打开独立分包。

3. 独立分包使用场景：

- 开发者可以根据需求，将某些具有一定功能独立性的页面配置到独立分包中，原因如下：
 - 当小程序从普通的分包页面启动时，需要首先下载主包；
 - 而独立分包不依赖主包即可运行，可以很大程度上提升分包页面的启动速度。

4. 注意：一个小程序中可以有多个独立分包。

配置独立分包

在 app.json 的 subpackages 字段中对应的分包配置项中，定义 independent 字段声明对应分包为独立分包。

```
1  "subPackages": [  
2    {  
3      "root": "packages/goodsPackage",  
4      "pages": [  
5        "pages/list/list",  
6        "pages/cart/cart"  
7      ]  
8    },  
9    {  
10     "root": "packages/payPackage",  
11     "name": "pay",  
12     "pages": [  
13       "pages/payment/payment"  
14     ],  
15     "independent": true // 开启当前为独立分包  
16   }  
17 ]
```


独立分包注意事项

独立分包属于分包的一种。普通分包的所有限制都对独立分包有效。

此外，使用独立分包时要注意：

- 独立分包中不能引用主包和其他分包中的资源，包括 js 文件、template、wxss、自定义组件等。
- 独立分包之间不能相互引用私有资源。
- 主包中的 `app.wxss` 对独立分包无效，应避免在独立分包页面中使用 `app.wxss` 中的样式；

分包预下载

什么是分包预下载：

分包预下载是指在进入小程序某个页面时，由框架自动预下载可能需要的分包，从而提升进入后续分包页面时的启动速度。

什么时候触发分包预下载：

- 当进入指定页面时，会触发分包的预下载行为。
- 对应下载规则，在 `app.json` 中使用 `preloadRule` 选项进行配置分包预下载规则

`preloadRule` 中，`key` 是页面路径，`value` 是进入此页面的预下载配置，每个配置有以下几项：

字段	类型	必填	默认值	说明
packages	StringArray	是	无	进入页面后预下载分包的 <code>root</code> 或 <code>name</code> 。 <code>__APP__</code> 表示主包。
network	String	否	wifi	在指定网络下预下载，可选值为： <code>all</code> ：不限网络， <code>wifi</code> ：（默认值）仅wifi下预下载

`app.json` 中示例配置如下：

```
1 // 分包预下载规格配置
2 "preloadRule": {
3   // key 指定页面路径：当进入此页面时会触发预下载
4   // value 指定规格对象
5   "pages/shopping/shopping": {
6     "packages": [ "goodsPackage" ], // 指定预下载的分包路径(root值)或者分包别名(name值)
7     // network指定什么网络模式下进行预下载，可取值：wifi（默认值）、all（所有网络都进行预下
      载，2G/3G/4G/5G/wifi等）
    }
```

```
8      "network": "all"
9    }
10  },
11  "subPackages": [
12    {
13      "root": "packages/goodsPackage",
14      "name": "goodsPackage",
15      "pages": [
16        "pages/list/list",
17        "pages/cart/cart"
18      ]
19    },
20    {
21      "root": "packages/payPackage",
22      "name": "payPackage",
23      "pages": [
24        "pages/payment/payment"
25      ],
26      "independent": true
27    }
28  ],
```

测试：

1. 编译代码，进入到 pages/shopping/shopping 页面后，查看调试器的控制台会打印以下内容：

```
1 preloadSubpackages: goodsPackage
2 preloadSubpackages: success
```

2. 当 "network": "wifi" 时切换不同网络类型 2G,



重新编译后，进入 shopping 页面，查看控制台没有打印 `preloadSubpackages: success`，说明未加载。

```
preloadSubpackages: goodsPackage
```

```
preloadSubpackages fail: network type does not match, '2g' !== 'wifi'
```

第十二章 微信小程序云开发-数据库

微信云开发是微信团队联合腾讯云推出的专业的小程序开发服务。开发者无需搭建后端服务器，可免鉴权直接使用平台提供的 API 进行业务开发。

开通微信云开发

1. 点击开发工具上方的 云开发



2. 在开通窗口输入云开发：环境名称，首月免费、次月开始19.9元/月



3. 等待一会开通成功后，会打开云开发页面：

当前环境 mxg-order-miniapp ▾

环境 ID mxg-order-miniapp-8e4h2e616d

配额: 基础配额 有效期至2024-09-16

续费 用量提醒与冻结 充值与账单 更多

云函数

在云端运行的代码，无需维护鉴权逻辑，直接调用微信官方接口，只需实现业务逻辑

前往创建云函数 >

数据库

免部署运维，可视化管理的文档型数据库，支持数据读写，数据库事务，自动备份回档等功能。

前往创建集合 >

存储

云端文件存储，自带 CDN 加速，支持在前端直接上传/下载，可在云开发控制台可视化。

前往上传文件 >

云后台

开箱即用的 Web 端管理系统，内置多种管理端应用，并支持自定义开发。

前往开通 >

云模板

开箱即用的 Web 端管理系统，内置多种管理端应用，并支持自定义开发。

前往开通 >

项目云开发初始化

1. 打开云开发控制台，点击【设置】环境设置】进行复制的 环境ID



2. 云开发初始化：打开项目的 `app.js` 文件，`onLaunch` 钩子函数中进行云开发初始化

官网: <https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/init.html>

wx.cloud.init 方法只能调用一次，多次调用时只有第一次调用生效。

```
1  onLaunch() {  
2  
3      // 云开发初始化  
4      wx.cloud.init({  
5          env: 'mxg-order-miniapp-8e4h2e6xxx', // 云开发控制台的【设置】环境设置中的`环境  
ID`进行复制到这里  
6          traceUser: true // 是否在将用户访问记录到用户管理中，在云开发控制台的【运营分析】用户  
访问】中可见  
7      })  
8  
9  }
```

云数据库入门开发

官网: <https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/database/init.html>

数据库概念

云开发提供了一个 JSON 数据库，顾名思义，数据库中的每条记录都是一个 JSON 格式的对象。一个数据库可以有多个集合（相当于关系型数据中的表），集合可看做一个 JSON 数组，数组中的每个对象就是一条记录，记录的格式是 JSON 对象。

关系型数据库和 JSON 数据库的概念对应关系如下表：

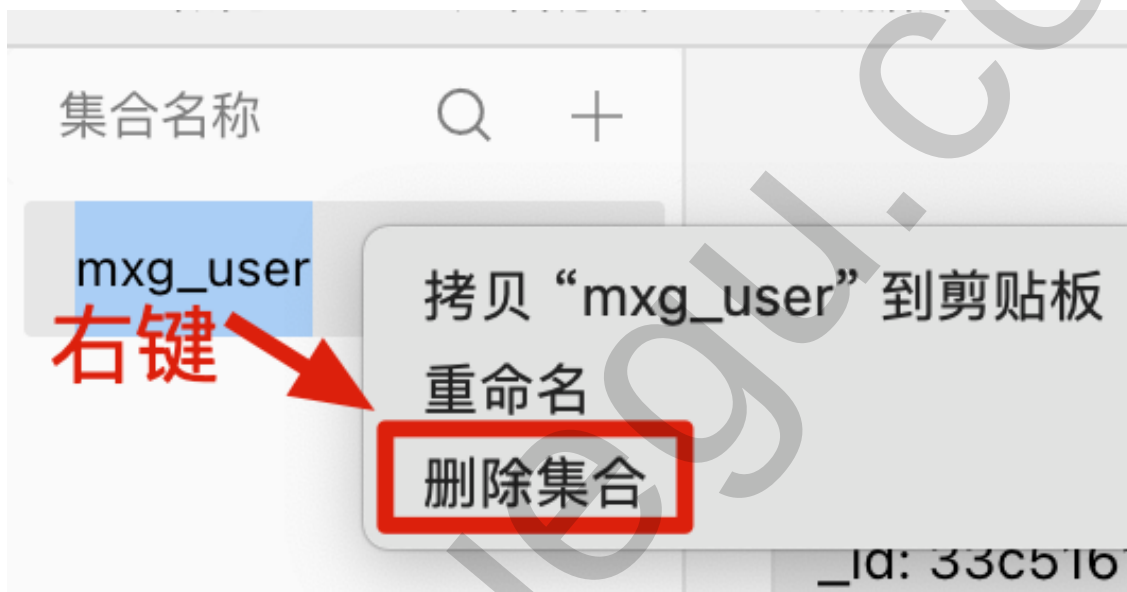
关系型	文档型
数据库 database	数据库 database
表 table	集合 collection
行 row	记录 record / doc
列 column	字段 field

控制台操作: 集合(数据表)、记录

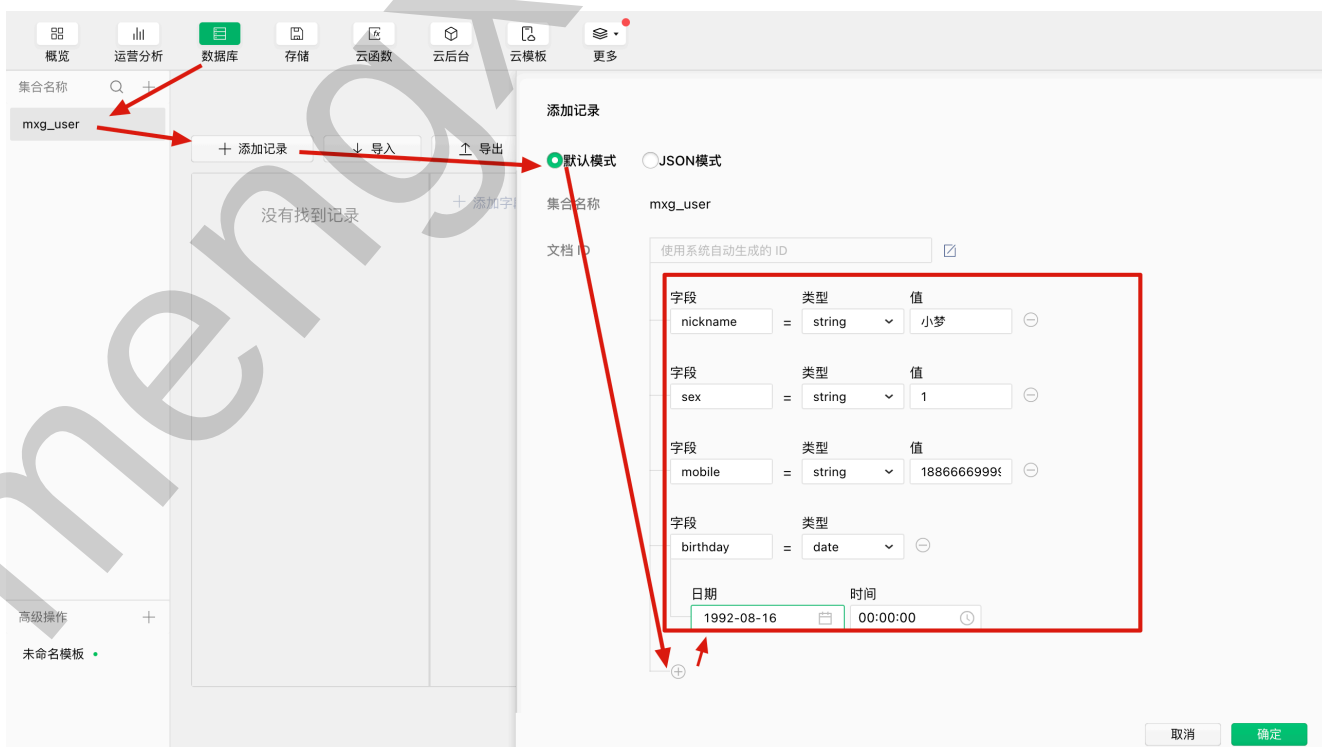
1. 新增集合（数据表）：



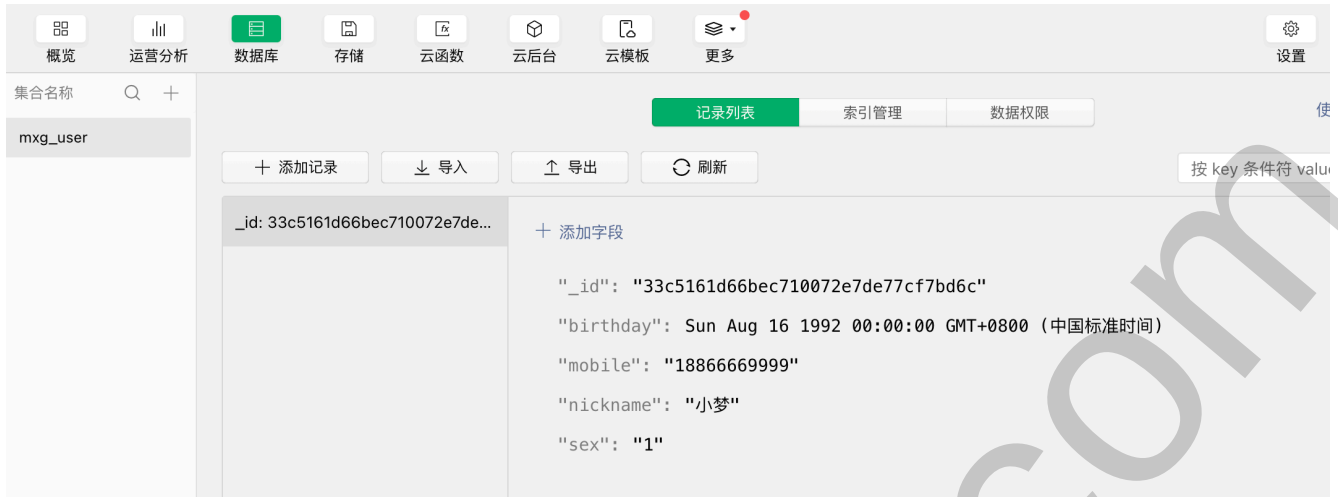
2. 删除集合（数据表）：右键单击集合名，然后点击删除集合



3. 向集合中添加记录（文档）：



添加记录后的效果：



4. 删除记录：右键单击 记录_id，然后点击删除记录



简单查询API

1. 查询：传统回调风格

```
1 // pages/index/index.js
2 Page({
3
4   /**生命周期函数--监听页面加载*/
5   onLoad() {
6     this.loadUserInfo()
7   },
8
9   /** 查询用户数据 */
10  loadUserInfo() {
11
12    // 查询：回调函数风格
13    wx.cloud.database().collection('mxg_user')
14      .get({
15        success: (res) => { // 请求成功,
16          // res是一个对象，其中有data属性是一个数组
17          console.log('请求成功', res)
18        },
19        fail: (err) => { // 请求失败
```

```
20     console.log('请求失败', err)
21   }
22 })
23
24 },
25
26 })
```

查询结果如下：

注意：要将集合的数据权限更改为：所有用户可读，参见上一小节

请求成功 ▼ {data: Array(1), errMsg: "collection.get:ok"} ⓘ

▼ data: Array(1)

▼ 0:

▶ birthday: Sun Aug 16 1992 00:00:00 GMT+0800 (中国标准时间) {}
▶ mobile: "18866669999"
▶ nickname: "小梦"
▶ sex: "1"
▶ _id: "33c5161d66bec710072e7de77cf7bd6c"

要将数据权限设置为：
所有用户可读

▶ __proto__: Object

length: 1

nv_length: (...)

▶ __proto__: Array(0)

errMsg: "collection.get:ok"

▶ __proto__: Object

2. 查询：ES6 Promise 风格

```
1  // pages/index/index.js
2  Page({
3
4    /**生命周期函数--监听页面加载*/
5    onLoad() {
6      this.loadUserInfo()
7    },
8
9    /** 查询用户数据 */
10   async loadUserInfo() {
11     // 查询：回调函数风格
12     // ....
13
14     // 查询：ES6 Promise 风格
15     wx.cloud.database().collection('mxg_user')
16       .get().then((res) => { // res.data是一个数组
17         console.log('请求成功2', res)
18       }).catch((err) => {
19         console.log('请求失败2', err)
20       })
21
22     // 查询：ES6 Promise 简写，注意方法名前加 async
```



```
23     try {  
24         /* const res = await wx.cloud.database().collection('mxg_user').get()  
25         console.log('请求成功3', res) */  
26         // 从响应结果中直接解构出 data 属性  
27         const { data } = await wx.cloud.database().collection('mxg_user').get()  
28         console.log('请求成功3', data)  
29     } catch (err) {  
30         console.log('请求失败3', err)  
31     }  
32  
33 },  
34  
35 })
```

数据库的增删查改 API 都同时支持 回调风格和 Promise 风格调用。

数据权限控制

问题：如果数据库中有数据，在小程序中通过API 进行查询数据，请求成功后的响应结果却没有数据，如下：

请求成功 ▾ {data: Array(0), errMsg: "collection.get:ok"}

▶ data: []

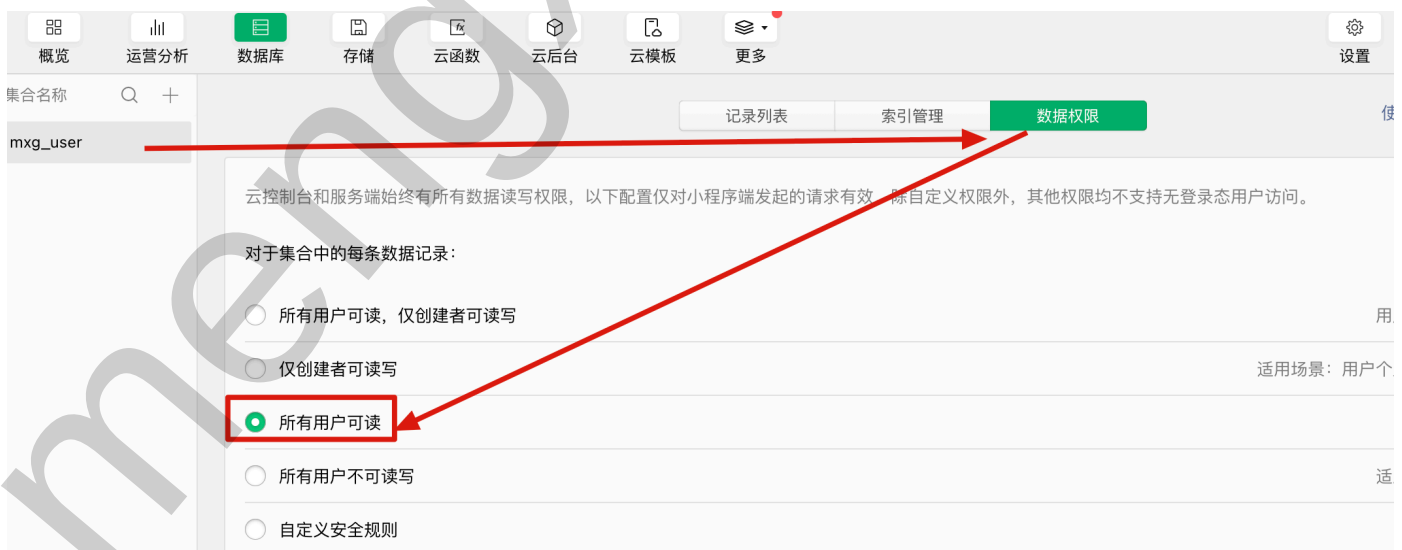
未查询到数据，原因：权限问题

errMsg: "collection.get:ok"

▶ __proto__: Object

原因：默认新增记录的数据权限仅创建者可读写；而使用控制台新增记录的创建者是管理员，而小程序的是使用用户，并不是同一用户，所以默认不能查询到数据。

解决：数据权限更改为：所有用户可读



数据库增删改查 API

查询记录 doc/where

1. 使用 `doc` 方法，传递唯一ID `_id` 值，查询一条数据，注意：响应结果中的 `data` 是一个对象，不是数组

```
1      data: {
2        userInfo: {}
3      },
4
5      async loadById() {
6        try {
7          // data 是一个对象，不是数组
8          const { data } = await wx.cloud.database().collection('mxg_user')
9            .doc('33c5161d66bec710072e7de77cf7bd6c') // 根据文档 _id 值查询一条数
10           .get()
11
12          // 更新状态数据
13          this.setData({userInfo: data})
14
15          console.log("data", data)
16        } catch (err) {
17          console.log('请求失败3', err)
18        }
19      },
20
21      onLoad() {
22        // this.loadUserInfo()
23        this.loadById()
24      },
```

data 值:

```
1  {
2    _id: "33c5161d66bec710072e7de77cf7bd6c",
3    birthday: Sun Aug 16 1992 00:00:00 GMT+0800 (中国标准时间),
4    mobile: "18866669999",
5    nickname: "小梦",
6    sex: "1"
7  }
```

2. 使用 `where` 方法，实现多条件查询

```
1  /**
2    * where 条件简单查询
```

```
3      */
4      async queryByWhere() {
5          // 响应对象中的 data 属性值是一个数组
6          const { data } = await wx.cloud.database().collection('mxg_user')
7              .where({ // where接收的对象中的字段条件是且的关系, 要同时满足
8                  nickname: '小梦',
9                  sex: '2'
10             })
11             .get()
12
13         console.log('where条件查询结果', data)
14     },
15
16     onLoad() {
17         this.queryByWhere()
18     },
19 }
```

新增记录 add

使用 `add` 方法往集合中插入一条记录。如果传入的记录对象没有 `_id` 字段, 则由后台自动生成 `_id`; 若指定了 `_id`, 则不能与已有记录冲突。

1. 比如: 我们想新增一个用户

```
1  /** 新增记录 */
2  addUser() {
3      // 新增: 回调风格
4      /* wx.cloud.database().collection('mxg_user')
5         .add({
6             // data 属性指定需新增的 JSON 数据
7             data: {
8                 //可自定义 _id 值, 未指定数据库会自动分配; 如果指定的id值存在, 会报错: multiple
9                 write,duplicate key
10                 // _id: '111',
11                 nickname: '涛姐',
12                 mobile: '19988880000',
13                 sex: '2',
14                 birthday: new Date('2000-10-08')
15             },
16             success: (res) => {
17                 // res 是一个对象, 其中有 _id 属性标记刚创建的记录的 id
18                 console.log('新增成功, _id为', res._id)
19             },
20             fail: (err) => {
21                 console.error('新增失败', err)
22             },
23         })
24     },
25 }
```

```
22         complete: (val) => {
23             console.log('新增完成', val)
24         }
25     }) */
26
27     // 新增: Promise风格
28     wx.cloud.database().collection('mxg_user').add({
29         data: {
30             nickname: '周董',
31             mobile: '16611110000',
32             sex: '1',
33             birthday: new Date('1888-07-08')
34         }
35     }).then((res) => {
36         console.log('新增成功2, _id为', res._id)
37     }).catch((err) => {
38         console.error('新增失败2, _id为', err)
39     })
40
41 },
42
43 /**
44  * 生命周期函数--监听页面加载
45  */
46 onLoad() {
47     // this.loadUserInfo()
48     // this.loadById()
49     this.addUser()
50 },
```

2. 如果开发工具 Console 报如下错:

Error: errCode: -502003 database permission denied | errMsg: Permission denied

```
✖ 新增失败 Error: errCode: -502003 database permission denied | errMsg: Permission denied 请前往云开发AI小助手:
oud.tencent.com/cloud-admin#/copilot?q=DATABASE_PERMISSION_DENIED
    at new b (<anonymous>:1:244855)
    at success (<anonymous>:1:256655)
    (env: macOS,mp,1.06.2402040; lib: 3.4.9)

Error: errCode: -502003 database permission denied | errMsg: Permission denied 请前往云开发AI小助手查看问题:
m/cloud-admin#/copilot?q=DATABASE_PERMISSION_DENIED
    at new b (<anonymous>:1:244855)
    at success (<anonymous>:1:256655)
```

原因: 没有数据库操作权限

解决: 修改每条数据记录的权限为: 所有用户可读, 仅创建者可读写



更新记录 update

使用 `update` 方法修改集合中的某些记录。

注意事项：API 调用成功不一定代表想要更新的记录已被更新，比如有可能指定的 `where` 筛选条件只能筛选出 0 条匹配的记录，所以会得到更新 API 调用成功但其实没有记录被更新的情况，这种情况可以通过 `stats.updated` 看出来

比如：更新一条数据，通过 `doc` 方法指定的记录 `_id` 作为更新条件

```
1  /** 更新记录 */
2  updateUserById() {
3      // 更新记录：默认只允许更新自己新增的记录，更新他人新增的记录响应结果是0条
4      wx.cloud.database().collection('mxg_user')
5          .doc('33c5161d66bec710072e7de77cf7bd6c') // doc方法指定 _id 值作为更新条件
6          .update({
7              data: {
8                  nickname: '小小梦',
9                  sex: '2'
10             }
11         }).then((res) => {
12             // res.stats.updated 是更新的记录数，如果是0：表示没有记录被更新，或没有权限更新成功
13             console.log(`更新成功${res.stats.updated}条`)
14         }).catch((err) => {
15             console.log('更新失败', err)
16         })
17 },
18
```

更新记录时的数据权限问题：

默认情况下，只允许更新自己新增的记录，不允许更新他人新增的记录：会响应成功，但 `stats.updated` 结果是 0。

如果要修改他人新增的记录，需要借助云函数才可以操作，后续会进行讲解，当前暂不讲解。

替换记录 set

如果需要替换一条记录，可以在记录上使用 `set` 方法，替换更新意味着用传入的对象替换指定的记录：

如果指定 ID 的记录不存在，则会自动创建该记录，该记录将拥有指定的 ID。

```
1  /** 替换记录 set */
2  async setUserById() {
3    try {
4      const res = await wx.cloud.database().collection('mxg_user')
5        .doc('33c5161d66bf03390736b13f5a63512e') // id存在则替换set中指定的data数据，不存在则新增set中指定的data数据
6        .set({
7          data: { // 替换为下面的数据
8            nickname: '小新',
9            sex: '2'
10         }
11       })
12       // res.stats.updated 是替换的记录数，res.stats.created 是id不存在时新增的记录数
13       console.log('替换记录成功', res)
14     } catch (err) {
15       console.log('替换记录失败', err)
16     }
17   },
```

替换前后的记录，如下图所示：

+ 添加字段 替换前 _id: 33c5161d66bec710072e7de... _id: f380561066bef0df07331cf4... _id: 33c5161d66bf03390736b13... _id: 8df0e92f66bf04bb0733bde... _id: ef45322e66bf05ed07390e6... _id: 33c5161d66bf064f07374c5... "_id": "33c5161d66bf03390736b13f5a63512e" "_openid": "oYkTM5QWJVH7GcPta_vf1vMWQsA0" "birthday": Sun Oct 08 2000 08:00:00 GMT+0800 (中国标准时间) "mobile": "19988880000" "nickname": "涛姐" "sex": "2"
+ 添加字段 替换后 _id: 33c5161d66bec710072e7de... _id: f380561066bef0df07331cf4... _id: 33c5161d66bf03390736b13... _id: 8df0e92f66bf04bb0733bde... _id: ef45322e66bf05ed07390e6... "_id": "33c5161d66bf03390736b13f5a63512e" "_openid": "oYkTM5QWJVH7GcPta_vf1vMWQsA0" "nickname": "小新" "sex": "2"

删除数据 `remove`

使用 `remove` 方法删除集合中的记录。

默认只能配合 `doc` 方法删除指定ID的一条记录；如果要全量删除记录或者删除他人新增的记录，需要使用云函数才可以操作。

```
1  async removeById() {
2    try {
3      const res = await wx.cloud.database().collection('mxg_user')
4        .doc('33c5161d66bec710072e7de77cf7bd6c') // 指定 _id 值作为条件
5        .remove()
6
7      console.log(`删除记录成功, ${res.stats.removed}条`, )
8    } catch (err) {
9      console.error('删除记录失败', err)
10     /*
11      报如下错: 请确认记录存在, 并且有相应的写权限
12      Error: document.remove:fail document.remove:fail cannot remove document
13      with _id 33c5...41fd2,
14      please make sure that the document exists and you have the corresponding
15      Write permission
16      */
17   }
18 },
```

查询排序 `orderBy`

使用 `orderBy(fieldPath: string, order: string)` 方法,

- `fieldPath` 参数1: 指定字段名
- `order` 参数2: 只能取 `asc` 或 `desc` (asc升序, desc降序)

示例: 先按birthday升序排列后, 如果birthday有相同值, 再按sex降序

```
1  async testOrderBy() {
2    const { data } = await wx.cloud.database().collection('mxg_user')
3    .orderBy('birthday', 'asc') // 按birthday升序排列, asc 升序 (越大越靠后), desc 降序
4    (越大越靠前)
5    .orderBy('sex', 'desc') // 先按birthday升序排列后, 如果birthday有相同值, 再按sex降序
6    排列
7    .get()
8
9    console.log('按birthday升序排序结果', data)
10  },
```


分页查询 `count`、`limit`、`skip`

基于 `count` 方法统计满足条件的总记录数

```
1  /**
2   * count 统计满足条件的总记录数
3   * @param {number} pageSize 每页显示多少条数据
4   */
5  async queryCount(pageSize = 3) {
6    // 响应对象中的 total 为统计的记录数
7    const { total } = await wx.cloud.database().collection('mxg_user').count()
8    // 计算有余数则商值+1页, 没有则就是商值
9    // 计算总页数 = 总记录数 % 每页显示的条数 > 0 ? 总记录数 / 每页显示的条数 + 1 : 总记录数 / 每页显示的条数
10   const totalPages = total % pageSize > 0 ? Math.floor(total / pageSize) + 1 :
Math.floor(total / pageSize)
11   console.log('count统计满足条件的总记录数', total, totalPages)
12 },
```

查询指定条数的记录 `limit`

`limit` 在小程序端默认及最大上限为 20, 在云函数端默认及最大上限为 100

- 示例: 每次最多查询3条记录返回

```
1  /** limit指定每次查询最多返回的记录数 */
2  async testLimit() {
3    const { data } = await wx.cloud.database().collection('mxg_user')
4      .limit(3) // 每次查询最多返回3条数据
5      .get()
6
7    console.log('limit查询指定条数的记录', data)
8  },
```

查询指定序列的后一条结果开始返回 `skip`


```
1  /** skip 查询指定序列的后一条结果开始返回 */
2  async testSkip() {
3      const { data } = await wx.cloud.database().collection('mxg_user')
4          .skip(3) // 从指定序列的下一个序列号开始返回数据，如：跳过前3条，从第4条开始返回
5          .get()
6
7      console.log('skip指定序列的后一条记录开始返回', data)
8  },
```

实现分页查询

实现分页查询，通过 `limit` 指定每页显示多少条数据；`skip` 指定每页从第几条往后开始查询，第n页：
(pageNum - 1) * pageSize

比如：每页查询10条数据，pageSize=10

- 第1页 `limit(10).skip(0)`
- 第2页 `limit(10).skip(10)`
- 第3页 `limit(10).skip(20)`
- 第4页 `limit(10).skip(30)`
- 第 pageNum 页 `limit(pageSize).skip( (pageNum-1) * pageSize )`

```
1  Page({
2
3      /**
4       * 分页查询：基于limit和skip实现
5       * @param {number} pageNum 当前页码
6       * @param {number} pageSize 每页显示多少条数据
7       */
8      async pageQuery(pageNum = 1, pageSize=3) {
9          const { data } = await wx.cloud.database().collection('mxg_user')
10             // .limit(3) // 每次最多查询3条（每页显示3条）
11             // .skip(0) // 第1页：序列为0，第2页：序列为3，第3页：序列为6，第n页：序列为 (n - 1) *
limit值
12             .limit(pageSize)
13             .skip( (pageNum - 1) * pageSize ) // 第n页：序列为 (n - 1) * limit值
14             .get()
15
16             console.log('skip指定序列的后一条记录开始返回', data)
17     },
18
19     onLoad() {
20         this.pageQuery(2)
21     },
22
23
24 })
```

高级查询操作符 `command`

基于 `where` 方法，并且配合暴露在 `db.command` 对象上的操作符，我们可以构造复杂的查询条件完成复杂的查询任务

`db.command` 操作符参考：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/reference-sdk-api/database/Command.html>

查询比较操作符 `eq/neq/lt/lte/gt/gte/in/nin`

比较操作符	说明
<code>eq</code>	等于
<code>neq</code>	不等于
<code>lt</code>	小于
<code>lte</code>	小于或等于
<code>gt</code>	大于
<code>gte</code>	大于或等于
<code>in</code>	字段值在给定数组中
<code>nin</code>	字段值不在给定数组中

示例：

```
1  /**
2   * where + db.command 查询指令，实现复杂条件查询
3   * 比较操作符使用
4   */
5  async queryByWhereCommand() {
6    const db = wx.cloud.database()
7    const _ = db.command
8    const { data } = await db.collection('mxg_user')
9      .where({
10        // age: _.eq(18) // 等价于 age: 18, 即: "age 等于 18" 的条件
11        // age: _.neq(18) // neq 用于指定一个“不等于”条件, 即: "age 不等于 18" 的条件
12        // age: _.gt(18) // gt 方法用于指定一个“大于”条件, 此处是 "age 大于 18" 的条件
13        // age: _.gte(18) // gte 方法用于指定一个“大于或等于”条件, 此处是 "age 大于或等于 18" 的条件
```

```

14      // 问题：如果看数据库中有满足条件的age值，但未查询到数据，可能数据类型是string，要改为
      number类型
15      // age: _.lt(18) // lt 方法用于指定一个 "小于" 条件，此处是 "age 小于 18" 的条件
16      // age: _.lte(18) // lte 方法用于指定一个 "小于或等于" 条件，此处是 "age 小于或等于
      18" 的条件
17      // age: _.in([18, 26]) // in 方法接收一个数组，用于字段值在给定数组中的条件，此
      处"age 等于 18 或 26"的条件
18      age: _.nin([18, 26]) // nin 方法接收一个数组，用于字段值不在给定数组中的条件，此
      处"age 不等于 18 和 26"的条件
19    })
20    .get()
21    console.log('command查询结果', data)
22  }

```

查询逻辑操作符 and/or/not/nor

逻辑操作符	说明
and	逻辑 "与" 的关系，同时满足多个条件
or	逻辑 "或" 的关系，满足多个条件中的一个
not	逻辑 "非" 的关系，不满足指定的条件
nor	逻辑 "都不" 的关系，不满足指定的所有条件。记录中没有对应的字段，则默认满足条件

示例：

and 逻辑与操作符使用：

- 查询 sex 等于 '2' 且 age 大于 18的用户
- 查询 age 大于等于18 且小于30 的用户

```

1  /**
2   * where + db.command 查询指令，实现复杂条件查询
3   * and 逻辑与操作符使用
4   */
5  async queryByCommandAnd() {
6    const db = wx.cloud.database()
7    const _ = db.command
8    const { data } = await db.collection('mxg_user')
9    .where(
10     // 查询 sex='2' 且 age > 18岁的用户
11     /* // 方式一： and接收一个数组[]操作符后，直接传递给where方法
12     _.and([
13       {sex: '2'},
14       {age: _.gt(18)}
15     ]) */

```

```
16      /* { // 方式二：更简洁的写法，上述条件等价于下方，对象中各字段隐式组成了“与”的关系
17         sex: '2',
18         age: _.gt(18)
19     }, */
20
21     // 查询 age 大于等于18且小于30的用户
22     {
23         // age: _.and(_.gte(18), _.lt(30)) //方式1：前置写法
24         // age: _.gte(18).and(_.lt(30)) //方式2：流式写法
25         age: _.gte(18).lt(30) //方式3：链式写法
26     }
27 )
28 .get()
29
30 console.log('command.and查询结果', data)
31 },
```

or 逻辑或操作符使用：

- 单字段的或查询：age 小于 18 或大于 30 的用户
- 多字段的或查询：age大于18岁 或 sex等于'1' 的用户

```
1  /**
2   * where + db.command 查询指令，实现复杂条件查询
3   * or 逻辑或操作符使用
4   */
5  async queryByCommandOr() {
6      const db = wx.cloud.database()
7      const _ = db.command
8      const { data } = await db.collection('mxg_user')
9      // 单字段的或查询：age 小于 18 或大于 30 的用户
10     /* .where({
11         // age: _.or(_.lt(18), _.gt(30)) // 方式1：前置写法，接收多个参数
12         // age: _.or([_.lt(18), _.gt(30)]) // 方式2：前置写法，接收一个数组参数
13         age: _.lt(18).or(_.gt(30)) // 方式3：流式写法
14     }) */
15
16     // 多字段的或查询：age大于18岁 或 sex等于'1' 的用户
17     .where(_.or([ // or接收一个对象数组，数组中每个对象条件取"或"的关系，
18         {
19             age: _.gt(18)
20         },
21         {
22             sex: '1' // 等价 sex: _.eq('1')
23         }
24     ]))
25
26     .get()
```

```
27
28     console.log('command.or查询结果', data)
29 },
```

not 逻辑非操作符使用：

- 查询 age 不等于 18的用户

```
1  /**
2   * where + db.command 查询指令，实现复杂条件查询
3   * not逻辑“非”操作符
4   */
5  async queryByCommandNot() {
6      const db = wx.cloud.database()
7      const _ = db.command
8      const { data } = await db.collection('mxg_user')
9          // 查询 age 不等于 18的用户
10         .where({
11             age: _.not(_.eq(18)) // 等价于 age: _.neq(18)
12         })
13         .get()
14
15     console.log('command.not查询结果', data)
16 },
```

nor 逻辑都不操作符使用：

- 查询 age 不小于18且不大于30 的用户 (即：[18~30]间的数值)

注意：nor 接收一个数组，会筛选出不存在 age 字段的记录；如果要求 age 字段必须存在，可以用 exists 指令

```
1  /**
2   * where + db.command 查询指令，实现复杂条件查询
3   * nor 逻辑“都不”操作符
4   */
5  async queryByCommandNor() {
6      const db = wx.cloud.database()
7      const _ = db.command
8      const { data } = await db.collection('mxg_user')
9          // 查询 age 不小于18且不大于30 的用户 (即：[18~30]间的数值)
10         .where({
11             // 注意：nor 接收一个数组，会筛选出不存在 age 字段的记录，如果要求 age 字段必须存在，
12             // 可以用 exists 指令
13             // age: _.nor([_.lt(18), _.gt(30)])
14         })
15         .get()
16 },
```

```
13     age: _.exists(true).and(_.nor([_.lt(18), _.gt(30)])) // 必须存在age字段且age
    不小于18且不大于30 的用户
14   })
15   .get()
16
17   console.log('command.not查询结果', data)
18 },
```

模糊查询 RegExp

官网参考: <https://developers.weixin.qq.com/miniprogram/dev/wxcloud/reference-sdk-api/database/Databases.RegExp.html>

1. 模糊查询单字段: 查询 `mxg_user` 集合中 `nickname` 字段中包含 `梦` 的记录

```
1 // 模糊查询
2 async queryByRegExp() {
3   const keyword = '梦'
4   // 数据库正则对象
5   const db = wx.cloud.database()
6   const { data } = await db.collection('mxg_user')
7   // 模糊查询 nickname 字段中包含keyword("梦")的记录
8   .where({
9     // nickname是字段名
10    nickname: db.RegExp({
11      regexp: keyword, // 搜索关键字
12      options: 'i', // 'i' 不区分大小写
13    })
14  })
15  .get()
16
17  console.log(`模糊搜索包含 【${keyword}】 的记录有`, data)
18 },
```

2. 模糊查询多字段, 如: 查询 `mxg_user` 集合中 `nickname` 或 `mobile` 字段中包含 `11` 的记录

```
1 // 模糊查询
2 async queryByRegExp2() {
3   const keyword = '11'
4   // 数据库正则对象
5   const db = wx.cloud.database()
6   const _ = db.command
7   const { data } = await db.collection('mxg_user')
8   // 模糊查询 `nickname` 或 `mobile` 字段中包含 `11` 的记录
9   .where(
10     _.or(
11       [
```

```
12      {
13          nickname: db.RegExp({
14              regexp: keyword,
15              options: 'i'
16          })
17      },
18      {
19          mobile: db.RegExp({
20              regexp: keyword,
21              options: 'i'
22          })
23      },
24  ]
25  )
26  )
27  .get()
28
29  console.log(`模糊搜索包含【${keyword}】的记录有`, data)
30  },
```

第十三章 小程序云开发-云函数

云函数调试: <https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/functions/local-debug.html>

什么是云函数

云函数是一段运行在云端的代码，无需管理服务器，在开发工具内编写、一键上传部署即可运行后端代码。

小程序内提供了专门用于云函数调用的 API。开发者可以在云函数内使用 `wx-server-sdk` 提供的 `getWXContext` 方法获取到每次调用的上下文（appid、openid 等），无需维护复杂的鉴权机制，即可获取天然可信任的用户登录态（openid）。

操作	云数据库	云函数
运行环境	运行在小程序中	运行在微信云端Node.js环境
功能丰富性	只能对云数据库增删改查	功能强大丰富(云数据库、文件上传下载等)
查询数据	每次最多返回20条	每次最多返回100条
更新数据	只允许更新自己创建的数据	允许更新所有人的数据
删除数据	只允许删除自己创建的数据	允许删除所有人的数据

云函数环境配置

1. 在项目根目录创建一个 `cloudfunctions` 文件夹，作为云函数的本地根目录。
2. 在项目根目录找到 `project.config.json` 文件，然后在文件中新增 `cloudfunctionRoot` 字段，值指定云函数的本地根目录 `cloudfunctions`

```
1 {
2   "cloudfunctionRoot": "cloudfunctions/"
3 }
```

project.config.json •

project.config.json > ...

```
1 {
2   "cloudfunctionRoot": "cloudfunctions/",
3   "description": "项目配置文件",
4   ...
5 }
```

3. 接着，我们在云函数根目录(`cloudfunctions`)上右键，在右键菜单中，可以选择创建一个新的 Node.js 云函数，具体如下一节

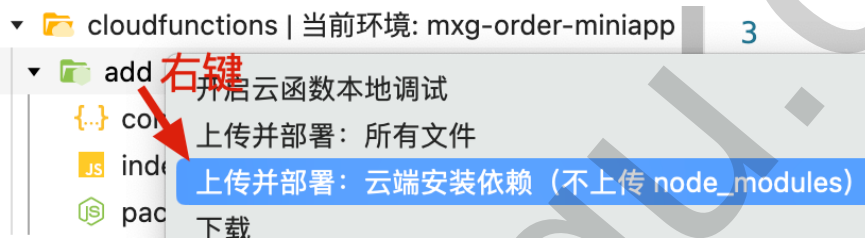
创建并上传部署云函数

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `helloworld`，即：
`cloudfunctions/helloworld/index.js`

```
1 // index.js 是云函数入口文件,
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境,
5 cloud.DYNAMIC_CURRENT_ENV 引用当前环境ID
6
7 /**
8  * 云函数入口函数，云函数被调用时会执行该文件导出的 main 方法
9  * 可以通过 wx-server-sdk 提供的 getWXContext 方法获取到每次调用的上下文（小程序appid、用户
10  * 登录态openid 等）
11  * @param {*} event 包含了调用端（小程序端）调用该函数时传过来的参数；
12  * @param {*} context
13  */
14 exports.main = async (event, context) => {
15   /* unionId 主体ID：是一个用户对于同一主体下的微信小程序 / 公众号 / APP的标识，开发者需要在微信开
16   放平台下绑定相同账号的主体。
17   在微信生态中，开发者可通过 unionId 实现多个小程序、公众号、甚至APP 之间的数据互通，通过
18   unionId 来区分用户的唯一性。*/
19   // 一个人，同主体下只有1个 unionId，可以有多个openid（因为同主体下可以有多个微信小程序）
```

```
16  const {OPENID, APPID, UNIONID} = cloud.getWXContext()
17  const { userInfo, a, b } = event
18  const sum = a + b
19
20  return {
21    event,
22    OPENID,
23    APPID,
24    UNIONID,
25    sum
26  }
27 }
```

- 注意：每次修改了云函数，都必需右键上传并部署，修改后的代码才会生效。



调用云函数

```
1  /**
2   * 调用云函数add
3   */
4  async testCallCloudFunction() {
5    // 调用云函数：回调方式
6    wx.cloud.callFunction({
7      name: 'helloworld', // 需调用的云函数名
8      data: { // 传递给云函数的参数，有大小限制（100K）
9        a: 1,
10       b: 2
11      },
12      success: (res) => {
13        console.log('success', res)
14      },
15      fail: (err) => {
16        console.log("fail", err)
17      }
18    })
19    // 调用云函数：Promise 方式
20    /* wx.cloud.callFunction({
21      name: 'helloworld', // 需调用的云函数名
22      data: {
23        a: 1,
24        b: 2
25      }
26    }).then(res => {
```

```
27     console.log('成功2', res)
28   }).catch(err => {
29     console.error('失败2', err)
30   })  */
31
32   // 调用云函数: async + await方式
33   try {
34     const res = await wx.cloud.callFunction({
35       name: 'helloworld', // 需调用的云函数名
36       data: {
37         a: 1,
38         b: 2
39       }
40     })
41     console.log('成功3', res)
42   } catch (err) {
43     console.error('失败3', err)
44   }
45 },
```

云函数实现新增记录

创建并上传部署云函数：实现新增记录

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `add01` 即： `cloudfunctions/add01/index.js`

```
1  // 云函数入口文件
2  const cloud = require('wx-server-sdk')
3
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6  // 云函数入口函数
7  exports.main = async (event, context) => {
8    const { addData } = event
9
10   return await cloud.database().collection('mxg_user')
11     .add({
12       data: addData
13     })
14
15 }
```

调用云函数新增记录

```
1  /** 调用云函数新增记录 */
```

```
2  async addByCloudFun() {
3      // 要新增的数据
4      const addData = {
5          nickname: '小快乐',
6          sex: '1',
7          birthday: new Date('1989-12-19'),
8          mobile: '17788889999'
9      }
10
11     // 调用云函数
12     const res = await wx.cloud.callFunction({
13         name: 'add01',
14         data: {
15             addData // addData: addData
16         }
17     })
18
19     console.log('云函数新增结果', res)
20 }
```

云函数实现更新记录

云函数拥有最高权限进行更新记录，即可以更新所有用户的记录。小程序中只允许更新自己创建的记录。

创建并上传部署云函数：实现更新记录

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `update01`，即：
`cloudfunctions/update01/index.js`

```
1  // 云函数入口文件
2  const cloud = require('wx-server-sdk')
3
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6  // 云函数入口函数
7  exports.main = async (event, context) => {
8      const {id, mobile} = event
9
10     return await cloud.database().collection('mxg_user')
11         .doc(id)
12         .where({ // 云函数支持where条件更新
13             _id: id
14         })
15         .update({
16             data: {
17                 mobile
18             }
19         })
```

```
20  
21 }
```

调用云函数更新记录

```
1  /** 调用云函数更新记录 */  
2  async updateByCloudFun() {  
3      const res = await wx.cloud.callFunction({  
4          name: 'update01', // 调用的云函数名  
5          data: { // 向云函数传递的数据  
6              id: 'f380561066bef0df07331cf412bfb5aa',  
7              mobile: '19911112222'  
8          }  
9      })  
10     console.log('云函数更新结果', res)  
11 }
```

云函数实现删除记录

云函数拥有最高权限进行删除记录，即可以删除所有用户的记录。小程序中只允许删除自己创建的记录。

创建并上传部署云函数：实现删除记录

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `remove01`，
即：`cloudfunctions/remove01/index.js`

```
1  // 云函数入口文件  
2  const cloud = require('wx-server-sdk')  
3  
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境  
5  
6  // 云函数入口函数  
7  exports.main = async (event, context) => {  
8      const { id } = event  
9  
10     return await cloud.database().collection('mxg_user')  
11     // .doc(id)  
12     .where({ // 云函数支持where条件删除  
13         _id: id  
14     })  
15     .remove()  
16  
17 }
```

调用云函数删除记录

```
1  /** 调用云函数删除记录 */
2  async removeByCloudFun() {
3      const res = await wx.cloud.callFunction({
4          name: 'remove01',
5          data: {
6              id: 'f380561066bef0df07331cf412bfb5aa'
7          }
8      })
9      console.log('云函数删除结果', res)
10 }
```

云函数实现查询记录

云函数中实现查询操作，每次最多可查询100条记录。小程序端每次最多查询20条记录。

创建并上传部署云函数：实现查询记录

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `get01`，
即：`cloudfunctions/get01/index.js`

```
1  // 云函数入口文件
2  const cloud = require('wx-server-sdk')
3
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6  // 云函数入口函数
7  exports.main = async (event, context) => {
8      // 通过云函数实现查询云数据库的集合记录
9      // 直接return返回响应结果，调用方通过 result.data 获取数组数据
10     // return await cloud.database().collection('mxg_user').get()
11
12     // 解构 data 数组数据返回，调用方通过 result 获取的就是数组数据
13     const { data } = await cloud.database().collection('mxg_user').get()
14     return data
15 }
```

调用云函数查询记录

```
1 // 调用云函数实现：查询记录
2 async getUserByCloudFun() {
3     const { result } = await wx.cloud.callFunction({
4         name: 'get01'
5     })
6     console.log('云函数查询记录响应结果', result)
7 }
```

云函数实现联表查询(多表关联查询) lookup

官网参考：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/reference-sdk-api/database/aggregate/Aggregate.lookup.html>

什么是联表查询：

- 就是传统数据库存中(如 MySQL) 中的多表关系查询，如：left join。
- 对应云开发数据库联表查询要基于 lookup 进行查询，在同一个数据库下的一个指定的集合做 left outer join (左外连接)。对该阶段的每一个输入记录，lookup 会在该记录中增加一个数组字段，该数组是被联表中满足匹配条件的记录列表。lookup 会将连接后的结果输出给下个阶段。

业务需求：

- 一件商品对应多种规格，如一件衣服的规格多种（颜色：红色、黑色，尺寸：S码、L码）
- 商品和规格的关系是一对多的关系，
- 所以数据库设计：创建一个 goods 商品集合和一个 goods_specs 规格集合，goods_specs 集合中定义一个 goodsId 字段，用于关联 goods 集合进行联表查询。

创建云数据库集合

1. 新增 goods 商品集合，添加以下记录：

```
1 { "_id": "25e993b76704950e0e600f7f438ee6e6", "content": "纯天然香甜可口豆  
浆", "goodsName": "豆浆" }
2 { "_id": "33c5161d670495480e607ee35c4697da", "content": "<p>大自然的搬运工  
</p>", "goodsName": "矿泉水" }
```

2. 新增 goods_specs 商品规格集合，添加以下记录：


```
1 { "_id": "8df0e92f670495950e52f941321f76f4", "specsName": "大  
怀", "goodsId": "25e993b76704950e0e600f7f438ee6e6" }  
2 { "_id": "ef45322e670495bf0e57aa0a232b4dcb", "specsName": "小  
杯", "goodsId": "25e993b76704950e0e600f7f438ee6e6" }
```

创建并上传部署云函数：实现联表查询商品和规格信息

- 在 `cloudfunctions` 上单击右键，点击 **新建 Node.js 云函数** `moreTableQuery`，
即：`cloudfunctions/moreTableQuery/index.js`

```
1 // 云函数入口文件  
2 const cloud = require('wx-server-sdk')  
3  
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境  
5  
6 // 云函数入口函数  
7 exports.main = async (event, context) => {  
8   // 多表关系查询商品及规格信息，goods 关联 goods_specs，  
9   // 下面等价于伪 SQL 操作：SELECT g.*, gs.* FROM goods g LEFT JOIN goods_specs gs  
10  ON g._id = gs.goodsId  
11  const db = cloud.database()  
12  return db.collection('goods').aggregate()  
13    .lookup({  
14      localField: '_id', // goods 集合中的主键id，用于关联使用  
15      from: 'goods_specs', // 要关联的集合名  
16      foreignField: 'goodsId', // goods_specs中用于关联的字段名 (goods_specs中的外  
17      键)  
18      as: 'specsList', // goods_specs查询出来的记录所显示的字段名  
19    })  
20    .match({ // 此处必需用 match 条件筛选 (等价于 where)  
21      // 筛选goods集合中_id=xxx的记录  
22      _id: '25e993b76704950e0e600f7f438ee6e6'  
23    })  
24    .end() // 不要少了end  
25  }
```

- 调用云函数 联表查询记录

```
1  /**
2   * 基于 lookup联表查询 (多表关联查询):
3   */
4  async queryMoreTableByLookup() {
5    // 调用云函数: lookup联表查询
6    const res = await wx.cloud.callFunction({
7      name: 'moreTableQuery'
8    })
9    console.log('lookup联表查询结果', res)
10  },
```

打印结果

```
{errMsg: "cloud.callFunction:ok", result: {}, requestID: "a6321766-2783-43e9-99b7-cc54844d9d60"}
  errMsg: "cloud.callFunction:ok"
  requestID: "a6321766-2783-43e9-99b7-cc54844d9d60"
  result:
    errMsg: "collection.aggregate:ok"
    list: Array(1)
      0:
        content: "纯天然香甜可口豆浆"
        goodsName: "豆浆"
        specslist: Array(2)
          0: { _id: "8df0e92f670495950e52f941321f76f4", specsName: "大杯", goodsId: "25e993b76704950e0e600f7f438ee6e6" }
          1: { _id: "ef45322e670495bf0e57aa0a232b4dcb", specsName: "小林", goodsId: "25e993b76704950e0e600f7f438ee6e6" }
        length: 2
```

第十四章 小程序云开发-存储

官网参考: <https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/storage.html>

云存储提供高可用、高稳定、强安全的云端存储服务, 支持任意数量和形式的非结构化数据存储, 如视频和图片, 并在控制台进行可视化管理。云存储包含以下功能:

- 存储管理: 支持文件夹, 方便文件归类。支持文件的上传、删除、移动、下载、搜索等, 并可以查看文件的详情信息
- 权限设置: 支持基础权限设置和高级安全规则权限控制
- 上传管理: 在这里可以查看文件上传历史、进度及状态
- 文件搜索: 支持文件前缀名称及子目录文件的搜索
- 组件支持: 支持在 `image`、`audio`、`video` 等组件中传入云文件 ID

在小程序端可以分别调用 `wx.cloud.uploadFile` 和 `wx.cloud.downloadFile` 完成上传和下载云文件操作。

上传文件到云端(图片/视频/文档等)

在小程序端可调用 `wx.cloud.uploadFile` 方法进行上传:

注意: 存储的文件名 (`cloudPath`) 命名限制:

- 不能为空
- 不能以/开头
- 不能出现连续 /
- 编码长度最大为850个字节
- 推荐使用大小写英文字母、数字，即[a-z, A-Z, 0-9]和符号 -, !, _, ., * 及其组合
- 不支持 ASCII 控制字符中的字符上(↑)，字符下(↓)，字符右(→)，字符左(←)，分别对应 CAN(24)，EM(25)，SUB(26)，ESC(27)
- 如果用户上传的文件或文件夹的名字带有中文，在访问和请求这个文件或文件夹时，中文部分将按照 URL Encode 规则转化为百分号编码。
- 不建议使用的特殊字符: ` ^ " \ { } [] ~ % # \ > < 及 ASCII 128-255 十进制
- 可能需特殊处理后再使用的特殊字符: , ; = & \$ @ + ? (空格) 及ASCII 字符范围: 00-1F 十六进制 (0-31 十进制) 以及 7F (127 十进制)

上传图片或视频

上传图片或视频的逻辑实现：

- 拍摄或从手机相册中选择图片或视频 `wx.chooseMedia`
- 上传图片或视频到云端存储 `wx.cloud.uploadFile`

```
1  /* 选择并上传图片或视频 */
2  chooseMedia() {
3      // 1. 拍摄或从手机相册中选择图片或视频。
4      wx.chooseMedia({
5          count: 1, // 最多可以选择的文件个数
6          mediaType: ['image', 'video'], // 文件类型: image图片、video视频、mix 可同时选择图片
和 视频
7          sourceType: ['album', 'camera'], //选择的来源: album从相册选择、camera使用相机拍摄
8          maxDuration: 60, // 拍摄视频最长拍摄时间, 单位秒。时间范围为 3s 至 60s 之间。不限制相
册
9          camera: 'back', // 默认使用front前置或back后置摄像头, 仅在 sourceType 为 camera 时
生效
10         success: (res) => {
11             // 本地临时存储信息
12             console.log(res.tempFiles[0])
13
14             // fileType值为 'image' 或 'video'
15             const {tempFilePath, fileType} = res.tempFiles[0]
16             // 上传到云端
17             this.uploadFile(tempFilePath, fileType)
18         }
19     })
20 },
21
22 // 上传文件到云端存储
23 uploadFile(tempFilePath, fileType='image') {
24     // 云端的文件路径, 不能以 / 开头, 且 / 不能连续出现, 推荐使用大小写英文字母、数字组合
```

```
25 // ios真机上传有 tmp_ 前缀, 如果有把它替换掉
26 const cloudPath = fileType +
tempFilePath.substring(tempFilePath.lastIndexOf('/') + 1).replace('tmp_', '')
27 // console.log('cloudPath', cloudPath)
28
29 // 开始上传
30 wx.cloud.uploadFile({
31   cloudPath, // 上传至云端的路径, 如: 'example.png'
32   filePath: tempFilePath, // 小程序临时文件路径
33   success: res => {
34     // 返回文件 ID
35     console.log(res.fileID)
36     this.setData({
37       fileType, //fileType: fileType
38       fileUrl: res.fileID
39     })
40   },
41   fail: (err) => {
42     wx.showToast({title: '上传失败', icon: 'none'})
43   }
44 })
45 },
```

上传成功后会获得文件唯一标识符, 即文件 ID, 后续操作都基于文件 ID 而不是 URL。

页面渲染代码:

```
1 <button bind:tap="chooseMedia" type="primary">上传图片或视频</button>
2
3 <video wx:if="{{fileType == 'video'}}" src="{{ fileUrl }}" style="width: 100%;"/>
4 <image wx:else src="{{ fileUrl }}" style="width: 100%;"/>
```

上传文档(word/ppt/pdf等)

上传文档的逻辑实现:

- 选择微信会话中（聊天记录中）的文件 `wx.chooseMessageFile`
- 上传文档到云端存储 `wx.cloud.uploadFile`

```
1 // 选择微信会话中的文件, 并上传到云端
2 chooseMessageFile() {
3   // 从微信会话中选择文件
4   wx.chooseMessageFile({
5     count: 1,
6     type: 'file',
```

```
7      extension: ['doc', '.docx', 'ppt', 'pptx', 'pdf'], // 限制选择的后缀名，仅
      type==file 时有效
8      success: (res) => { // 箭头函数
9          const {path, type} = res.tempFiles[0]
10         console.log('选择会话中的文件res', res, path, type)
11         // 上传到云端
12         this.uploadFile(path, type)
13     }
14 })
15 },
```

页面模板代码

```
1 <button bind:tap="chooseMessageFile" type="warn">上传文件</button>
```

下载云端图片/视频并保存到相册

可以根据文件 ID 下载文件，用户仅可下载其有访问权限的文件：

- `wx.cloud.downloadFile` 下载云端文件到本地，返回一个本地临时地址
- `wx.saveImageToPhotosAlbum` 保存图片到系统相册
- `wx.saveVideoToPhotosAlbum` 保存视频到系统相册

```
1 // 下载云端图片或视频，并保存到系统相册
2 async downloadMedia() {
3     const { fileUrl } = this.data
4     if (!fileUrl) return
5
6     // 下载
7     wx.cloud.downloadFile({
8         fileId: fileUrl.trim(), // 指定云端文件id, .trim()去除左右空格
9         success: (res) => {
10             // 返回临时文件路径
11             console.log('下载文件res', res)
12             const {header, tempFilePath} = res;
13             // 通过header获取下载的文件类型
14             // 模拟器是 Content-Type: "image/jpeg", IOS真机是 Content-Type:
15             ["image/jpeg"] 取数组第1个元素
16             let contentType = header['Content-Type']
17             if (contentType instanceof Array) contentType = contentType[0]
18             contentType = contentType.split('/')[0] // 如 "image/jpeg" 按/分隔为
19             [image, jpeg], 然后[0]为image
20             // console.log('contentType', contentType)
21             // 展示对应媒体组件video、image
```

```
20         this.setData({fileType: contentType})
21
22         // 保存图片或视频到系统相册
23         this.saveToPhotosAlbum(tempFilePath, contentType)
24     },
25     fail: (error) => {
26         wx.showToast({title: '下载失败', icon: 'error'})
27     }
28 })
29 },
30
31 // 保存图片或视频到系统相册
32 async saveToPhotosAlbum(tempFilePath, fileType) {
33     try {
34         if (fileType == 'image') {
35             // 保存图片到系统相册
36             await wx.saveImageToPhotosAlbum({filePath: tempFilePath})
37         } else if (fileType == 'video') {
38             // 保存视频到系统相册
39             await wx.saveVideoToPhotosAlbum({filePath: tempFilePath})
40         } else {
41             wx.showToast({title: '不支持保存到相册', icon: 'error'})
42             return
43         }
44         wx.showToast({title: '保存成功'})
45     } catch (error) {
46         wx.showToast({title: '保存失败', icon: 'error'})
47     }
48 }
```

页面模板代码

```
1  <!--
2      输入框输入FileID后，渲染时报错：
3      Failed to load local image resource /pages/cloud-storage/cloud://mxg-order-
miniapp-8
4      原因：input输入框默认最多输入140个字符，而输入的文件ID值超过了140个，被截取掉了，从而地址就
是错误的，所以导致渲染失败
5      解决：使用 maxLength="-1" 不限制输入的字符长度
6  -->
7  <input model:value="{{fileUrl}}" maxLength="-1"
8      placeholder="请输入fileID" style="border: 1px solid;"/>
9  <button type="primary" bind:tap="downloadMedia">
10      下载图片或视频保存到相册中
11  </button>
```

下载云端文档并预览

可以根据文件 ID 下载文件，用户仅可下载其有访问权限的文件：

- `wx.cloud.downloadFile` 下载云端文件到本地，返回一个本地临时地址
- `wx.openDocument` 新开页面打开文档预览

```
1 // 下载并预览文档
2 async downloadDocument() {
3   console.log('文档下载链接', this.data.fileUrl)
4   const { fileUrl } = this.data
5   if (!fileUrl) {
6     wx.showToast({title: '链接不能为空', icon: 'error'})
7     return
8   }
9   // 下载文件
10  try {
11    const res = await wx.cloud.downloadFile({
12      // 文件 ID: 防止有空格，去除空格
13      fileId: fileUrl.trim(),
14    })
15    // 返回临时文件路径
16    console.log('文档下载res', res)
17    const { tempFilePath } = res;
18
19    // 新开页面打开文档预览
20    wx.openDocument({
21      filePath: tempFilePath, // 本地临时路径
22      showMenu: true, // 显示右上角菜单
23    })
24  } catch (error) {
25    wx.showToast({title: '文件下载失败', icon: 'error'})
26    console.error('文件下载失败', error)
27  }
28 },
```

页面模板代码

```
1 <button type="primary" bind:tap="downloadDocument">
2   下载并预览文档
3 </button>
```

删除云端文件

可以通过 `wx.cloud.deleteFile` 删除云端存储的文件：


```
1 deleteFile (e) {  
2   const { fileId } = e.currentTarget.dataset  
3   if (!fileId) return  
4   // 删除云端文件  
5   wx.cloud.deleteFile({  
6     fileList: [fileId],  
7     success: res => {  
8       wx.showToast({title: '删除成功'})  
9     },  
10    fail: err => {  
11      wx.showToast({title: '删除失败'})  
12    }  
13  })  
14 },
```

页面模板代码

```
1 <button bind:tap="deleteFile" data-file-id="{{fileUrl}}">删除云端文件</button>
```

第十五章 内容管理平台（CMS）

介绍

官网参考：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/extensions/cms/introduction.html>

内容管理：基于云开发搭建的可视化的内容管理平台（CMS），提供了丰富的内容管理功能，开通简单，无需编写代码即可使用，支持文本、富文本、Markdown、图片、文件、关联类型等多种类型的可视化编辑，易于二次开发，并与云开发的生态体系紧密结合，助力开发者提升开发效率。

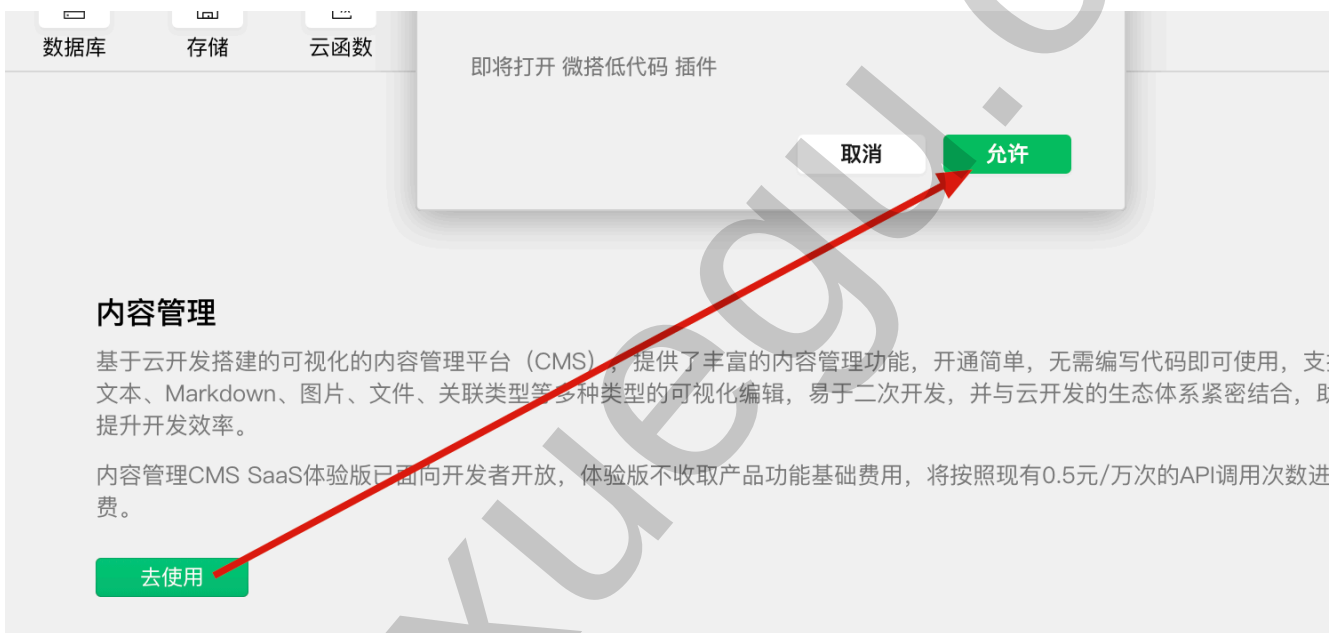
功能特性	介绍
免开发	基于模板配置生成内容管理界面，无须编写代码
功能丰富	支持文本、图片、文件、枚举等多种类型内容的可视化编辑，并且支持配置运营工具
系统集成	支持 Webhook 触发，可以方便的与外部系统集成
数据源兼容	支持管理已有的云开发数据，也可以在 CMS 后台创建新的内容和数据集合
部署简单	可在云开发控制台扩展管理界面一键部署和升级，也可通过项目提供的脚本自动部署

开通内容管理

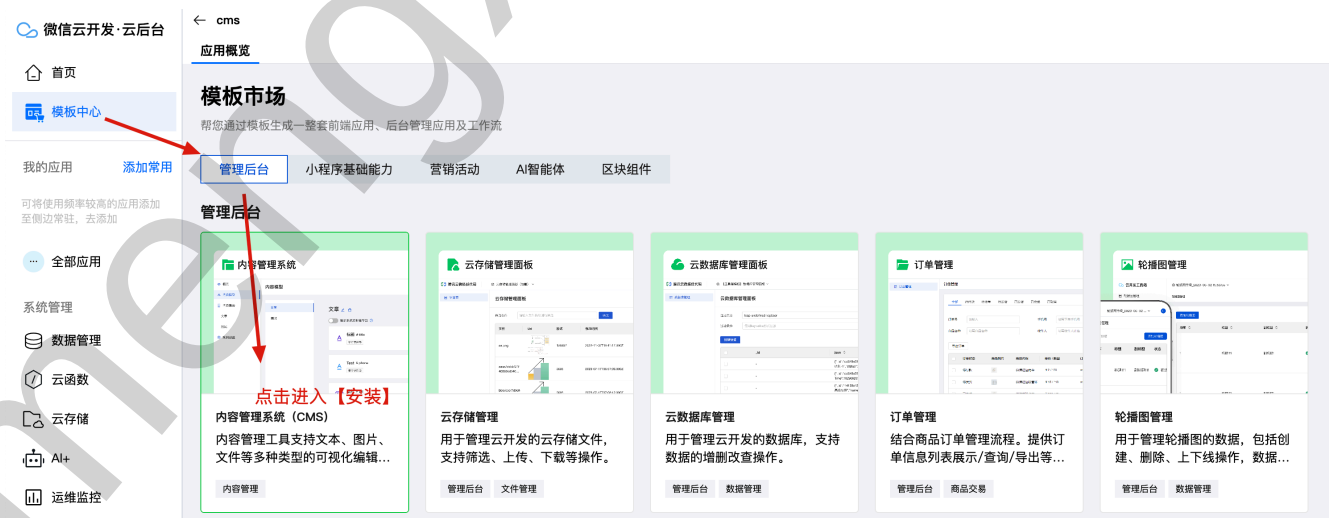
1. 打开云开发控制台，工具顶部 Tab 栏中，如下图点击「更多」-「内容管理」



2. 点击「去使用」，然后弹窗中点击「允许」，即可开通或前往 CMS 管理系统



3. 点击【模板中心】菜单，右侧找到【内容管理系统】点击进入



4. 如果出现如下弹窗，点击【继续使用云开发CMS】



5. 在云模板详情中点击右上角的【安装】，需要等待一会安装完成



6. 安装生成cms应用后，点击【查看详情】，再点击【打开管理端】



内容管理系统 (CMS)



内容管理系统 (CMS)

内容管理系统 (CMS) 是云开发·云模板推出的Headless内容管理平台，提供开箱即用的内容管理功能。该模板基于模型配置，动态生成内容管理界面，无须编写代码即可使用，快速管理业务数据。支持字符串、数字、多媒体、图片、文件、富文本、Markdown、关联类型等数十种内容类型的可视化编辑。

用户及权限管理

打开管理端

目录

功能特性

快速上手

第 1 步：安装云模板

第 2 步：进入应用详情页操作

使用场景

1. 为小程序应用增加一个运营管理

后台

2. 快速开发内容型的网站应用、小

程序应用

常见问题

新版CMS是否支持将内容数据存

至云开发环境数据库内？

创建项目

在微信云开发·云后台中，点击【全部应用】》【内容管理系统】

微信云开发·云后台

首页

模板中心

我的应用

添加常用

可将使用频率较高的应用添加
至侧边常驻，去添加

全部应用

系统管理

数据管理

内容管理

内容管理系统 (CMS)

+ 新建应用

点击【创建新项目】

我的项目



创建新项目

体验云开发全

基于云开发数据，

前往使用

创建项目

X

* 项目名

MxgOrderCms

11 / 15

* 显示名

梦学谷点餐小程序管理

10 / 15

项目介绍

项目介绍，如官网内容管理

0 / 30

取消

创建

内容模型管理（集合结构管理）

创建内容模型（创建集合）

点击菜单【内容模型】，然后点击右上角【+新建模型】选择【新建CMS模型】

概览

内容模型

品 内容模型

+ 新建模型

导出模型

导入模型

+ 新建模型

导出模型

导入模型

新建云开发数据模型(推荐)

支持表格视图管理、类型校验、关联关系、自动解析、云开发SDK访问

新建CMS模型

CMS模型，仅支持数据管理

A

单行字符串

在创建模型弹窗中，如下图输入相关信息（表信息）

模型数据是否存储至云开发环境数据库：

开启后，数据会存储在云开发环境的数据库，无法通过OpenAPI操作数据（推荐）

关闭后，数据会存储在CMS中心化数据库，可以通过OpenAPI操作数据

创建模型

×

* 模型名称

goods

5 / 15

* 展示名称

商品信息

4 / 15

描述信息

描述信息，会展示在对应内容的管理页面顶部，可用于内容提示，支持 HTML 片段

0 / 200

模型数据是否存储至云开发环境数据库



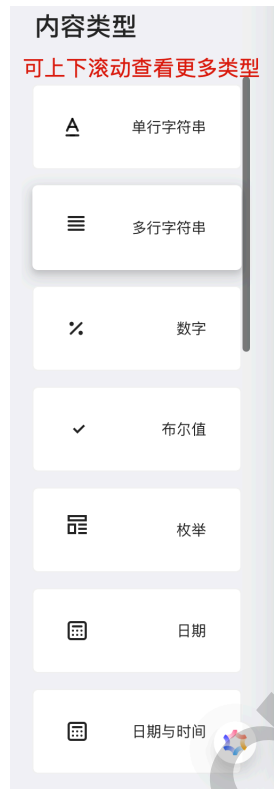
如果关闭，数据存储在CMS中心化数据库，可通过OpenAPI获取数据。

如果开启，模型数据则存储在云开发环境的数据库，且无法通过OpenAPI获取数据。

取消

创建

创建内容类型（字段数据类型）



支持的内容类型（字段数据类型）

单行字符串

添加一个 **payOrderId** 支付渠道订单号 字符串类型，限制最大长度为100

添加【单行字符串】字段 ×

* 数据库字段名

1 payOrderId 10 / 15

* 展示名称

2 支付渠道订单号 7 / 15

描述

3 如：微信支付订单号 9 / 200

默认值

此值的默认值

最小长度

最大长度

最小长度，如 1 4 100

是否必需

☐

在创建内容时，此段是必需填写的

多行字符串

数字

添加一个 **totalAmount** 订单总金额 数字类型，默认值0，最小值0

添加【数字】字段 ×

* 数据库字段名

1 totalAmount 11 / 15

* 展示名称

2 订单总金额 5 / 15

描述

字段描述，如博客文章标题
0 / 200

默认值

3 0

最小值

4 0

最大值

最大值，如 1000

是否必需

☒

在创建内容时，此段是必需填写的

布尔值

枚举

添加一个 **payState**支付状态 枚举类型：1-等待支付，2-支付成功，3-支付失败，4-支付超时

添加【枚举】字段

×

* 数据库字段名

1 payState 8 / 15

* 展示名称

2 支付状态 4 / 15

描述

3 1-等待支付，2-支付成功，3-支付失败，4-支付超时 27 / 200

默认值

4 1

枚举元素类型

5 数字

枚举元素

6 等待支付 1 支付成功 2 支付失败 3 支付超时 4

+ 添加枚举元素

是否必需

7

在创建内容时，此段是必需填写的

日期

日期与时间

添加一个 **payEndTime**支付完成时间 日期时间类型，时间存储格式选择【时间字符串】，如：20080820192859

添加【日期与时间】字段

X

包含日期和时间，如 2020-09-01 10:11:07

* 数据库字段名

1 payEndTime

10 / 15

* 展示名称

2 支付完成时间

6 / 15

描述

字段描述，如博客文章标题

0 / 200

默认值

请选择日期



时间存储格式

3 时间字符串



- 文件
- 图片
- 多媒体
- 邮箱地址
- 电话号码
- 网址
- 富文本
- Markdown
- 关联

数组

添加一个数组类型的 **goodsDetails** 订单商品明细 字段，存放订单中的所有商品信息

添加【数组】字段

X

* 数据库字段名

1 goodsDetails

12 / 15

* 展示名称

2 订单商品明细

6 / 15

描述

字段描述，如博客文章标题

0 / 200

是否必需



在创建内容时，此段是必需填写的

- JSON 对象

内容集合（增修改查记录）

内容集合模块，主要是使用页面化管理集合（表）中的数据，也就是对记录进行增修改查。

商品信息

导出到云开发数据模型 增加检索 + 新建 导入数据 导出数据

☐	文档 ID	商品名称	商品主图	内容	markdown内容	操作
☐	QtLdqyTuLDe7QZCy	鸡腿汉堡		<p>欢迎使用富文本编...	-	编辑

点击图片 访问链接